

Teamcenter 10.1

Application Administration Guide

Proprietary and restricted rights notice

This software and related documentation are proprietary to Siemens Product Lifecycle Management Software Inc.

© 2013 Siemens Product Lifecycle Management Software Inc. All Rights Reserved.

Siemens and the Siemens logo are registered trademarks of Siemens AG. Teamcenter is a trademark or registered trademark of Siemens Product Lifecycle Management Software Inc. or its subsidiaries in the United States and in other countries. All other trademarks, registered trademarks, or service marks belong to their respective holders.

Contents

Proprietary and restricted rights notice	2
Getting started with administering applications	1-1
Administering Teamcenter-wide functionality	1-1
Administering application-specific functionality	1-2
Configuring Security Services for Teamcenter	2-1
Configuring Security Services for Teamcenter	2-1
Configuring the rich client	2-1
Configuring the thin client	2-4
Configuring other Teamcenter clients	2-5
Configuring menu and toolbar commands	3-1
Configuring menu and toolbar commands	3-1
Command Suppression interface	3-2
Using Command Suppression	3-3
Configuring access to administrative applications and utilities	4-1
Configuring access to applications and utilities	4-1
Authorization interface	4-2
Basic concepts of using Authorization	4-2
Configuring access to Teamcenter administrative applications	4-3
Configuring access to Teamcenter utilities	4-4
Sharing authorization rules with other Teamcenter sites	4-5
Establishing security and access rules for your data	5-1
Establishing security and access rules for your data	5-1
Administering preferences	6-1
Administering preferences	6-1
Configuring search	7-1
Configuring search	7-1
Implementing quick search	7-4
Implementing advanced search	7-5
Configuring full-text search	7-8
Implementing simple search	7-19
Configuring Smart Codes in Teamcenter Rapid Start	8-1
Configuring Smart Codes in Teamcenter Rapid Start	8-1
Smart Codes example	8-1
Enable Smart Codes	8-3

Editing the Smart Codes configuration file	8-3
Import and export configuration files	8-22
Defining queries and reports	9-1
Defining queries and reports	9-1
Configuring Teamcenter mail	10-1
Configuring Teamcenter mail	10-1
Configure Teamcenter to work with external e-mail for subscription notices ...	10-1
Configuring instant messaging	11-1
Configuring instant messaging	11-1
Configuring standard notes, custom notes, and parametric requirements	12-1
Configuring standard notes, custom notes, and parametric requirements	12-1
Configure standard note and parametric requirement creation privileges	12-1
Set the delimiter that separates parameters in parametric requirement text ...	12-2
Enable multiple revisions of a parametric requirement to be attached to an item or item revision	12-2
Configure the behavior of custom notes	12-2
Configuring document management	13-1
Configuring document management	13-1
Working with document management templates	13-1
Creating item revision definition configurations (IRDCs)	13-3
Creating dispatcher service configurations	13-4
Configuring UNIX printers for batch printing	13-4
Configuring Microsoft Office templates	14-1
Configuring Microsoft Office templates	14-1
Microsoft Office Word export templates	14-1
Microsoft Office Excel export templates	14-14
Configuring Web Browser for your site	15-1
Configuring Web Browser for your site	15-1
Configuring Lifecycle Visualization	16-1
Configuring Lifecycle Visualization	16-1
Modifying Lifecycle Visualization configuration files	16-1
Specify visualization licensing options	16-1
Rich client embedded viewer compatibility	16-2
Viewing data from multiple Teamcenter servers	16-3
Multiple File Management System (FMS) client caches (FCCs)	16-4
How group and role changes affect viewer behavior	16-5
Visualization preferences	16-6
Search schemas for the Lifecycle Viewer	16-7
Creating text and watermark stamps using an MDS file	16-7
Specify new types of visualization datasets	16-20
Specify dataset types with multiple named references	16-20

Configure automatic markups	16-21
Configure the Image Preview view	16-22
Configure the JT Preview	16-23
Configure image preview in Relation Browser	16-24
Configure image preview in the Summary view	16-24
Configuring Teamcenter Integration for NX	17-1
Configuring Teamcenter Integration for NX	17-1
Configuring attribute mapping	17-1
Glossary	A-1
Index	Index-1

Chapter

1 Getting started with administering applications

Administering Teamcenter-wide functionality	1-1
Administering application-specific functionality	1-2

Chapter

1 *Getting started with administering applications*

Administering Teamcenter-wide functionality

Administering Teamcenter requires the administration of both:

- Teamcenter-wide functionality that operates across multiple Teamcenter applications, such as search, security rules, and document management
- Application-specific functionality, such as Access Manager, Business Modeler IDE, Workflow Designer, and many other applications

This guide documents the administrative tasks required for Teamcenter-wide functionality.

Tip Help for specific applications is provided in individual guides.

For more information about these individual guides, see [Administering application-specific functionality](#).

This guide provides documentation for the following administrative tasks:

- [Configuring password access to Teamcenter using Security Services](#)
- [Configuring menu and toolbar commands using command suppression](#)
- [Configuring access to administrative applications and utilities using the Authorization application](#)
- [Establishing security and access rules for your data](#)
- [Configuring the different types of search mechanisms](#)
- [Defining customized queries and building reports](#)
- [Configuring Teamcenter mail and/or configuring external e-mail systems to work with Teamcenter](#)
- [Configuring instant messaging](#)
- [Configuring access to, and display of, standard notes, custom notes, and parametric requirements](#)

- [Configuring document management to control Teamcenter behavior with documents throughout their life cycle, such as item creation, checkin, checkout, deletion and revision](#)
- [Configuring Microsoft Office export templates](#)
- [Specifying a default home page for Web Browser](#)
- [Configuring Lifecycle Visualization, including modifying configuration files, specifying licensing options, creating text and watermark stamps, specifying new dataset types, configuring JT and image thumbnail preview, and configuring automatic markup](#)
- [Configuring attribute mapping for Teamcenter Integration for NX.](#)

Administering application-specific functionality

The following applications are used to perform administrative tasks in the Teamcenter rich client.

For more information about each task, see the application-specific documentation.

Teamcenter application-specific documentation	Tasks
Access Manager Guide	Define and maintain rules and access control lists.
Authorized Data Access License Guide	Administer authorized data access licenses.
Audit Manager Guide	Configure audit logs that track actions that take place on specified objects in the database.
Business Modeler IDE Guide	Create data model objects, such as business objects, properties, lists of values, and naming rules.
Classification Administration Guide	Create and manage the classification hierarchy and attribute dictionary.
Content Management Guide	Set up authoring and publishing tools, including creating and maintaining topic types, publication types, style sheets, and style types.
My Teamcenter Guide	View components, select objects, navigate hierarchies, work with summaries, details, impact analyses, and related information.
Organization Guide	Define your company's virtual organization, manage storage volumes, and define calendars.
PLM XML/TC XML Export Import Administration Guide	Create transfer mode objects that help users import and export Teamcenter objects and system data.

**Teamcenter
application-specific
documentation**

Tasks

Project and Program Guide

Set up projects, including assigning users and implementing access control based on project membership.

Query Builder Guide

Define customized searches for objects in local and remote Teamcenter databases.

Report Builder Guide

Create and manage report definitions.

Setup Wizard Guide

Create your company's virtual organization using an input file, such as a file generated from Microsoft Exchange or the UNIX **etc/passwd** file.

Subscription Monitor Guide

Configure Subscription Monitor process daemons, manage subscription tables, and create and manage subscription reports.

Validation Manager Guide

Configure validation processes that provide a decision mechanism by which users ensure that item revisions meet compliance before they are released.

Volume Management Guide

Define and maintain volume management and hierarchical storage management policies.

Workflow Designer Guide

Design workflow process templates that incorporate your company's business practices and procedures.

Chapter

2 *Configuring Security Services for Teamcenter*

Configuring Security Services for Teamcenter	2-1
Configuring the rich client	2-1
Configuring the thin client	2-4
Configuring other Teamcenter clients	2-5

Chapter

2 *Configuring Security Services for Teamcenter*

Configuring Security Services for Teamcenter

When Security Services is enabled, passwords are managed by the external identity service provider, rather than by Teamcenter. All other user administration tasks remain within Teamcenter.

Security Services is optional. The services must be installed and an external identity service provider must be configured.

For more information about installing Security Services, see [Security Services Installation / Customization](#).

Configuring the rich client

After Security Services is installed and the external identity service provider configured, configure Security Services or Teamcenter by manually entering the Security Services environment variables in the **tc_profilevars** properties file and the rich client properties in the **site_specific.properties** file.

Enable Security Services for the rich client by setting the **TC_SSO_SERVICE** and **TC_SSO_LOGIN_URL** environment variables.

Note In the rich client and ITK, the ability to set or change a password in the Teamcenter database is disabled when the **TC_SSO_SERVICE** environment variable is set. The **Change Password** command remains available from the **Actions** menu. In the Organization application, the password text box and blank password check box are disabled.

For administrative purposes, the administrator can choose not to set the **TC_SSO_LOGIN_URL** environment variable. The following table lists the logon scenarios for users logging on to the Teamcenter rich client, before logging on to any other Teamcenter product:

Environment variables set

No Security Services environment variables are set.

TC_SSO_SERVICE is set in the **tc_profilevars** properties file.

TC_SSO_LOGIN_URL is set in the **client_specific.properties** file (two-tier installation). For a four-tier installation, it is in the **site_specific.properties** file.

Logon scenario

Security Services is not enabled. User authentication is performed by Teamcenter:

- The rich client displays its standard logon dialog box; users enter the user IDs and passwords defined in Teamcenter.
- To access other Teamcenter products, the rich client user must log on again.

Security Services is enabled for the rich client and server. User authentication is performed by external identity service provider:

- Security Services displays a browser logon page and starts a Security Services user session. Users enter user IDs defined in the Teamcenter database, but passwords are defined in the external identity service provider.

If applet-free mode is configured, the sign-on process is performed without a Web browser and session applet. This mode requires HTTP 401-based authentication.

If zero sign-on is configured using Kerberos authentication, no Teamcenter logon challenges appear. Zero sign-on in applet free mode requires HTTP 401-negotiate authentication.

For information about configuring Kerberos authentication, see [Security Services Installation / Customization](#).

For information about the results of different logon configurations, see [Getting Started with Teamcenter](#).

- The logon uses default values for Teamcenter group and database. Users can change the group default using the **User Settings** dialog box; the default database can be changed using the

Environment variables set	Logon scenario
TC_SSO_SERVICE is set in the tc_profilevars properties file.	IIOP_SERVER_n.HOST property or the HTTP_SERVER_n.URI property in the rich client site_specific.properties file.
	Users can access other Teamcenter products that implement Security Services without logging on again during a Security Services user session.
	Security Services is enabled for the rich client server only. User authentication is performed by the external identity service provider: - Security Services does not start a user Security Services session; the rich client displays its standard logon dialog box, but users enter passwords defined in the external identity service provider. - To access other Teamcenter products, user must log on again. A Security Services session is essential for logging on to other products using authentication by the external identity service provider. This configuration is for the following special uses: - Administrators can start multiple sessions as different user IDs to act on behalf of several users. - Users can log on using a group or database other than the default values.

When one external identity provider serves multiple Teamcenter sites, and each site authorizes a different set of users, the administrator must set the **TC_SSO_APP_ID** environment variable to identify the site. This site ID must match a site ID registered with the Security Services service. This environment variable is optional; when used, it must be set in both the **tc_profilevars** file for the server and the **site_specific.properties** file for the rich client client.

When Security Services is installed and enabled, the rich client user logon defaults to the Teamcenter database, behaving as if users had entered user IDs and passwords in its standard logon dialog box but left the database blank. The default database is the database defined by either the **IIOP_SERVER_n.HOST** property (for

two-tier rich client server configurations) or the **HTTP_SERVER_n.URI** property (for four-tier rich client configurations) in the **client_specific.properties** file. These properties are set using Teamcenter Environment Manager: the IIOP-based protocol enables CORBA based communications and is implemented for a two-tier configuration, the HTTP-based protocol enables SOAP-based HTTP communication and is implemented for a four-tier configuration. To specify a different default database for Security Services logons, you must manually modify the appropriate property in the rich client **site_specific.properties** file.

Configuring the thin client

To enable Security Services for Teamcenter thin client, the **TC_SSO_SERVICE** and **TC_SSO_LOGIN_PAGE** environment variables must be set.

Note In the thin client, the ability to set or change a password is disabled when the **TC_SSO_SERVICE** environment variable is set unless the **TC_SSO_CHANGE_PASSWORD_PAGE** environment variable is also set. When the **TC_SSO_CHANGE_PASSWORD_PAGE** environment variable is set, a request to change a password is redirected from Teamcenter to the Change Password page of the external identity service provider.

The following table lists the logon scenarios for users logging on to Teamcenter thin client (before logging on to any other Teamcenter product):

Environment variables set	Logon scenario
No Security Services environment variables are set.	Security Services is not enabled. User authentication is performed by Teamcenter: • The thin client displays its standard logon page; users enter the user ID/password defined in the Teamcenter database. • To access other Teamcenter products, thin client users must log on again.
TC_SSO_SERVICE and TC_SSO_LOGIN_PAGE are set in the tc_profilevars file.	Security Services is enabled for the thin client. User authentication is performed by external identity service provider: • Security Services displays the browser logon page and starts a Security Services session; user enters password defined in external identity service provider and user ID defined in Teamcenter. • Users can access other Teamcenter products that implement Security Services without logging on again during a Security Services-defined user session.

When registering the thin client with Security Services (while configuring Security Services components) the **Application Root URL** value corresponding to the thin client should be of the form: *http://host:portnumber*. In particular, it should not include the thin client context root.

For more information about configuring Security Services, see [Security Services Installation/Customization](#).

Configuring other Teamcenter clients

To enable Security Services for all other Teamcenter clients, such as command line utilities, and ODBC Driver, only the **TC_SSO_SERVICE** environment variable must be set. When this environment variable is set, user authentication is performed by the external identity service provider (the user must enter the password defined in the external identity service provider); otherwise, the logon is unchanged.

Chapter

3 *Configuring menu and toolbar commands*

Configuring menu and toolbar commands	3-1
Command Suppression interface	3-2
Using Command Suppression	3-3
Suppress commands	3-3
Display commands	3-4
Suppress commands for roles	3-5
Display commands for roles	3-6
Customize Command Suppression	3-6

Chapter

3 *Configuring menu and toolbar commands*

Configuring menu and toolbar commands

The Command Suppression application enables you to control the display of menu and toolbar commands within Teamcenter applications.

The display of menu commands can be suppressed for specific groups or roles, allowing you to control who can perform various tasks within Teamcenter. For example, if you want only managers to import and export data, you can suppress the **Import** and **Export** commands on the **Tools** menu for all roles except the **Manager** role. You can:

- Suppress the display of entire menus.
- Suppress the display of commands for an entire group hierarchy.
- Suppress the display of commands for an entire group.
- Suppress the display of commands for users who are assigned a specific role within a group.
- Suppress the display of specific commands on a designated menu.

To suppress or display commands of Teamcenter applications, see [Suppress commands](#) and [Display commands](#).

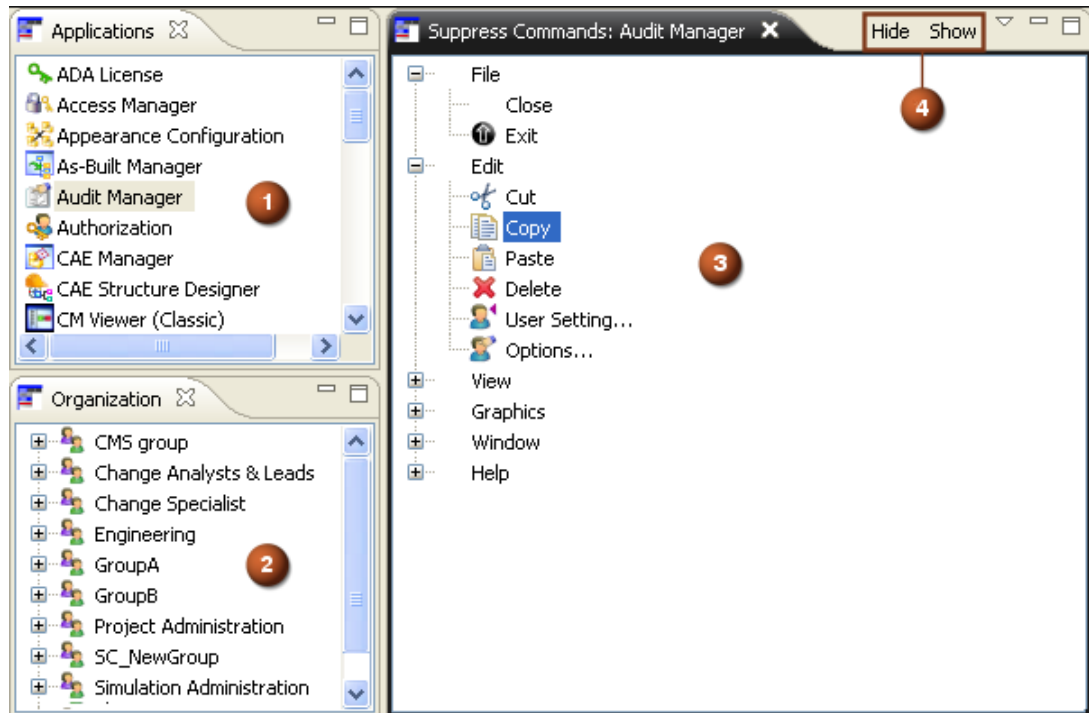
To suppress the display of commands for a role across groups, see [Suppress commands for roles](#) and [Display commands for roles](#).

Note The Command Suppression application can be used to suppress commands in both the rich client interface and the thin client interface. However, thin client menu commands can also be suppressed by a preference. Menu commands can be listed as values for the **WEB_menu_entry_suppressions** preference, which overrides the configuration specified by the Command Suppression application.

If the value of the preference is null, the configuration specified by the Command Suppression application is applied to the thin client interface.

For more information about using this preference to suppress thin client commands, see the [Thin Client Interface Guide](#).

Command Suppression interface



- 1 **Applications** tree pane Displays applications with menu commands you want to suppress. Select an application from the tree to display the application commands in the **Suppress Commands** tree.
- 2 **Organization** tree pane Displays the groups and roles you can select when suppressing application commands.
- 3 **Suppress Commands** tree pane Displays the menus, submenus, and commands associated with the application selected in the **Applications** tree.
 - Menu nodes correspond to the menu bar displayed when the selected application is loaded (for example, **Edit**, **View**, and **Tools**). Expand the menu node, by clicking the plus symbol , to display the submenus and commands.
 - Submenu nodes correspond to the submenus displayed when the menu is accessed in the loaded application (for example, **New** and **Save As**). Expand the submenu node, by clicking the plus symbol , to display the submenus and commands.
 - Command nodes represents the commands associated with the selected application and are the lowest level in the command

tree hierarchy (for example, **Item** and **Process**).

In addition to suppressing menus, submenus, and commands for the rich client, you can also suppress menus, submenus, and commands for the thin client by expanding the **Thin Client** branch.

Note Thin client menu suppression can also be implemented by using the **WEB_menu_entrySuppressions** preference. The preference method *cannot* be combined with the Command Suppression application.

For more information about using the preference method, see the [Thin Client Interface Guide](#).

4 **Hide** and **Show** buttons

Enable you to either suppress application commands or reverse the suppression of a command.

- By selecting commands from the command tree, clicking **Hide**, and saving the change, the command is no longer visible when a member of the group/role accesses the application.
- By selecting a suppressed (hidden) command from the command tree (indicated by a red line through the command node), clicking **Show**, and saving the change, the command is visible when a member of the group/role accesses the application.

Using Command Suppression

Suppress commands

1. Select the application from the **Applications** tree containing the commands that you want to suppress.
2. Select a group, subgroup, or role from the **Organization** tree.
3. Select a menu, submenu, or command from the **Suppress Commands** tree.
4. Click **Hide**.

A red line appears through the menu, submenu, and/or command nodes selected for suppression  **Open** .

5. Choose **File**→**Close**.

You are prompted to save the changes.

Key points

- Command suppression is inherited within the group hierarchy. Applying command suppression to a parent group or role within a parent group applies the suppression to any and all subgroups or roles associated with the parent group.
- To change command suppression in another application, select the application in the **Applications** tree and perform the previous steps.
- When you use Command Suppression to suppress the **Tools** menu in My Teamcenter, the **Export** menu command is shown as available on the **Tools** menu. However, you cannot make any selections from the **Tools** menu.

- Thin client menu commands can also be suppressed based on preference settings. Menu commands can be listed as values for the [WEB_menu_entrySuppressions](#) preference, which overrides the configuration specified by the Command Suppression application.

If the value of the preference is null, the configuration specified by the Command Suppression application is applied to the thin client interface.

For more information about using this preference to suppress thin client commands, see the [Thin Client Interface Guide](#).

For more information about using this preference, see the [Preferences and Environment Variables Reference](#).

- Command suppression is inherited within the group hierarchy. When you remove command suppression from parent groups, this causes the suppression to be removed from any and all subgroups or roles associated with the parent group. Similarly, when you suppress a new command for a parent group, this suppresses the command from any and all subgroups or roles associated with the parent group.

This inheritance does not work for a subgroup or associated role if you explicitly add or remove a command from the suppressed list for that subgroup or associated role. When the inheritance is broken, both the parent group and subgroups have independent lists of suppressed commands.

- Command Suppression cannot suppress commands that are:
 - o Contributions that are done statically in the code (for example, **Window** menu), with the exception of **savePerspective**, **resetPerspective**, and **closePerspective** in the **Window** menu.
 - o Contributions done using Eclipse actions.

Display commands

1. Select the application from the **Applications** tree containing the hidden commands that you want to display.
2. Select a group, subgroup, or role from the **Organization** tree.

3. Select a previously hidden menu, submenu, or command from the **Suppress Commands** tree.
4. Click **Show**.
The red line disappears from the menu, submenu, and/or command nodes selected for display.
5. Choose **File→Close**.
You are prompted to save the changes.

Key points

- Current suppressed commands for the group or role are designated by a red line through the node  **Open**.
- Command Suppression is inherited within the group hierarchy. Removing Command Suppression from a parent group or role within a parent group, causes the suppression to be removed from any and all subgroups or roles associated with the parent group.

Suppress commands for roles

You can suppress commands for roles without selecting the group to which it belongs.

1. In My Teamcenter, choose **Edit→Options**.
2. Click **Index**.
3. Click **New**.
4. In the **Create new preference** pane, type the following statement in the **Name** box:

```
com.teamcenter.rac.ui.perspectives.navigatorPerspective\\*\\role\\HIDDEN_COMMANDS
```
5. In the **Create new preference** pane, type a comma-separated list of actions to suppress in the **Values** box.

Note For a list of action names, open the **com.teamcenter.rac.common_1.0.0.jar** file and extract the **com\teamcenter\rac\common\actions\actions.properties** file.

6. Click **Create**.

The following list contains examples of suppressing commands:

Note Site-wide command suppressions, for example, all groups and all roles, are not supported.

- To suppress the **Cut**, **Copy**, and **Paste** actions for My Teamcenter applications for the **DBA** role only, type the following value in the **Name** box and type **cutAction,copyAction,pasteAction** in the **Values** box:

```
com.teamcenter.rac.ui.perspectives.navigatorPerspective\\*\\DBA\\HIDDEN_COMMANDS
```

- To suppress the **Cut**, **Copy**, and **Paste** actions for My Teamcenter applications for all the roles defined in the **dba** group, type the following value in the **Name** box and type **cutAction,copyAction,pasteAction** in the **Values** box:

```
com.teamcenter.rac.ui.perspectives.navigatorPerspectives\\dba\\*\\HIDDEN_COMMANDS
```

- To suppress the **Cut** action for the Structure Manager application for all groups and the **DBA** role only, type the following value in the **Name** box and type **cutAction** in the **Values** box:

```
com.teamcenter.rac.pse.PSEApplication\\*\\DBA\\HIDDEN_COMMANDS
```

For more information about using Teamcenter command IDs when suppressing commands, see the [Client Customization Programmer's Guide](#).

Display commands for roles

You can display commands for roles without selecting the group to which it belongs.

1. In My Teamcenter, choose **Edit→Options**.
2. Click **Index**.
3. Select the preference in the **Preferences list** pane.
4. Click **Delete**.

Customize Command Suppression

You can customize the Teamcenter rich client to extend the functionality of suppressing commands to include custom Teamcenter applications.

1. Copy the **TC_CS.APPLICATIONS** key and its value from the **portal.properties** file to the **portal_user.properties** file.
2. Append the path name of the new application to the value.

For information about customizing Command Suppression menus and toolbars, see the [Client Customization Programmer's Guide](#).

Chapter

4 *Configuring access to administrative applications and utilities*

Configuring access to applications and utilities	4-1
Authorization interface	4-2
Basic concepts of using Authorization	4-2
Group hierarchy support for authorization rules	4-3
System-level authorization rules	4-3
Configuring access to Teamcenter administrative applications	4-3
Configuring access to Teamcenter administrative applications	4-3
Configure access to applications by group or by role in group	4-4
Configuring access to Teamcenter utilities	4-4
Configuring access to Teamcenter utilities	4-4
Configure access to utilities by group or by role in group	4-4
Sharing authorization rules with other Teamcenter sites	4-5
Sharing authorization rules with other Teamcenter sites	4-5
Export authorization rules	4-5
Import authorization rules	4-5

Chapter

4 *Configuring access to administrative applications and utilities*

Configuring access to applications and utilities

The Authorization application enables you to control access to Teamcenter administrative applications and utilities based on users' group membership or their role in a group.

For example, you can:

- Grant all members of the **DBA Lite** group access to the Organization application, regardless of their role within the group.
- Grant users who occupy the **importer** role in the **DBA Lite** group access to the PLM XML/TC XML Export Import Administration application.

The Authorization application works in conjunction with other Teamcenter applications to control access to product features and data, as follows:

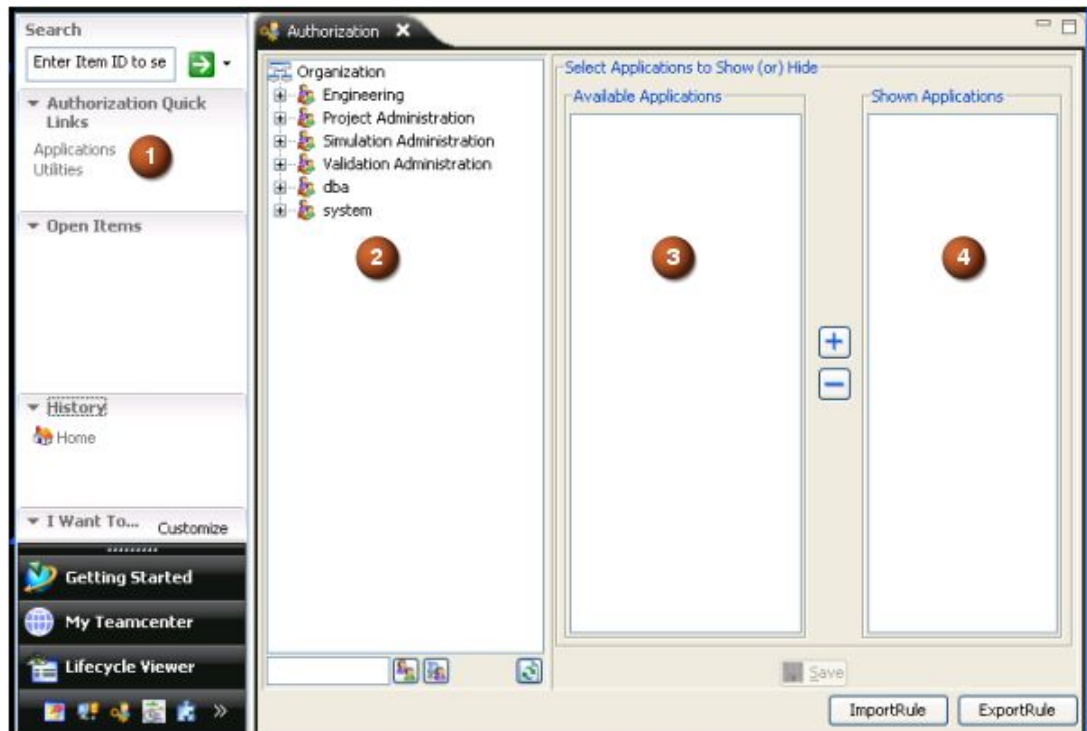
- Access to product features is controlled using the Command Suppression application.

For more information, see the [Configuring menu and toolbar commands](#).

- Access to operations on objects, such as delete, copy, and change ownership, is controlled by configuring rules in Access Manager.

For more information, see the [Security Administration Guide](#) and the [Access Manager Guide](#).

Authorization interface



- | | | |
|---|----------------------------------|--|
| 1 | Authorization Quick Links | Enables you to choose either utilities or applications for configuration. |
| 2 | Organization tree | Displays the groups and roles in your organization. You can choose which utilities or administrative applications are displayed in the interface for a selected group or for a selected role within a group. |
| 3 | Available Applications | Displays the list of administrative utilities or applications that can be shown or hidden. |
| 4 | Shown Applications | Displays the list of administrative utilities or applications that are shown in the interface for the selected group or role in group. |

Basic concepts of using Authorization

Authorization rules allow you to control access to Teamcenter administrative applications and utilities based on groups.

System-level rules are delivered as part of your Teamcenter installation. You can create additional rules to support your business processes using the Authorization application.

Group hierarchy support for authorization rules

Groups within the organization tree can be configured into one or more hierarchies. Each group has exactly one parent group (unless it is at the root of the hierarchy, when it has no parent group), and each group can have one or more child groups (subgroups).

Authorization rules are inherited within the group hierarchy, as follows:

- Rules defined for a parent group are inherited by all subgroups of the parent group.
- Rules defined at the subgroup level apply only to that subgroup.

Note In the event that two subgroups of different parentage share the same name, rules defined for one parent group are not inherited by the same-name subgroup of the other parent group. For example, if both the **Manufacturing** group and the **Design** group have a **Validation** subgroup, authorization rules defined for the **Manufacturing** group apply only to the **Validation** subgroup that is directly related to the **Manufacturing** group. Likewise, authorization rules defined for the **Design** group apply only to the **Validation** subgroup that is directly related to the **Design** group.

System-level authorization rules

System-level authorization rules are those rules delivered as part of your standard Teamcenter installation that govern access to administrative applications and utilities. By default, Teamcenter supplies two groups for administrative purposes, the **Project Administration** group and the **dba** group.

Project Administration group members only have access to the Project application, which allows them to create, delete, modify, and add users to or remove users from projects. **dba** group members are granted access to all Teamcenter administrative applications and utilities.

Often, administrative tasks are assigned at a functional level corresponding to your business practices. For example, responsibility for administering user data such as personal and organization information may be assigned to one group, while a different group may be responsible for designing workflow processes. In such cases, **dba** group privileges are more broad and powerful than is necessary or desirable. Authorization enables you to create authorization rules to model access to administrative tools to your business processes.

Configuring access to Teamcenter administrative applications

Configuring access to Teamcenter administrative applications

The following applications are supported for access configuration using Authorization:

Access Manager

ADA License

Organization

PLM XML/TC XML Export Import
Administration

Appearance Configuration	Project
Audit Manager	Setup Wizard
Authorization	Subscription Monitor
Business Modeler IDE	Workflow Designer
Classification Administration	

You can configure access to these applications by group or by role in group.

Note You can also set the **TC_authorization_mode** preference to specify whether to evaluate all the group memberships of users and their role in those groups when authorizing access to an application or to evaluate their current group logon and role in that group.

Configure access to applications by group or by role in group

1. Click the **Applications** link in the **Quick Links** section of the navigation pane.
2. Expand the **Organization** tree and click the group  or role  to whom you want to grant or deny application access.
3. Select the application that you want to grant access to from the **Available Applications** list. Click **Add**  to move the application to the **Shown Applications** list.

Tip If the **Available Applications** list is empty, click any group or role symbol in the **Organization** tree to refresh the list.

4. Click **Save**.

Configuring access to Teamcenter utilities

Configuring access to Teamcenter utilities

The following utilities are supported for access configuration using Authorization:

data_share	dsa_util	purge_invalid_subscriptions
data_sync	export_recovery	update_project_data
database_verify	fscadmin	

You can configure access to these utilities by group or by role in group.

Configure access to utilities by group or by role in group

1. Click the **Utilities** link in the **Quick Links** section of the navigation pane.
2. In the Authorization application pane, expand the **Organization** tree and click the group  or role  to whom you want to grant or deny utility access.

3. Select the utility that you want to grant access to from the **Available Utilities** list. Click **Add**  to move the utility to the **Shown Utilities** list.

Tip If the **Available Utilities** list is empty, click any group or role symbol in the **Organization** tree to refresh the list.

4. Click **Save**.

Sharing authorization rules with other Teamcenter sites

Sharing authorization rules with other Teamcenter sites

Authorization rules can be exported to an operating system directory as an XML file that can then be imported at another Teamcenter site, allowing you to synchronize authorization rules between sites that share data.

Export authorization rules

1. In the Authorization application pane, click the **exportRule** button.
2. In the **exportRule** dialog box, navigate to the directory location where you want to save the rule file.
3. Type a name for the file in the **File name** box.

Note The file is output in XML format; therefore, the file name must end in **.xml**.

4. Click the **exportRule** button.

The authorization rule file is saved in the operating system directory that you specified in step 2.

Import authorization rules

1. In the Authorization application pane, click the **importRule** button.
2. In the **importRule** dialog box, navigate to the directory containing the authorization rule file that you want to import.

Note Rule files are XML files.

3. Select the authorization rule file.
4. Click the **importRule** button.

The authorization rule file is imported in to Teamcenter.

Chapter

5 *Establishing security and access
rules for your data*

Establishing security and access rules for your data 5-1

Chapter

5 *Establishing security and access rules for your data*

Establishing security and access rules for your data

Teamcenter administrators establish rules that control the security of the data managed by Teamcenter. These rules define the actions that can be performed on a specific type of object. *Accessors* are entities created for security purposes and are based on a user's context in a given session. This context is comprised of the user, their group, role, site, and the projects to which they are assigned. Access Manager controls access to objects based on the accessor, the object on which the action is being performed, the state of the object, and other accessor-related parameters.

Typical administrative tasks related to establishing and maintaining data security include:

- Creating and managing rules.
For more information, see [Security Administration Guide](#)
- Creating and managing access control lists that specify accessors and the privileges that they have to a given object.
For more information, see [Access Manager Guide](#)
- Creating and managing workflow access control lists that control access to data that is in process.
For more information, see [Workflow Designer Guide](#)
- Setting up projects, assigning team members to the projects, and creating the associated group/project-level security rules.
For more information, see [Project and Program Guide](#)
- Defining work contexts.
For more information, see [My Teamcenter Guide](#)
- Controlling access to incremental change objects, revision rules, snapshots, and view types.
For more information, see the [Structure Manager Guide](#).
- Controlling access to classification objects.
For more information, see the [Classification Administration Guide](#).

Chapter

6 *Administering preferences*

Administering preferences	6-1
-------------------------------------	-----

Chapter

6 *Administering preferences*

Administering preferences

At Teamcenter 10, preferences are redesigned to place preference access fully under customer control. The redesign of preferences introduces a new category of preferences, named *system* preferences. Additionally, the redesign of preferences removes the concept of *scope*. Access to each preference is determined by its *protection scope*.

- The protection scope of hierarchical preferences can be easily changed by system administrators from their default shipped value to meet business needs.
- The protection scope of system preferences cannot be changed.
- The protection scope defines where Teamcenter starts the hierarchical search and controls where preference instances can be created.

For more information about how preference values can be defined at different levels depending on the defined protection scope, see the [Preferences and Environment Variables Reference](#).

When upgrading to Teamcenter 10 or later from a version prior to 10, legacy preference data must be converted to the new paradigm. Typically this conversion occurs silently during a TEM upgrade. However, if your preference data includes custom preferences stored in an XML file for import into the database you must convert the custom preferences before upgrade. This manual conversion process allows you to specify the default settings for each of your custom preferences.

A *custom* preference is any preference created by the customer that is not shipped in the Teamcenter code and is managed in an XML file.

Note Custom preferences do not include preferences you create based on shipped Teamcenter preferences.

Example Teamcenter ships with several instances of the *object-type_default_relation* preference (**ItemRevision_default_relation**, **WEB_itemrevision_default_relation**, **PhysicalPart_Company_default_relation**, and so on). If you created additional instances of this preference for additional object types, they are not considered custom preferences for the purposes of this conversion.

If your pre-Teamcenter 10 preference data includes custom preferences, you must convert the custom preferences before you upgrade to Teamcenter 10. You can convert the custom preferences using one of two methods:

- Run the **preferences_manager** utility with the **upgradexml** mode and its assorted arguments.
- Run the **upgrade_preferences_file.pl** Perl script.

Note This particular script must use the Teamcenter-specific version of Perl. You must run the script by entering:

tcperl upgrade_preferences_file.pl

The **tcperl** command instructs the system to use the Teamcenter-specific version of Perl.

Both conversion methods use the following arguments for identifying the preferences to be converted, providing the new default settings for the custom preferences, and specifying output details:

-input_file

Specifies the full path to the XML file containing the legacy preference data to be converted. You can only specify one file at a time.

-definition_information

Specifies the location of the definition information text file you must create. This file contains the desired default protection scope and environment variable capability for each custom preference listed in the input file.

The content of the definition information text file must be in the format:

preference-name;protection-scope;environment-variable-capability

The *protection-scope* and *environment-variable-capability* settings are optional. If these values are not specified, the values of the **-default_protection_scope** and **-default_env_variable_status** arguments are used.

Valid values for the *protection-scope* setting are **User**, **Role**, **Group**, **Site**, or **System**.

Valid values for the *environment-variable-capability* setting are **true** (indicating that the preference can also be set using an environment variable) or **false**.

Tip You can create a one-to-one correspondence between each preference listed in the input file and the specified default settings in the definition file. Alternatively, omitting default settings in the definition information file for any given preference listed in the input file applies the values of the **-default_protection_scope** and **-default_env_variable_status** arguments to the preference.

If the **-default_protection_scope** argument is not specified, the system assigns that preference a default protection scope of **User**.

If the **-default_env_variable_status** argument is not specified, the system assigns that preference the environment variable capability of **false**.

-separator (optional)

Specifies the separator used between each entry in the definition information file.

-default_protection_scope (optional)

Specifies the default protection scope assigned to any preferences included in the input XML file but not included in the definition information text file, or when the information supplied in the definition information file is incorrect.

Valid values are **User**, **Role**, **Group**, **Site**, or **System**.

The *protection scope* determines who has access to create or modify a preference instance.

For more information about protection scope, see the [Preferences and Environment Variables Reference](#).

-default_env_variable_status (optional)

Specifies the default environment variable capability assigned to any preferences included in the input XML file but not included in the definition information text file, or when the information supplied in the definition information file is incorrect.

This setting determines whether the preference can be used as an environment variable. If enabled (set to **true**), the system first checks for an environment variable of the same name. If it exists, its value is used.

Valid values are **true** and **false**.

For more information about enabling environment variable capability, see the [Preferences and Environment Variables Reference](#)

-correct_errors

Corrects errors encountered whenever possible. Types of errors that can be corrected include incorrect types, array status, protection scope, and category errors. Errors that must be manually corrected are indicated in the report file.

-output_file

Specifies the full path to the XML file to which the system writes the converted preference data.

This is a mandatory argument. If not specified, the system displays the command line help, which includes instructions for specifying this argument.

If the file already exists in the specified directory, it is overwritten.

-report_file

Specifies the full path to the log file to which the system logs the results of the conversion. If not specified, the system creates the log file in the directory specified by the **TEMP** environment variable.

Chapter

7 *Configuring search*

Configuring search	7-1
Implementing quick search	7-4
Quick search preferences	7-4
Implementing advanced search	7-5
Configuring Advanced Search preferences	7-6
Creating queries for Advanced Search	7-7
Configuring full-text search	7-8
Configuring FTS preferences	7-9
Default FTS queries	7-12
Indexing the Autonomy search engine	7-13
Batch indexing	7-14
Manually indexing search objects	7-15
Secure indexed data	7-15
Add new types for indexing	7-16
Indexing legacy data	7-16
Real-time indexing	7-17
Implementing synonym search	7-17
Configuring synonym search	7-17
Implementing simple search	7-19
Simple search preferences	7-20

Chapter

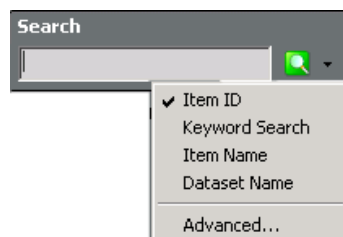
7 *Configuring search*

Configuring search

Teamcenter provides three different search mechanisms. You can use any or all of the search mechanisms in conjunction with each other.

By default, all three mechanisms are enabled. You can modify the behavior of each mechanism. Configuration is required to make full use of **Advanced Search**.

Quick Search



Display	Displays a single-field search form, providing a quick input method for users to search the Teamcenter database.
Behavior	Generates searches based on a single criteria (such as item ID, keyword, item name, or dataset name) selected from a shortcut menu. Administrators can set preferences to determine which criteria display in the menu.
Use	Provides quick searching of items and datasets. It requires users to know the item ID, item name, or the dataset name. Quick searches are limited to a single user. They cannot be stored by the user, nor shared with other users.
Availability	This search mechanism is available by default in both the rich client and thin client. No configuration is required to use this search with its default search queries. However, you can add custom queries.
Configuration options	• Configure which objects appear in the selection list (Item ID, Keyword Search, and so on). • Implement keyword search functionality by configuring and starting full text search. For more information, see Configuring full-text search. • Create custom queries.

Setup

For information about configuring this search mechanism, see [Implementing quick search](#).

Advanced Search**Display**

Displays preconfigured search forms allowing users to specify multiple search criteria relevant to the type of information or object being sought.

Behavior:

Generates searches based on preconfigured search forms. The search forms are derived from search queries. Hundreds of search queries are shipped with Teamcenter: You can create additional search queries (also called saved searches) using Query Builder.

Use

Distributes in-depth, customized search forms throughout the site and/or throughout a global enterprise.

You can also incorporate full-text search capability into the search forms, allowing users to search for datasets by attribute value and search dataset files by keyword.

Building query definitions requires knowledge of the Teamcenter POM (persistent object manager) schema, which is a hierarchical arrangement of classes, subclasses, and attributes, particularly when including reference attributes in the query.

Availability

Advance searches are available by default in both the rich client and thin client. No configuration is required for this search functionality to appear. However, making full use of this search method requires creating search queries in Query Builder, and installing and configuring the Autonomy full-text search engine.

- Configuration**
- Configure full-text search.
 - Configure indexing.
 - Create custom queries.

Setup For information about implementing and configuring this search mechanism, see [Implementing advanced search](#).

Simple Search

Business Object Type:

Select Property

type filter text

- Authorizing Paragraph
- BOM View Revisions(Name)
- Date Archived
- Date Created
- Date Last Backup
- Date Released
- Description
- Gov Classification
- Group ID(Name)

Edit Clause

Date Released

Operator: =

Value: 23-Feb-2011 00:00

Selected Search Criteria

	Property	Oper...	Value
	Owner(Id)	=	mkuser4
AND	Date Created	=	01-Feb-2...
AND	Date Released	=	23-Feb-2...

Display Displays the available properties for the selected business object, allowing users to create searches without an in-depth knowledge of the Teamcenter POM schema. Only the relevant properties for the selected object display. Users do not need to understand the placement of attributes within the POM schema.

Behavior: Generates searches based on one or more property values of selected business objects.

Users select an object type, and then build a search form by selecting properties and specifying criteria.

Use Provides a tool for users to create their own customized searches to search the local Teamcenter database, without a strong knowledge of the POM schema.

These searches are limited to a single user. They cannot be stored by the user, nor shared with other users.

Availability Simple searches are available in the rich client only. It appears by default. No configuration is required for this search functionality, although you can set preferences to modify its behavior.

- Configuration**
- Modify default display settings.
 - Filter the business object types available for search.

Setup For information about configuring this search mechanism, see [Implementing simple search](#).

Implementing quick search

Quick Search is available by default in both the rich client and thin client. No configuration is required to use this search with its default search queries.

You can add custom search items to the **Quick Search** menu by:

- Building the query in Query Builder.

For more information, see the [Query Builder Guide](#).

- Updating two locale files:

`TC_ROOT\lang\textserver\en_US\qry_text_locale.xml`
`TC_ROOT\lang\textserver\en_US\weblocal_locale.xml`

For more information about adding quick search items, the administrator should see the [Client Customization Programmer's Guide](#).

- Updating the two **Quick Search** preferences with the name and attribute of the custom query.

For more information, see [Quick search preferences](#).

Quick search preferences

You must update the following preferences when adding custom queries to the **Quick Search** menu:

- [Quick_Access_Queries](#)

Specifies which queries appear in the **Quick search** menu. Valid values are query names defined in the `qry_text_locale.xml` locale file.

By default, the following queries are defined:

- o **General**
- o **Item**
- o **Item Revision**
- o **Remote**
- o **Checked-Out Dataset**

- [Quick_Access_Queries_Attribute](#)

Specifies the criteria attribute displayed for a query in the **Quick Search** menu using the following format:

internal-query-name_SearchAttribute=L10N-key

The internal query name is specified in the `qry_text_locale.xml` locale file. The L10N key is the **User Entry L10N Key** value specified for the given query.

In the following example, a custom query named **Find PDFs** is created. The value of the L10N key is set to **datasettype_name**.

Attribute	User Entry L10N Key	User Entry Name		Default Value
dataset_type.datasettype_name	datasettype_name	Name	=	PDF

- **WEB_simple_search_default**

Defines the default search option used while performing a **Quick Search** from thin client.

- **WEB_simple_search_queries**

Specifies which queries display in the **Quick Search** box in the thin client.

This preference accepts multiple strings as values. Each string must be a valid query name. By default, the following values are specified:

- o **Item ID**
- o **Keyword Search**
- o **Item Name**
- o **Dataset Name**

Implementing advanced search

Advanced Search is available by default in both the rich client and thin client. No configuration is required to use this search with its default search queries.

This search mechanism is the most far-reaching of the Teamcenter searches. Use this mechanism to:

- Display hundreds of saved queries available by default. Users access the saved queries as search forms in which they can type search criteria.
- Create additional, custom saved queries.
- Distribute saved queries throughout your local site or throughout a global enterprise.
- Implement full-text search with **Advanced Search**, allowing users to search for datasets by attribute value and search dataset files by keyword.

While **Advanced Search** is available by default, making full use of this search method requires creating search queries in Query Builder and installing and configuring the Autonomy full-text search engine.

For more information about creating custom saved queries, see [Creating queries for Advanced Search](#).

For more information about installing the Autonomy full-text search engine, see [Configuring full-text search](#).

For more information about configuring the Autonomy full-text search engine, see [Configuring FTS preferences](#).

Configuring Advanced Search preferences

You can use the following preferences to modify the behavior of the **Advanced Search** mechanism:

- **Change_Search_Default**

Determines which saved query displays in the search panel by default.

This preference accepts a single string as a value. The value must be a valid saved query.

- **QRY_dataset_display_option**

Determines whether the latest version or all versions of a dataset object are displayed when query results are returned.

Set this preference to **1** to display all versions of a dataset object. Set to **2** to display only the latest version of the dataset object.

- **QRY_query-name_SortKeys**

Determines the class attributes used to sort the query results. This preference must be used in conjunction with the **QRY_query-name_SortOrder** preference, which determines sort order.

The value type of the class attribute must be a primitive value type. For example, char, int, double, or date. If a query name contains one of the following characters, this character must be replaced with the underscore (_) character:

- o Space character
- o Return character (\n)
- o Tab character (\t)

- **QRY_query-name_SortOrder**

Determines the sort order of query results. This preference must be used in conjunction with the **QRY_query-name_SortKeys** preference, which determines the class attributes used to sort query results.

Set this preference to **1** to sort query results in ascending order. Set to **2** to sort query results in descending order.

- **QRY_search_type_hierarchy**

Determines whether subtypes are included in query results. The system uses type hierarchy functionality to query types and subtypes.

Administrators can add this as a site preference in the **Options** dialog box, which can be accessed from the **Edit** menu, or in the **tc_preferences.xml** preferences file. Users can set this, as a user preference, to **true** from the search interface:

- o In the rich client, choose **Edit→Options** to display the **Options** dialog box. Select the **Search** folder from the tree on the left. Select the **Query Options** tab. Then select the **Enable Hierarchical Type Search** check box. Checking this check box automatically adds this preference to the database, and sets it to **true**.
- o In the thin client, select the **Options** link from the **Settings** menu in the Navigation bar to display the **Options** dialog box. Click the **Search** tab. Click **Type hierarchy on search**, and then select **Yes** from the list. Selecting this check box automatically adds this preference to the database and sets it to **true**.

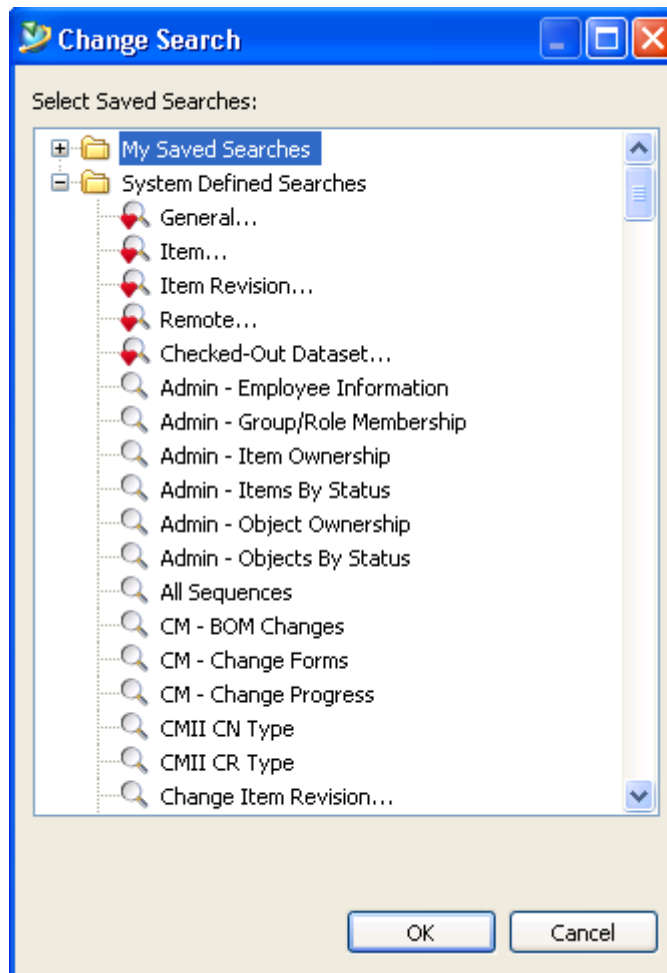
Creating queries for Advanced Search

Query Builder enables you to write complex queries based on the Teamcenter data schema, a hierarchical arrangement of classes, subclasses, and attributes. Query Builder provides hints to assist you in navigating the schema. The hints present a relationship as a starting point, for example, the relationship between an item and its item revision, and then provide you with the steps to build that relationship into your search definition.

Queries created in Query Builder display in the **Saved Queries** tree. Saved queries can be used for searches and to generate reports.

In the context of **Advanced Search**, these queries also appear in the **System Defined Searches** tree, accessed by clicking the **Select a Search** button  on the **Advanced Search** view to display the **Change Search** dialog box.

The following graphic illustrates a few of the saved searches available by default.



For information about creating custom saved queries, see the [Query Builder Guide](#).

Configuring full-text search

When the Autonomy search engine is installed and configured at your site, you can perform full-text searches on dataset files.

Full-text searches can be combined with searches for metadata, allowing you to search for datasets by attribute value as well as searching dataset files by keyword. For example, you can search for all documents containing the keyword **cat** that belong to owning user **jsmith** and have been modified after **December 1, 2007**.

- Wildcards can be used when performing keyword searches; however, partial keywords without wildcard characters cannot be used.
- If real time indexing is enabled, the full-text search index is updated automatically whenever objects or their named references are saved, modified, or deleted.

For information about performing specific types of full text searches in the rich client, see the [Rich Client Interface Guide](#).

You must perform the following tasks to configure full-text search:

- Install the Autonomy full-text search engine
For information about installing full-text search, see the appropriate server installation guide (for [Windows](#) or [UNIX/Linux](#)) and the [Teamcenter Environment Manager Help](#).
- Define a set of configuration preferences
For information about the set of full-text search engine preferences, see [Configuring FTS preferences](#).
- Run the **build_fts_index** utility to build keyword indexes for the Autonomy search engine on an object-by-object basis for both the properties of dataset objects and the contents of dataset files.
For information about this utility, see the [Utilities Reference](#).

Configuring FTS preferences

You can modify the behavior of the Autonomy search engine using preferences.

You must set the following full-text search preferences for the Autonomy search engine to run.

- **TC_fts_aci_port**
Specifies the Autonomy Content Infrastructure (ACI) action port number. This port is used to send action query commands to the Autonomy IDOL server.
This preference accepts a single string as a value. The value must be the number of the port used by the ACI. This setting should be the same as the **Port** setting in the Autonomy IDOL server's configuration file.
- **TC_fts_database_name**
Specifies the name of the Autonomy database in which Teamcenter database objects are indexed.
This preference accepts a single string as a value. The value must be a valid database name, in which the Teamcenter database objects are indexed.
- **TC_fts_engine**
Specifies the search engine used by Teamcenter's full text search (FTS) framework.
Search engines are used for searching within text documents such as MS Word documents, PDF files, and HTML files. An opportunity to install the Autonomy search engine (includes the IDOL server and the File System Fetch connector) is provided during installation using the Teamcenter Environment Manager. Alternatively, you can define a different plug-and-play search engine using this preference.
This preference accepts a single string as a value.
- **TC_fts_host**
Specifies the IP address of the host machine running the Autonomy server.
If the Teamcenter server and the Autonomy server are installed on two different machines, set the **TC_TMP_DIR** environment variable with the UNC path of

the Teamcenter server machine to a directory which can be accessed by the Autonomy server. For example:

```
%TC_TMP_DIR%=\ahi3w099\temp
```

In this situation, the Teamcenter server first creates temporary indexing files in the *TC_TMP_DIR* directory, then the Autonomy server reads the files for indexing.

Note The **TC_TMP_DIR** environment variable is located in the **tc_profilevars.bat** file in the *TC_DATA* directory, or in the **start_TcServer1.bat** file in the *TC_ROOT* directory.

This preference accepts a single string as a value. The value must be the valid IP address of the host machine running the Autonomy server.

- **TC_fts_index_port**

Specifies the index port number. This port is used to index documents for the Autonomy IDOL server.

This preference accepts a single string as a value. The value must be the number of the Autonomy IDOL server's index port. This preference setting should be the same as the **IndexPort** setting in the Autonomy IDOL server's configuration file.

- **TC_fts_fsf_host**

Specifies the IP address or the name of the host machine on which Autonomy File System Fetch is running. The host and port specified by the **TC_fts_fsf_port** preference are used by the FTS to send the import command to File System Fetch, which imports and indexes documents on the IDOL server.

This preference accepts a single string as a value. The value must be a valid IP address or host name.

- **TC_fts_fsf_port**

Specifies the port number on which Autonomy File System Fetch is listening. The port and the host specified by the **TC_fts_fsf_host** preference are used by the FTS framework to send the import command to File System Fetch, which imports and indexes documents on the IDOL server.

This preference accepts a single string as a value. The value must be a positive integer.

- **TC_fts_query_port**

Specifies the query port number used to query Autonomy IDOL server content.

This preference accepts a single string as a value. Teamcenter installation provides an opportunity to install the Autonomy search engine. If you choose this option, this preference is automatically set to **9000**.

- **TC_fts_service_port**

Specifies the service port number used to query the IDOL server service.

This preference accepts a single string as a value. Teamcenter installation provides an opportunity to install the Autonomy search engine. If you choose this option, this preference is automatically set to **9002**.

You must set the following full-text search preferences when the Teamcenter server and the Autonomy server are installed on different machines.

- **TC_fts_use_shared_tmp_dir**

Specifies whether the Autonomy server uses a network-shared folder as a temporary work directory.

If the Teamcenter server and the Autonomy server are installed on different machines this preference must be set to **true**.

- **TC_fts_windows_autonomy_server**

Specifies whether the Autonomy server is installed on a Windows platform.

- **TC_fts_shared_tmp_dir_unix**

Specifies the temporary work directory for the Autonomy server running on UNIX.

If the Teamcenter server and the Autonomy server are installed on different machines, you must set either this preference (UNIX) or the **TC_fts_shared_tmp_dir_wnt** preference (Windows).

Note Settings for the **-workdir** and **-db** arguments of the **build_fts_index** utility override this preference.

- **TC_fts_shared_tmp_dir_wnt**

Specifies the temporary work directory for the Autonomy server running on Windows.

If the Teamcenter server and the Autonomy server are installed on different machines, you must set either this preference (Windows) or the **TC_fts_shared_tmp_dir_unix** preference (UNIX).

You can use the following optional full-text search preferences to further modify the Autonomy search engine.

- **TC_fts_index_retries**

Specifies the number of times a connection with the Autonomy server is attempted before exiting. When the number of attempts is exceeded, the system displays an error.

Accepts a single string as a value. Must be single positive integer.

- **TC_fts_index_timeout**

Specifies how long, in milliseconds, the client waits for data from the server before closing the connection. When the length of time is exceeded, the system displays an error.

Accepts a single string as a value. Must be single positive integer.

- **TC_fts_indexing_wait_timeout**

Specifies the amount of time, in seconds, the FTS search engine is allowed to index one object. Because this preference is used during real-time indexing for the indexing of work space objects, its setting significantly affects system performance.

Accepts a single string as a value.

- **TC_fts_real_time_indexing**

Specifies whether object indices are created or updated into the FTS index database in real time, when the objects are created, modified, or deleted.

- **TC_fts_indexed_types**

Specifies types whose objects can be indexed. The FTS framework uses this preference to determine if a given object should be indexed. If this preference is not defined, the FTS framework does not filter out any objects.

Accepts multiple strings as values. Each string must be a valid object type name.

Note The Autonomy search engine supports numerous data formats.

For information about Autonomy information connectivity, supported formats, and related resources, see the Autonomy Web site:

<http://www.autonomy.com/content/Technology/idol-functionality-information-connectivity/index.en.html>

- **type-name_indexed_properties**

Specifies the properties to be indexed to the Autonomy database for a particular object type. This preference supports type (hierarchy) inheritance.

Accepts multiple strings as values. Each string must be a combination of a valid type name and one or more valid property names.

- **TC_fts_max_results**

Specifies the number of query results returned by the full text search engine. Use this preference to avoid potential performance overhead on the Autonomy server.

Accepts a single string as a value. Must be single positive integer.

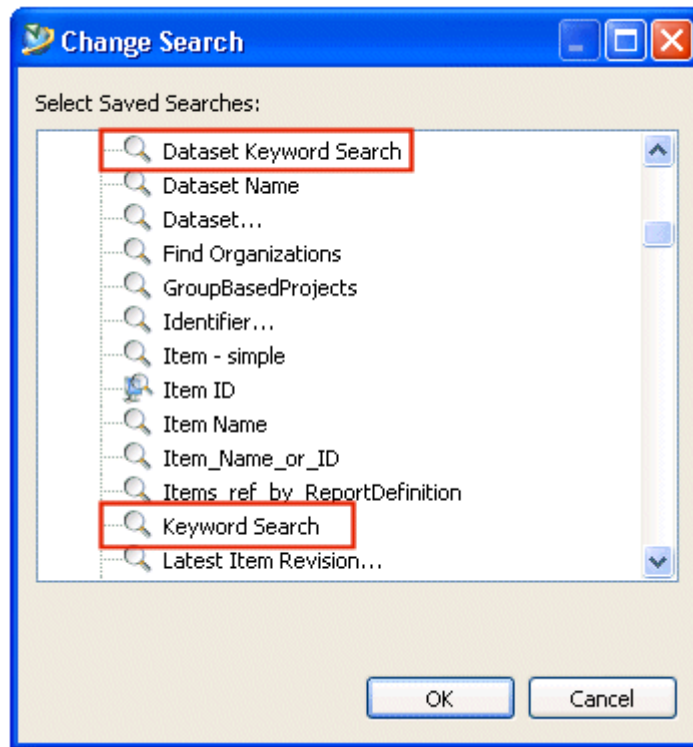
For more information about the behavior and default settings of these preferences, see the individual preference topics in the [Preferences and Environment Variables Reference](#).

Default FTS queries

After the Autonomy search engine is installed and configured at your site, you can perform full-text searches on dataset files. Two full text search queries are shipped with Teamcenter:

- **Dataset Keyword Search**
- **Keyword Search**

Access these two full text search queries, along with all saved searches shipped with Teamcenter, by clicking the **Select a Search**  button on the **Advanced Search** view to display the **Change Search** dialog box.



Additionally, you can also create custom saved queries.

For more information, see [Creating queries for Advanced Search](#).

Indexing the Autonomy search engine

Indexing is required for keyword searches. If indexing is configured, keyword searches are available from **Quick Search** and **Advanced Search**.

A keyword search queries the Autonomy search engine on an object-by-object basis for both the properties of dataset objects and the contents of dataset files.

The prerequisites for creating an index for the Autonomy search engine are:

- Full-text search is installed.

For information about installing full-text search, see the appropriate server installation guide (for [Windows](#) or [UNIX/Linux](#)) and the [Teamcenter Environment Manager Help](#).

- The following full-text search preferences are set:

TC_fts_index_port

Specifies the index port number used to index documents for the Autonomy IDOL server.

TC_fts_index_retries

Specifies the number of times a connection with the Autonomy server is attempted before exiting.

TC_fts_index_timeout

Specifies how long, in milliseconds, the client waits for data from the server before closing the connection.

TC_fts_indexed_types

Specifies which object types are indexed.

Note The Autonomy search engine supports numerous data formats.

For information about Autonomy information connectivity, supported formats, and related resources, see the Autonomy Web site:

<http://www.autonomy.com/content/Technology/idol-functionality-information-connectivity/index.en.html>

TC_fts_indexing_wait_timeout

Specifies the amount of time, in seconds, the FTS search engine is allowed to index one object.

When these prerequisites are met, you can create an index using one of the following methods:

- Batch indexing

Use the **build_fts_index** utility to create and administer indexing.

For more information, see *Batch indexing*.

- Manual indexing

Choose **Tools**→**Check Index Status** and use the **Check Index Status** dialog box to view the current status of the selected object. You can also use this dialog box to enable and disable indexing on the selected object.

For more information, see *Manually indexing search objects*.

Batch indexing

An administrator can use the **build_fts_index** utility to create keyword indexes for the Autonomy search engine on an object-by-object basis. These indexes become part of the full-text keyword search functionality and can index both the properties of dataset objects and the contents of dataset files. If a dataset file is not of a document type supported by Autonomy, the utility can invoke a user-specified filter program to convert the file to a supported format.

Use the utility to manage all aspects of the index, including:

- Types of objects to index
- Queries to be indexed
- Number of files indexed in a given batch
- Logging
- Reports

For more information about using the utility, see the *Utilities Reference*.

Manually indexing search objects

1. In My Teamcenter, select the dataset(s) to be indexed.
2. Choose **Tools**→**Check Index Status**.

The **Check Index Status** dialog box appears showing the current indexing status for the selected objects. The indexing status may be:

- **Updated**

Indicates the index for the object is current.

- **Not Updated**

Indicates the object is not indexed.

- **Type Not Supported**

Indicates the object is not specified by the [TC_fts_indexed_types](#) preference and is not supported for indexing.

3. In the **Check Index Status** dialog box, select the objects you want to index.

Note To select the objects, hold the Shift key for adjacent values and the Ctrl key for nonadjacent values.

4. Click **Reindex** to run the indexing process for the selected objects.

The system displays indexing errors, if any, and updates the status in the dialog box.

Note If the full-text search capability is not installed, the system displays a dialog box with this message: **The FTS engine could not be found.**

5. Click **Close** to close the dialog box.

Secure indexed data

Any user can view indexed data using a Web browser on the host upon which **TcServer** is installed, regardless of a user's permissions.

To ensure security of Teamcenter data, run the Teamcenter client and **TcServer** on separate hosts and perform the following steps:

1. Authorize a specific machine as an IDOL and **TcServer** host in the **idol.cfg** server section by:
 - Setting **AdminClients** to **localhost,127.0.0.1**.
 - Setting **QueryClients** to **localhost,127.0.0.1**.
2. Create a **temp** directory on the host machine.
3. Share the **temp** directory, using a UNC path and full permissions.
4. Set the **TC_TMP_DIR** environment variable to **\\host-machine\temp**.
This allows the file attached to the dataset to be indexed.

5. (Optional) If necessary, you can install **TcServer** on additional hosts. If so, add the additional host names to **AdminClients** and **QueryClients**.

Add new types for indexing

To add new types for indexing, you must first add the new type to the list of types to be indexed in the **TC_fts_indexed_types** preference, and then create a new indexed properties preference for the type.

Note The Autonomy search engine supports numerous data formats.

For information about Autonomy information connectivity, supported formats, and related resources, see the Autonomy Web site:

<http://www.autonomy.com/content/Technology/idol-functionality-information-connectivity/index.en.html>

1. Choose **Edit→Options** to open the **Options** dialog box and select the **Index** tab to display the preference index.
2. Select **Configuration.Full Text Search** to filter the preference index to full-text search preferences.
3. Click the **TC_fts_indexed_types** preference from the preference list.
4. Add the new type to be indexed to the **Current Values** box. For example, add **Part Revision**.
5. Click **Modify**.
The new type is added to the preference.
6. Select the **New** tab at the bottom of the dialog box and create a new **dataset_indexed_types** preference for the new type.
In this example, create a new **Part Revision_indexed_types** preference.
7. Select **Site** for the scope.
8. Select **True** for **Multiple Values**.
9. In the **Values** box, add the properties you want to index. For example, add **object_name** and **object_desc**.
10. Click **Create**.
11. To see your changes, close the rich client session and restart the Teamcenter server.
12. Manually index the new type.
For more information, see *Manually indexing search objects*.

Indexing legacy data

You can export indexed data from a previous database by exporting all indexed data from the old database to a series of compressed files named **output-0.idx.gz**,

output-1.idx.gz, and so on. These files are placed in the export path on the Autonomy server host.

For more information about this procedure, see the [Upgrade Guide](#).

Real-time indexing

Real-time indexing updates the full-text search index automatically, whenever objects or their named references are created, saved, modified, or deleted.

Enable real-time indexing with the [TC_fts_real_time_indexing](#) preference.

Tip Before creating a large amount of data (more than one hundred objects), disable real-time indexing. After the data is created, use the **build_fts_index** utility to index the data. After indexing, enable real-time indexing if desired.

Because this functionality indexes each object configured as an indexed type at the time it is created, disabling real-time indexing before creating large amounts of data improves performance.

For more information about using the utility, see the [Utilities Reference](#).

Implementing synonym search

Administrators enable *synonym search* to broaden searches.

To configure synonym search:

- Enable synonym search by setting the [TC_fts_synonym_search](#) site preference.
- Create a synonym search text file.
- Configure the Autonomy full text search engine IDOL server.

Note You must configure IDOL server to use the synonym file before you index the data to search.

Configuring synonym search

1. Set the [TC_fts_synonym_search](#) site preference to **true**.
2. Create and populate a synonym search text file.
 - a. Create a text file in the IDOL server **IDOL/content** directory.
The file name is later specified in the IDOL server configuration file **[SynonymType]** section.
 - b. Create sections for each language defined in IDOL server configuration file.
For example:


```
[EnglishASCII]
[GermanUTF8]
```
 - c. In each section, create a line for each word that gets synonyms. For example:


```
[EnglishASCII]
cat
```

```
dog
[GermanUTF8]
Katze
Hund
```

- d. List synonym strings next to each word and save the file.

Note

Use commas to separate words.

Do not put spaces before or after a comma.

Individual terms can contain spaces, but cannot contain punctuation.

For example:

```
[EnglishASCII]
cat,feline,grimalkin,moggy,mouser,puss,pussy,tabby
dog,bitch,cur,hound,mans best friend,mongrel,mutt,pooch,puppy
[GermanUTF8]
Katze,Mietze,Mietzekatze,Mietzekater,Kater,Mulle,Kätzchen
Hund,Wau Wau,Hündin,Töle,Kläffer,Hündchen,Welp
```

- e. Save and close the file.
3. Configure the Autonomy full text search engine IDOL server to use a synonym file.
- a. Open the IDOL server configuration file, **IDOL.cfg**, in a text editor.
- b. In the IDOL server configuration file **[FieldProcessing]** section, set up a synonym process so the IDOL server can determine when to apply synonym settings. For example:

```
[FieldProcessing]
0=SynonymMatch
```

- c. Create a section for the synonym process with create a property for the process, and then identify the fields that you want to associate with the process. For example:

```
[SynonymMatch]
Property=ApplySynonymMatch
PropertyFieldCSVs=*/DRETITLE,*/DRECONTENT
```

Note

Synonym properties must point to a defined synonym job.

In this example, the IDOL server returns only documents for synonym queries when the DRETITLE or DRECONTENT field values match the query.

Caution

The property you create must not have the same name as the process. When you identify fields, use the */FieldName* format to match root-level fields, **/FieldName* to match all fields except root-level, or */Path/FieldName* to match fields pointed to by the specified path.

- d. List the property that you have created in a **[Properties]** section. For example:

```
[Properties]  
0=ApplySynonymMatch
```

- e. Create a section for the property in which you set the **SynonymType** parameter to the name of the synonym job that specifies the settings IDOL server should apply to synonym queries. For example:

```
ApplySynonymMatch]  
SynonymType=Synonym_job
```

- f. In the IDOL server configuration file's **[Synonym]** section, list the synonym job whose settings you want to apply when a synonym query is sent to IDOL server. For example:

```
[Synonym]  
0=Synonym_job
```

Note You can set up multiple jobs, but normally only one is needed.

- g. Define a section for your synonym job using the same name as the synonym job already specified, and then add settings to apply to synonym queries. For example:

```
[Synonym_job]  
File=animals.txt  
MaxExpandLevel=1
```

- h. Save the configuration file and restart IDOL server.

Implementing simple search

Simple Search is available by default in the rich client. Users can use this mechanism to create business object searches based on one or more property values, without an in-depth knowledge of the Teamcenter POM schema. This mechanism provides a powerful search tool to end-users, with only the following limitations:

- Only **WorkspaceObjects** can be specified for this type of search.
To allow users to search for other POM objects, you must build queries containing these objects using Query Builder. Such queries are accessed from **Advanced Search**.
- Only attribute properties and typed referenced properties can be specified for this type of search.
To allow users to search for run-time properties, relation properties, and compound properties, you must build queries containing these properties using Query Builder. Such queries are accessed from **Advanced Search**.
- Scope is limited to the users local Teamcenter database.
To allow users to search remote Teamcenter databases, use the **Remote** saved query, or build a custom query using Query Builder. Such queries are accessed from **Advanced Search**.

- Ad hoc and Classification searches are not supported.
- Search results display in the **Search Results** view. Users can refresh, compare, save, and assign these search results. However, the searches themselves cannot be saved or shared.

No configuration is required before users can begin using this search mechanism. However you can modify its behavior by setting preferences.

For more information about **Simple Search** preferences, see [Simple search preferences](#).

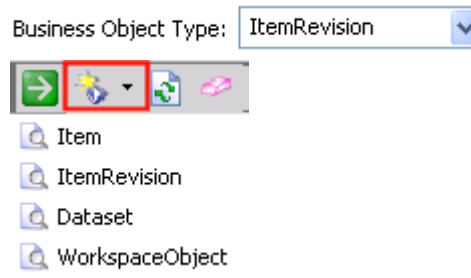
Simple search preferences

You can use the following preferences to modify the behavior of the **Simple Search** mechanism:

- **Default_Business_Object_To_Search**
Specifies which business object displays when the **Simple Search** view is opened.
- **Favorite_Business_Objects_To_Search**
Specifies which business object types appear in the:

Business Object Type list

Business Object Type
button



- **Searchable_Business_Objects**
Filters the business object types that display in the **Business Object Type** list, limiting the list to the business objects specified by this preference. Limit the list to only those business objects used at your site, allowing users to locate business object types quickly.

This preference can be set as a site preference by an administrator and as a group or role preference by a group administrator.

Chapter

8 *Configuring Smart Codes in Teamcenter Rapid Start*

Configuring Smart Codes in Teamcenter Rapid Start	8-1
Smart Codes example	8-1
Enable Smart Codes	8-3
Editing the Smart Codes configuration file	8-3
Editing the configuration file	8-3
Configuration file parameters	8-4
Configuration file parameters	8-4
ID	8-4
Action	8-4
Label	8-4
Choice – optional	8-4
Type	8-5
Repository	8-6
File name	8-7
Generate	8-7
Restriction	8-7
ReuseNumber	8-8
CounterKey	8-8
Length	8-9
LeadingChar	8-9
Prefix	8-9
Suffix	8-9
FieldSet	8-9
StepLength	8-10
StepOffset	8-10
StepFill	8-10
FirstValue	8-10
SecondValue	8-10
Select	8-11
ForType	8-11
Extends	8-11
Parameter aliases	8-13
Parameter aliases	8-13
Selection expression	8-19
Section	8-21
Import and export configuration files	8-22

Chapter

8 *Configuring Smart Codes in Teamcenter Rapid Start*

Configuring Smart Codes in Teamcenter Rapid Start

Item IDs are created when a user makes selections from various sections in the New Item dialog box. Smart Codes allows you to preconfigure the label and contents of each section that makes up the item ID. This provides uniformity and ease-of-use in creating item IDs.

The basic steps for configuring Smart Codes are as follows:

1. Enable Smart Codes as described in [Enable Smart Codes](#).
2. Edit the **tcx_sc_config.txt** configuration file as described in [Editing the configuration file](#).

The configuration file is located in the **TC_ROOT\install\tcx\data\EN\SmartCodes** directory. It defines the order, dependency, and label for each section in the item ID.

3. Import the configuration file into the database as described in [Import and export configuration files](#).

Smart Codes example

The following figure shows an example of the **New Item** dialog box with Smart Codes enabled and configured. The table below it describes each section in the dialog box. In this example, the ID is composed of five sections. The first four sections are defined by lists of values (LOVs) and the last section is a counter that can be automatically assigned.

This example is used throughout the remainder of this topic.

Label	Entry type	Values	Number of characters
Facility	LOV	• Detroit – DT • St. Louis – SL • Ann Arbor – AA • Dallas – DA • Huntsville – HU • Boston – BO	2
Product Family	LOV	• Structural – ST • Mechanical – ME • Fastener – FA	2
Structural, Mechanical, or Fastener	Filtering LOV	• Structural – ST	2
Note A filtering LOV is a list of values that varies based on a previously selected option. In this example, the LOV that displays in this section is		Brackets – B Plates – P Frames – F • Mechanical – ME • Motors – M • Drives – D • Gears – G • Fastener – FA	

Label	Entry type	Values	Number of characters
	based on the value selected as the product family.	Bolted connection – B Clamps – C	
Source	LOV	Make in house – 1 Preferred vendor – 2 Tier-2 vendor – 3	1
Number	Counter	User can manually enter a number or can click the Assign button to automatically generate a unique number.	Up to 6 digits

Enable Smart Codes

Before users can create new items using the Smart Codes functionality, it must be enabled. This is done using the **TCX_Smart_Codes** preference.

1. Log on to My Teamcenter.
2. Choose **Edit→Options**.
3. Click **Index** and enter **TCX_** in the **Search on preference name** box.
The system displays all preferences that begin with **TCX**.
4. Select **TCX_Smart_Codes**.
5. In the **Current values** box, enter **1**, and then click the **Modify** button.
6. Click **Cancel**.

To disable Smart Codes, enter **0** in the **Current values** box.

Editing the Smart Codes configuration file

Editing the configuration file

The **tcx_sc_config.txt** file is located in the **TC_ROOT\install\tcx\data\EN\SmartCodes** directory. This file is used to define the parameters for each section in the item ID (for example, order, labels, and counters).

Each section in the item ID has a corresponding section in the configuration file. For example, **Facility** has a corresponding section in the configuration file that looks as follows:

```

ID : FACILITY_CODE
    Action      : DEFINE
    Label       : Facility
    Choice      : 1
    Type        : LOV
    Repository  : PN_Facility_CODE
    Generate    : COMBO
    ReuseNumber : NONE
    Fortype     : Item:Item.item_id

```

Configuration file parameters

Configuration file parameters

Each section in the configuration file contains a list of parameters and corresponding values. Some are mandatory and some are optional. Parameters without any influence are ignored. Each parameter must adhere to the following format:

Parameter: Value

ID

The ID is a unique section ID. The ID is a string with a maximum length of 63 characters. If you use similar ID names for different types, use the type name as a part of the section ID.

Example:

```
ID : ITEM_PROJECT
```

Action

The action entry defines the action that is taken on this section when the configuration file is imported into the database. There are four possible modes:

CREATE	The section is created if and only if it does not exist.
DEFINE	The section is created if it does not exist or modified if it does exist.
MODIFY	The section is modified if and only if it does exist.
DELETE	The section is deleted if no references exist.

Note To delete part-numbering schemes, you must import the configuration file multiple times.

Label

This parameter defines the default label value for the section. The label values might be overloaded with property files.

Choice – optional

Use this parameter to define what is displayed in the menu for a LOV. For example, if the LOV has a value of **DT|Detroit**, and the **Choice** parameter is set to **0**, **DT** is used in the item ID and displayed in the menu. If the **Choice** parameter is set to **1**, **DT** is used in the item ID and **Detroit** is displayed in the menu. If the **Choice**

parameter is omitted, **DT** is used in the item ID and **DT|Detroit** is displayed in the menu.

The following table details the Facility section from the example described earlier in this topic.

	Choice : 0		Choice : 1		Choice Omitted	
LOV Content	Item ID	Selection Menu	Item ID	Selection Menu	Item ID	Selection Menu
DT Detroit	DT	DT	DT	Detroit	DT	DT Detroit
SL St. Louis	SL	SL	SL	St. Louis	SL	SL St. Louis
AA Ann Arbor	AA	AA	AA	Ann Arbor	AA	AA Ann Arbor
DA Dallas	DA	DA	DA	Dallas	DA	DA Dallas
HU Huntsville	HU	HU	HU	Huntsville	HU	HU Huntsville
BO Boston	BO	BO	BO	Boston	BO	BO Boston

If you want to define multiple values within a selection, start with values of similar length. For example:

01|Y| Extern purchased with supporter

01|N| Extern purchased without supporter

02|N| Intern produced

03|N| From other site

01, 02, and 03 are used in the item ID, Y/N can be used for select entry of optional section, and the text is displayed (Choice: 2).

Type

This parameter defines the type of values used for the section. Some of the type definitions also require the [Repository](#) parameter.

Type	Repository	Allowed values
ALPHA	None	Any character
ALPHANUM	None	Any character and digit
NUMBER	None	Digits
MASK¹	Definition mask	Values specified by mask
FIXED	Value ²	Computed value
LOV	LOV name ²	Value from LOV
LOVPERS	LOV name ²	Value from LOV snapshot
LIST	List of values separated by a comma (,)	Value from repository list
DEFAULT	None	Computed value

¹Supported in the rich client only.

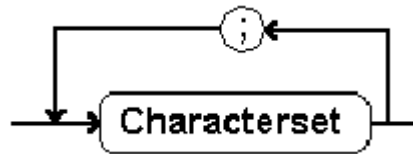
²Use parameter aliases for resolving.

Repository

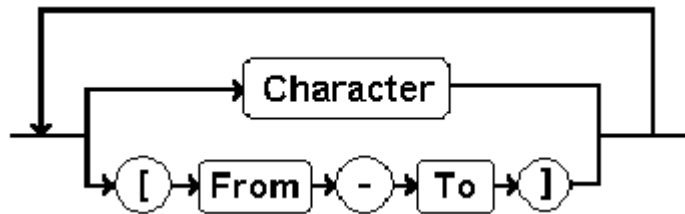
The **Repository** parameter defines a data source for the specified **Type** parameter. For example, if **Type** is set to **LOV**, the **Repository** section is used to define which LOV to use.

Definition Mask

The definition mask defines allowed combinations of characters. The current implementation supports the following definition:



where **Characterset** is:



Example:

[0-9];[A-Z];[A-Z] for number 0AA to 9ZZ

Pattern Matching Mask

Specifies a mask for pattern matching.

LOV Name

Defines the name of an existing LOV. If the LOV does not exist, an error occurs. The LOV name can be defined by parameter aliases. This capability allows the composition of depending LOVs, where the LOV name is composed of previously selected values. The implicit dependencies mechanism automatically displays all sections necessary for the composed LOV name.

Example:

```
ID : ITEM_PROJECT
  Action_ : DEFINE
  Label   : Project
  Type    : LOV
  Repository : ECS_Project
  Generate : COMBO
  FirstValue : EDF
ID : ITEM_PROJECT_P
  Action_ : DEFINE
  Label   : Project Mnemonic
  Type    : LOV
  Choice  : 1
```

```
Repository : ${Param.ITEM_PROJECT}_project_site
Generate   : COMBO
ForType    : Item:Procurement.item_id
```

The **ITEM_PROJECT** section is used only as a selector for the correct LOV in **ITEM_PROJECT_P** and is not a part of the composed ID. It is automatically displayed before the **ITEM_PROJECT_P** section, because the **ITEM_PROJECT_P** section is implicitly dependent on **ITEM_PROJECT**.

List of Values

A simple list is stored directly in the **Repository** parameter. The values are separated with commas (,) and the total length must not exceed 250 characters.

Example:

```
Repository : 1,2,3,None
```

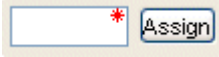
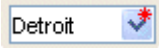
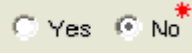
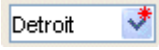
You can also use multiple values separated by pipes (|).

File name

This parameter defines an external file as the source for the section. The file name can be composed dynamically using the functionality of parameter aliases.

Generate

This parameter defines how the section displays in the dialog box.

Generate setting	Display example	Valid types
ASSIGN		ALPHA, ALPHANUM, NUMBER, MASK, LOV, LOVPERS, LISTPERS, FILE, FILEPERS
LOV		LOV, LIST
OPTION		LOV, LIST
COMBO		LOV, LIST
EDIT		ALPHA, ALPHANUM, NUMBER, MASK, PATTERN, FIXED

Restriction

This parameter defines whether or not the user is allowed to change the value of the section. This is usually combined with an LOV or assigned value.

Restriction setting	Effect	Valid generate options
SUGGESTIVE	The user is allowed to change values.	ASSIGN

Restriction setting	Effect	Valid generate options
RESTRICTIVE	The user is not allowed to change values.	LOV

ReuseNumber

This parameter controls the behavior for generating numbers using a counter. The new number can be generated from scratch, from a stored list, or as the next value after an existing one.

For example, assume that numbers 1, 2, 3, 4, and 5 are used in part numbers. Then, the part number using number 3 is deleted. If **ReuseNumber** is set to **NONE**, the next number generated is 6. If **ReuseNumber** is set to **FALLBACK**, the next number used is 3, because it is the first available number.

ReuseNumber setting	Effect	Valid types
NONE	The next value generated is based on the last one used.	All
FALLBACK	If a Repository parameter exists, the next value is loaded from the repository. Otherwise, the next value is generated based on the last one used. If the generated ID is not used (for example, the process was cancelled), it is saved into the repository and used the next time a number is generated.	All

Note Use **FALLBACK** if you have only short assigned parts. They are consumed faster than expected.

CounterKey

This parameter defines the name of the counter used for the assign. Each counter has unique name, but multiple sections may share the same counter. For example, you can make the counter dependent on the project but not on the group, so you can use the same counter for single and mirror part numbers. The **CounterKey** parameter supports parameter aliases.

Example:

```
CounterKey : CONS_$LIST($Param.ITEM_PROJECT;0)-$LIST($Param.ITEM_PROJECT_P;0)
-$LIST($Param.ITEM_DOCUMENT_ORIGINATOR_CODE_P;0)-$LIST($Param.ITEM_DOCUMENT_TYPE_CODE_P;0)
```

Note The **CounterKey** parameter does not generate implicit dependencies.

Length

This parameter defines the maximum length of the value. This is used by the **ALPHA**, **ALPHANUM**, and **NUMERIC** types. For an assign without a **FirstValue** parameter, the first assigned value is completed with **LeadingChar** to this length.

LeadingChar

This parameter defines the leading character used to complete the generated number. For **NUMERIC** and **ALPHANUM** types, the default value is **0**. For **ALPHA** types, the default value is **A**.

Prefix

The prefix is a fixed value that precedes the section value. The user cannot change this value. This parameter is similar to a section with a **FIXED** type but differs in the display and evaluation. The **Prefix** parameter allows fixed string values only.

Suffix

The suffix is a fixed value that follows the section value. The user cannot change this value. This parameter is similar to a section with a **FIXED** type, but differs in the display and evaluation. The **Suffix** parameter allows fixed string values only.

FieldSet

This parameter allows the setting of item master and item revision master form fields. The values can be derived from the section values using parameter aliases. There is a defined support for multiple generated numbers. Multiple entries within one section are possible. Each entry has following syntax:

```
I|IM|IR|IRM.attribute_name=parameter_aliases_expression
```

where:

I = Item

IM = Item master form

IR = Item revision

IRM = Item revision master form

form_attribute_name = Any valid form attribute name

parameter_aliases_expression = Expression defining the value, using parameter aliases

Restrictions:

- This parameter is active only in sections that are part of the composed item ID.
- The values are set after property rules and before **item_*_msg**. If there are mandatory fields in the form, their values must be defined by a property rule or the **FieldSet** parameter.

Example:

- `FieldSet: IM.PROJECT=$Param.ITEM_PROJECT`

Puts the selected project into the Item Master Form in the **PROJECT** field.

StepLength

This parameter defines the increment/decrement size on the counter value if an assign is performed. Negative values mean decrement. The default value is 1. This parameter allows you to generate numbers in different steps.

Example:

- `StepLength : 5`

Generates counter values of 5, 10, 15, and so on.

StepOffset

This parameter defines the relative offset to each step start. It works together with the [StepLength](#) parameter and is added to the generated value. The default and minimum value is 0. The maximum value is `StepLength-1`.

Example:

- `StepLength : 5`
`StepOffset : 2`

Generates counter values of 7, 12, 17, and so on.

The algorithm is `LastNumber += ( LastNumber % StepLength ) + StepLength + StepOffset`.

StepFill

This parameter defines the number of IDs (and items) that can be simultaneously generated by Smart Codes. The default and minimum value is 1. Multiple numbers are generated by larger values. If the **StepFill** parameter is larger than 1, the numbers from the counter repository cannot be used, but the unused ones are automatically saved and may be used by another section with the same counter key.

Example:

- `StepLength : 5`
`StepOffset : 2`
`StepFill : 2`

Generates counter values of 7 and 8, 12 and 13, 17 and 18, and so on.

FirstValue

This parameter defines the initial value for a new counter. The value is not checked by other rules. It is your responsibility to enter a correct value.

SecondValue

This parameter defines the second value for a counter. It is used if the counter value is equal to the value of the [FirstValue](#) parameter. The value is not checked by other rules. It is your responsibility to enter a correct value.

Select

This parameter allows you to exclude a section in a dependency on an expression. The expression supports the =, !=, <, > operators. Parameter aliases can be used for the left-side and right-side values.

The implicit dependencies mechanism automatically displays all sections necessary for the left- and right-side values.

Example:

```
ID : ITEM_MIRROR
Action      : DEFINE
Label       : Left/Right
Choice      : 1
Type        : LIST
Repository  : Y|Ja,N|Nein
Generate    : OPTION
ReuseNumber : PERSISTENT
ID : ITEM_MIRROR_COUNTER
Action      : DEFINE
Label       : Number
Type        : NUMBER
Generate    : ASSIGN
ReuseNumber : FALLBACK
Length      : 3
StepLength  : 2
StepFill    : 2
StepOffset  : 1
Select      : $Param.ITEM_MIRROR==Y
FirstValue  : 001
CounterKey  : ITEM_$Param.ITEM_PROJECT-$Param.ITEM_AECMA_MAINCHAP-$Param.
ITEM_AECMA_SUBCHAP-$Param.ITEM_AECMA_PRODUCT
Extends     : ITEM_AECMA_PRODUCT
ID : ITEM_SINGLE_COUNTER
Action      : DEFINE
Label       : Number
Type        : NUMBER
Generate    : ASSIGN
ReuseNumber : FALLBACK
Length      : 3
FirstValue  : 001
Select      : $Param.ITEM_MIRROR==N
CounterKey  : ITEM_$Param.ITEM_PROJECT-$Param.ITEM_AECMA_MAINCHAP-$Param.
ITEM_AECMA_SUBCHAP-$Param.ITEM_AECMA_PRODUCT
Extends     : ITEM_AECMA_PRODUCT
```

The choice in **ITEM_MIRROR** defines whether the **ITEM_MIRROR_COUNTER** or **ITEM_SINGLE_COUNTER** section is displayed.

ForType

This parameter defines the item type for which the section is used. It specifies the first section of the composed ID. The format is **Item:ItemType.item_id**, where the *ItemType* is the concrete item type. You can only have one section for each item type.

Extends

This parameter is used to defines the order of the item ID sections. The value of this parameter represents the section that the current section should follow. In the following example, **FACILITY_CODE** is the first section in the item ID (as defined by the **ForType** parameter). The next section in the

item ID is **Product_Family_Code** because its **Extends** parameter is set to **FACILITY_CODE** (meaning, it follows **FACILITY_CODE**). The **SUB_CLASS** follows **Product_Family_Code**, and so on.

This entry may contain multiple values separated by comma (.). The **Extends** parameter defines an explicit dependency.

```

ID : FACILITY_CODE
  Action      : DEFINE
  Label       : FACILITY CODE
  Choice      : 1
  Type        : LOV
  Repository  : PN_Facility_CODE
  Generate    : COMBO
  ReuseNumber : NONE
  ForType     : Item:Item.item_id

ID : Product_Family_Code
  Action      : DEFINE
  Label       : PROD FAMILY CODE
  Choice      : 1
  Type        : LOV
  Repository  : Product_Family_Code
  Generate    : COMBO
  ReuseNumber : NONE
  Extends     : FACILITY_CODE

ID : SUB_CLASS
  Action      : DEFINE
  Label       : SUB CLASS
  Choice      : 1
  Type        : LOV
  Repository  : SUB_LOV_PFC_$LIST($Param.Product_Family_Code;0)
  Generate    : COMBO
  ReuseNumber : NONE
  Extends     : Product_Family_Code

ID : SOURCE_CODE
  Action      : DEFINE
  Label       : SOURCE CODE
  Choice      : 1
  Type        : LOV
  Repository  : SOURCE_CODE
  Generate    : COMBO
  ReuseNumber : NONE
  Extends     : SUB_CLASS

ID : COUNTER_1
  Action      : DEFINE
  Label       : Number
  Type        : NUMBER
  Generate    : ASSIGN
  ReuseNumber : NONE
  CounterKey  : Counter_A
  Length      : 6
  FirstValue  : 000000
  LastValue   : 999999
  Extends     : SOURCE_CODE
  
```

Parameter aliases

Parameter aliases

Parameter aliases is a standard mechanism for creating string values that depend on Teamcenter content. Smart Codes does not support parameter aliases for items, item revisions, or datasets.

In most cases, parameters contain the section values and string formatting values. You can use parameter aliases to get system values.

Aliases are objects whose methods properties can be used. There are IMAN-related aliases with the same properties as IMAN objects and a parameter alias with application-specific properties (with no relation to IMAN). Functions allow access to system values and formatting strings. Not all aliases are supported at all times and some of them are application-specific. All are case sensitive. Be sure classes, types, and parameter names are correct.

Alias	Content
\$I. \$Item.	Item properties
\$IM.	Item master form
\$IR. \$ItemRevision.	Item revision properties
\$IRM.	Item revision master form
\$Dataset.	Dataset properties (application-specific)
\$PartAttr.	UGPartAttr properties (application-specific)
\$SpecTag.	Additional IMAN object (application-specific)
\$User.	Current user. All Teamcenter user properties are available.
\$Group.	Current group. All Teamcenter group properties are available.
\$Param.	Additional parameters (application-specific)

Method	Content
GRM(P1[,P2])	Parameter P1 IMAN_Relation or *, if any. Note If multiple related objects are found, use the object with the lowest name. The names are case sensitive. Parameter P2 IMAN Class[/iMAN Type]. Any type if omitted.
NR(P1[,P2])	Parameter P1 Named reference. Parameter P2 IMAN Class[/iMAN Type]. Any type if omitted.

Method	Content
SELECT(P1)	Search elements relative to the current element. If multiple elements are found the first one is used. Parameter – P1 Selection expression. See Selection expression .
COUNT(P1)	Search elements relative to the current element. Returns the count of found elements. Parameter – P1 Selection expression. See Selection expression .

Function	Content		
\$SYSTEM(P1)	Use system value		
	Parameter – P1	Content	Format
	TIME	Current time	8 hex characters
	LASTTIME	Last time value	8 hex characters
	COUNTER	Counter from the start of the application	Number
	LASTCOUNTER	Last counter value	Number
\$FORMAT(P1;P2;...)	Format string P1 with formatting options P2 , P3 , etc. The options can be repeated.		
	Parameter - P1		Formatted string.
	Parameter - P2		Formatting option.
			Formats the string to valid filename. Replaces all invalid characters with underscores.
	FILENAME		
			Formats an integer value. The format specification P3 is the C-printf specification.
	INTEGER		
			Formats a double value. The format specification P3 is the C-printf specification.
	DOUBLE		
			Formats a string value. The format specification P3
	STRING		

Function	Content
	is the C-printf specification.
	Formats a date value. Value NOW in P1 means the current date/time. Value LASTTIME means the time last used by the NOW parameter. The format specification P3 is the C-printf specification.
	DATE
	UPPERCASE
	Uppercase the string.
	LOWERCASE
	Lowercase the string.
\$REPT(P1;P2)	Repeats a string n times.
	Parameter - P1 Repeated string.
	Parameter - P2 Count of repetitions.
\$LEFT(P1;P2)	Left P2 characters from P1 string.
	Parameter - P1 Truncated string.
	Parameter - P2 Count number of truncated characters from left.
\$RIGHT(P1;P2)	Right P2 characters from P1 string.
	Parameter - P1 Truncated string.
	Parameter - P2 Count number of truncated characters from right.
\$MID(P1;P2[;P3])	Middle P3 characters from P1 string, starting with P2 .
	Parameter - P1 Truncated string.
	Parameter - P2 Position where truncation starts.
	Parameter - P3 Count number of truncated characters. Up to end if omitted.

Function	Content
\$LIST(P1;P2[;P3])	P2-th part of P1 string, separated by P3. Parameter - P1 Truncated string. Parameter - P2 Position where truncation starts. The first one is position 0. Parameter - P3 Separator. if omitted.
\$RLIST(P1;P2[;P3])	P2-th last part of P1 string, separated by P3. Parameter - P1 Truncated string Parameter - P2 Position where truncation starts. The first one is position 0. Parameter - P3 Separator. if omitted.
\$PREF(P1[;P2])	Value from environment variable. Parameter - P1 Preference name or environment variable name. Parameter - P2 Preference scope {ALL USER ROLE GROUP SITE}.
\$SUBSTITUTE(P1;P2;P3[;P4])	Replace text P2 in P1 with value of P3. The change start from occurrence P4, if specified. \$SUBSTITUTE can be used as well. Parameter - P1 Source string to be modified. Parameter - P2 Substring that is replaced. Parameter - P3 Substring that is used instead of P2. Parameter - P4 The index of first occurrence of P2 to be replaced. The first P4-1 occurrences of P2 stay unchanged.

Function	Content
\$TRANS(P1[;P2;P3;P4;P5]...)[;P6])	The value in P1 is compared with P2, P4, etc. A match is replaced with P3, P5, etc. If no match is found, P6 is used. \$TRANSLATE can be used as well. Parameter - P1 Translated value. Parameter - P2,P4,... Compared value. Parameter - P3,P5,... Used value. Parameter - P6 Default value if no match. If omitted, use the P1.
\$TR(P1;P2;P3)	In the string P1 are all characters from P2 replaced with characters from P3. \$TRANSLITERATE can be used as well. Parameter - P1 Transliterated value. Parameter - P2 Set replaced of characters. Special characters are prefixed by \. \nnn represents an octal, \xnn a hexadecimal number. \[abrnftv] are translated to ASCII. Parameter - P3 Set of replacing characters. No special characters are allowed. If the set is shorter than the P2 set, the last character from P3 is used. If you want to replace all characters defined in P2 with an underscore, put only one underscore in P3.

Function	Content
\$LOVTRANS(P1;P2[;P3])	The string P1 is used as a key in the values list of the specified LOV. If there's a match, then the value description is returned. Otherwise, an empty value is returned. Parameter - P1 Translated value. Parameter - P2 Name of the translation LOV. Parameter - P3 Options (not yet supported).
\$FIRSTVALUE(P1[;[P2[...]]])	The values in P1, P2,... are scanned through. The first non-empty value is used. Parameter - P* Translated value, checked for content.
\$CLASSQUERY(P1;P2=P3)	Select object from the class P1, which attribute P2 does have value P3. Parameter - P1 Investigated class. Parameter - P2 Attribute name. Parameter - P3 Searched value. The searched value also supports parameter aliases. The defined symbols depend on the application.
\$TAG(P1)	Named tag evaluation. The available tag names are defined by the application. Parameter - P1 Tag n. For example, CURRENT_TASK.
\$CONF([-]P1[;[-]P2[...]])	Set up the parameter aliases evaluator. Parameter - P1,P2,... Configured option. If prefixed with -, the option switched out.
EVO	Empty value on empty object. When an empty object occurs in the definition, the remaining part of the definition is ignored. Otherwise, the remaining part is put in the result as a string.

Function	Content
\$MAPEXT(P1;P2)	Map string P1 with value from file P2 . Parameter - P1 Mapped value. Parameter - P2 Mapping file. The file has format map=value. The map value is not case sensitive. If the file or value are not found, the content does not change.

Examples:

- Ideas drawing exists YES/NO
`$TRANSLATE($Dataset.CONF(EVO).SELECT(GRM(IDEAS_DRAWING).CT(Dataset)).object_string;;NO;YES)`
- Translated user name (over an external file)
`$MAPEXT($User.person.user_name;%IMAN_LOCAL%\data\user_map.txt)`

Selection expression

Method	Content
GRM(P1[+P2[+...]])	Parameter P1, P2, ... Teamcenter relation or *, if any. For example, GRM(IMAN_specification+IMAN_Rendering). Note If multiple related objects are found, all of the objects are used.
PARENT(P1[+P2[+...]])	Parameter P1, P2, ... Teamcenter parent relation or *, if any. Note If multiple related objects are found, all of the objects are used.

Method	Content	
CT(P1[;P2[+P3[+...]])	Parameter P1	Teamcenter class or *, if any.
		Note If multiple related objects found, all of the objects will be used.
	Parameter P2, P3, ...	Teamcenter type. Any type if omitted. For example, CT(Item;Item + Document) .
NR(P1)	Parameter P1	Named reference
REFERENCE(P1)	Parameter P1	Reference attribute name
REFERRED_BY(P1;P2)	Parameter P1	Class of referring object
	Parameter P2	Reference attribute name
ELEMENT(P1)	Parameter P1	Index within the selected elements. 1 is the first element, 2 is the second, and so on. -1 is the last element, -2 is the second to the last element, and so on. The objects are sorted according to their object name. Non-workspace objects are placed unsorted at the end.
HASSTATUS(P1[xP2[x...]])	Parameter P1, P2, ...	Statuses that are concatenated with the “or” (x~) or “and” (x~&) operator. Operators must not be combined. For example, HASSTATUS(Released Invalidated) accepts objects with one or both of the statuses.

Method	Content	
CLASSQUERY(P1,P2,P3)	Parameter P1	Class name
	Parameter P1	Attribute name. This parameter supports MLS (multiple language support).
	Parameter P1	Attribute value. This parameter supports parameter aliases.
ORDERBY(P1[,P2])	Parameter P1	Attribute name. Must be of type date. The elements are reordered. The sort order is lost after a new parse step. The functions CT , ELEMENT , and HASSTATUS don't change the sort order.
	Parameter P1	Sort order (ASC DESC).
WHERE(P1)	Parameter P1	Evaluated expression in format V1[== != < <= > >=]V2 , where V1 and V2 are parameter aliases evaluated for each selected object.

Examples:

- Select one form where the **name_lang** content equals the **object_name** of the **\$SpecTag** object.

```
CLASSQUERY(MY_NAMES_LIST;name_lang==$SpecTag.object_name).REFERRED_BY
(Form;data_file).ELEMENT(1)
```

- Select item revisions that have at least one UGPART attached as a specification.

```
SELECT=CT(ItemRevision).WHERE($COUNT(GRM(IMAN_specification).CT(DATASET;UGPART))>0)
```

Section

The section value is accessed by the keyword **\$Param.** followed by the section ID (for example, **\$Param.ITEM_PROJECT**).

If you want to compose a string with a section value appended by text, you must use brackets (for example: **\${Param.ITEM_PROJECT}_LOV**).

A common function is to get a value of the list in the section value. Use the **\$LIST** function as follows: **\$LIST(\$Param.ITEM_PROJECT;2)** for the third value of the item project (**X** from **01|OK|X|My first project**).

Import and export configuration files

Configuration files can be imported and exported using the **smartuibldr_configure** utility. This utility is located in the *TC_ROOT\bin* directory. All imported modifications are immediately available (it is not necessary for the user to log off).

When you import a file, the contents of your configuration text file are written to the Teamcenter database. When you export a file, the information in the Teamcenter database are written to your configuration file.

For more information about the **smartuibldr_configure** utility, see the [Utilities Reference](#).

To import a configuration file, enter the following:

```
smartuibldr_configure  
-imp=drive-letter:\file-location\my-config-file.txt -u=username -p=password -g=group
```

Note If your configuration file contains **Action** parameters with a value of **DELETE**, you must import the configuration file multiple times.

To export a configuration file, enter the following:

```
smartuibldr_configure  
-exp=drive-letter:\file-location\my-config-file.txt -u=username -p=password -g=group
```

Chapter

9 *Defining queries and reports*

Defining queries and reports	9-1
------------------------------------	-----

Chapter

9 *Defining queries and reports*

Defining queries and reports

Teamcenter administrators and other privileged users create customized queries for administrative and general use. These queries can be presented as forms in the search tool or they can be used as the basis for more structured reports created using Report Builder. In addition, you can export saved queries to XML files that you can share with other Teamcenter sites.

Query Builder enables you to write complex queries based on the Teamcenter data schema, which is an hierarchical arrangement of classes, subclasses, and attributes. Query Builder provides hints to assist you in navigating the schema. The hints present a relationship as a starting point, for example, the relationship between an item and its item revision, and then provide you with the steps to build that relationship into your search definition.

Report Builder enables you to create and manage report definition templates using HTML, Excel, or XML style sheets.

Typical administrative tasks related to building queries and formatting reports include:

- Defining the requirements for your queries.
- Navigating the schema and defining search clauses to create queries that match your requirements.
- Creating property finder formatter (PFF) objects that allow you to gather and report data related to the objects returned by a query.
- Maintaining the XML file containing the hints used to navigate the schema in Query Builder.
- Importing and exporting search definitions.
- Defining and managing report definition templates.

For more information, see the [Query Builder Guide](#) and the [Report Builder Guide](#).

Chapter

10 Configuring Teamcenter mail

Configuring Teamcenter mail	10-1
Configure Teamcenter to work with external e-mail for subscription notices . . .	10-1

Chapter

10 *Configuring Teamcenter mail*

Configuring Teamcenter mail

Administrators can configure Teamcenter mail to:

- Specify the mail gateway server, port, and character set used to send mail.
- Allow e-mail addresses to exceed 32 characters in length.
- [Notify users of appearance updates.](#)
- [Notify users of important schedule dates, milestones, and task completion.](#)
- Enable and disable the capability to send operating system e-mail from within Teamcenter.
- Specify the interval at which Teamcenter checks for new mail.
- Work with external e-mail for subscription notices.

For more information, see the [Preferences and Environment Variables Reference](#).

Configure Teamcenter to work with external e-mail for subscription notices

To use external (operating system) e-mail for subscription and workflow notification, you must:

- Set mail gateway preferences:
 - o **TC_subscription=ON** to enable display and use of the **Tools® Subscribe** menu command.
 - o **Mail_server_name=a_valid_SMTP_mail_server**

To view notification e-mail sending records, set **TC_audit_manager=ON**, **TC_audit_manager_version=3**, and **SCM_notification_history=true**.

For information about mail gateway preferences, see the [Preferences and Environment Variables Reference](#).

- For the **Person** objects associated to users to be notified, ensure the **E_Mail** address fields are set correctly in the Organization application.

For information about creating and editing **Person** objects, see the [Organization Guide](#).

- Start the **subscriptionmgrd** subscription monitor process daemon.

For information about configuring and starting the **subscriptionmgrd** process daemon, see the [Subscription Monitor Guide](#).

Chapter

11 Configuring instant messaging

Configuring instant messaging	11-1
---	------

Chapter

11 *Configuring instant messaging*

Configuring instant messaging

Teamcenter rich client and thin client users use Microsoft Office Communicator to see when other users are available for instant messaging.

Teamcenter users can view the current status of the owning and last modified users, and, from within a Teamcenter application, can initiate communication using Microsoft Office Communicator.

This capability:

- Is supported only on Windows systems with Microsoft Office Communicator 2007.
- Lets you access all available Microsoft Office Communicator features.
- Is available in Teamcenter and is enabled and controlled by preference settings.
 - o The **OCS_use_presence_display** preference specifies whether the functionality is integrated with Teamcenter.
 - o The **OCS_use_email_property** preference specifies whether to use the e-mail ID from the e-mail ID field of the user **Person** object.
 - o The **OCS_company_email_domain** preference value specifies a domain name if the **OCS_use_email_property** preference value is **false**.
- For the rich client, integration is implemented in:
 - o My Teamcenter **Summary** view
 - o My Teamcenter **Viewer** view
 - o Workflow **perform-signoffs** panes
 - o Systems Engineering
 - o Structure Manager
- For the thin client, integration is supported only for Microsoft Internet Explorer and is implemented in:
 - o My Teamcenter **Summary** page
 - o Workflow **perform-signoffs** dialog boxes

Note

Microsoft Office Communicator must be installed and running on your computer to use the Teamcenter integration with Microsoft Office Communicator.

- If you try to use this feature when Microsoft Office Communicator is not installed and running on your computer, Teamcenter displays the symbol color corresponding to the Microsoft Office Communicator **Presence Unknown** status.
- If you try to use the Teamcenter integration with Microsoft Office Communicator to contact a user for whom the system cannot find an e-mail ID, the system displays a message listing possible reasons for the inability to continue the communication.

Chapter

*12 Configuring standard notes,
custom notes, and parametric
requirements*

Configuring standard notes, custom notes, and parametric requirements 12-1

Configure standard note and parametric requirement creation privileges 12-1

Set the delimiter that separates parameters in parametric requirement text . . 12-2

Enable multiple revisions of a parametric requirement to be attached to an item or
item revision 12-2

Configure the behavior of custom notes 12-2

Chapter

12 *Configuring standard notes, custom notes, and parametric requirements*

Configuring standard notes, custom notes, and parametric requirements

Standard note or parametric requirement objects are used to provide information about parts and documents that is based on the established practices of a design program. The following aspects of standard note and parametric requirement behavior can be configured by a Teamcenter administrator:

- Creation privileges
Users must fill a role specified in the **Ads0CreateStandardNoteAuthority** list of values or have DBA privileges with bypass set to be able to create a standard note or parametric requirement.
- Delimiter between parameters in note text
- Ability to attach multiple revisions of a standard note or parametric requirement to an item or item revision

Configure standard note and parametric requirement creation privileges

Standard notes and parametric requirements can only be created by privileged users. You can specify the roles that are allowed to create notes and requirements and then specify access privileges based on these roles.

To specify the roles that are allowed to create standard notes and parametric requirements, add the roles to the **Ads0CreateStandardNoteAuthority** list of values. You can then grant access privileges based on those roles.

For more information about updating the list of values, see the [Business Modeler IDE Guide](#).

For information about applying access privileges, see the [Access Manager Guide](#).

Set the delimiter that separates parameters in parametric requirement text

A standard note or parametric requirement contains:

- Text
- A parameter and applicable values, using the following syntax:

```
text [parameter name: parametric value1 delimiter parametric  
value2 delimiter ..... parametric value n]
```

The default delimiter separating the parameters in the note is a comma (,). To change the delimiter, set the **Fnd0ParamReqDelimiter** global constant.

For more information about setting global constants, see the [Business Modeler IDE Guide](#).

Enable multiple revisions of a parametric requirement to be attached to an item or item revision

Standard notes and parametric requirements can be attached to an item or item revision. By default, only one revision of a standard note or parametric can be attached to an item or item revision. You cannot attach multiple revisions of the same note or requirement to a single item or item revision.

However, you can enable multiple revisions of the same standard note or parametric requirement to be attached to an item or item revision by setting the **AllowMultipleRevisionsofStdNotes** global constant.

For more information about setting global constants, see the [Business Modeler IDE Guide](#).

Configure the behavior of custom notes

Custom notes provide unique information about an individual part or document, and they can be attached to an item or item revision. The following behavior can be configured for custom notes:

- By default, a custom note item can be attached to a single item or multiple revisions of the same item. A custom note item cannot be attached to multiple items or to revisions of multiple items.

To enable a custom note item to be attached to multiple items or revisions of multiple items, set the **Fnd0AttachCustomNoteToMultiItems** business constant to **true**.

- By default, multiple revisions of a custom note cannot be attached to an item or item revision.

To enable multiple revisions of a custom note to be attached to an item or item revision, set the **Fnd0AllowMultipleRevofCustomNote** global constant to **true**.

For more information about setting global constants, see the [Business Modeler IDE Guide](#).

Chapter

13 *Configuring document management*

Configuring document management	13-1
Working with document management templates	13-1
Create document management templates in the rich client	13-1
Attach datasets to a document management template in the rich client	13-2
Create document management templates in the thin client	13-2
Attach datasets to a document management template in the thin client	13-2
Releasing document management templates	13-3
Release a document management template and attached datasets in the rich client	13-3
Release a document management template and attached datasets in the thin client	13-3
Creating item revision definition configurations (IRDCs)	13-3
Creating dispatcher service configurations	13-4
Configuring UNIX printers for batch printing	13-4
Defining printer device and page specifications for UNIX printing	13-5
UNIX configuration file example	13-8

Chapter

13 *Configuring document management*

Configuring document management

Document management functionality controls system behavior at specific times in the life cycle, such as at item creation, checkin, checkout, delete, save as, and revise. Managed behaviors include:

- Creating and naming source and derived visualization datasets.
- Creating specified relations.
- Applying markup rules.

Document management functionality must be configured by [creating document management templates](#) in Teamcenter and then creating [item revision definition configurations \(IRDCs\)](#) and [dispatcher service configurations](#) in the Business Modeler IDE.

Working with document management templates

Document management templates provide a mechanism for establishing standard documents to be used for a project.

Multiple datasets can be attached to a single document management template. The templates are associated with revisions using an item revision definition configuration (IRDC).

When a user creates a revision that is controlled by an IRDC, the template files (for example, Microsoft Word documents or Excel spreadsheets) are copied for the new revision and the copied files can be downloaded to a local directory by selecting the **Check Out Item Revision On Create** option during revision creation.

Note Document management template revisions and the attached datasets must be in released status before they can be used in an IRDC. Objects created from IRDC templates are new documents that by default do not have a release state.

Create document management templates in the rich client

1. Choose **File**→**New**→**Item**.
2. In the **New Item** dialog box, select **DMTemplate** from the **Select Type** list.

3. Click **Next**.
4. Type an item ID, revision, and name for the item, or click **Assign** to automatically generate the item ID and revision identifiers.
5. (Optional) In the **Description** box, type a description of the document management template.
6. Click **Finish**.

Attach datasets to a document management template in the rich client

1. [Create a dataset](#) and import the native document file, such as a Microsoft Word document or Excel spreadsheet, into the dataset.
2. Copy and paste or drag the dataset in to the document management template revision.
3. (Optional) Repeat the previous steps to associate additional datasets with the document management template revision.

Create document management templates in the thin client

1. Choose **New→Item**.
2. Select **DMTemplate** from the **Type** list.
3. Click **Next**.
4. In the **ID** box and **Rev ID** box, type an identifier and revision for the document management template.

Note To have the system assign the identifiers, leave the **ID** and **Rev ID** boxes blank.

5. In the **Name** box, type a name for the document management template.
6. (Optional) In the **Description** box, type a description of the document management template.
7. Click **Finish**.

Attach datasets to a document management template in the thin client

1. [Create a dataset](#) and import the native document file, such as a Microsoft Word document or Excel spreadsheet, into the dataset.
2. Cut or copy and paste the dataset in to the document management template revision.
3. (Optional) Repeat the previous steps to associate additional datasets with the document management template revision.

Releasing document management templates

To use a document management template in an item revision definition configuration (IRDC), the template and attached datasets must be in released status. You can use the **TCM Release Process** workflow process template to apply release status.

For more information about applying release status using the **TCM Release Process** workflow process template, see [Release a document management template and attached datasets in the rich client](#).

In addition, you can design your own workflow process template using Workflow Designer.

For more information about creating workflow process templates, see the [Workflow Designer Guide](#).

Release a document management template and attached datasets in the rich client

1. In My Teamcenter, select the document management template revision.
2. Choose **File**→**New**→**Workflow process**.
3. In the **New Process** dialog box, select **TCM Release Process** from the **Process Template** list.
4. Click the **Attachments** tab.
5. Expand the document template revision and select the datasets to be released.
6. Click **OK**.

Release a document management template and attached datasets in the thin client

1. In My Teamcenter, select the document management template revision.
2. Choose **New**→**Workflow Process**
3. Type the name of the workflow process in the **Process Name** box.
4. (Optional) Type a description of the workflow process in the **Description** box.
5. Select **TCM Release Process** from the **Process Template** list.
6. Click **OK**.

Creating item revision definition configurations (IRDCs)

An item revision definition configuration (IRDC) defines how an item revision is handled. IRDCs standardize item revision behavior at specific times in the life cycle, such as at item creation, checkin, checkout, save as, and revise. Create an IRDC for every item revision business object for which you want standardized behavior.

When you create an IRDC, you:

- Specify the item revision object to which the IRDC applies.

- Specify the conditions under which the IRDC applies.
- Select a template.
- Specify IRDC dataset criteria.
- Specify IRDC check-in behavior.
- Define deep copy rules for the item revision.

For more information about creating item revision definition configuration objects, see the [Business Modeler IDE Guide](#).

Creating dispatcher service configurations

A dispatcher service configuration is an object that defines the visualization file format that a dataset file is translated into. For example, it may specify that CAD design files are translated into 3D visualization files or that Microsoft Word documents are translated into PDF files. Dispatcher service configurations are used by item revision definition configurations (IRDCs).

For more information, see the [Business Modeler IDE Guide](#).

Configuring UNIX printers for batch printing

You can print to any printer specified in the UNIX `/app_defaults/vvcp.operating-system.cfg` configuration file. You can configure UNIX printers by modifying printer device and page specification settings in the ***Print** section of the configuration file.

This program supports several printers, including those described in the following table.

Printer type	Description	Comments
PostScript	Postscript printers	Black and white, and color.
HPGL	HPGL Plotter	4 to 256 pen colors; vector and raster.
Calcomp907	Calcomp 907 Pen Plotter	256 colors; vector only.
Calcomp951	Calcomp 951 Electrostatic Plotter	256 colors; vector and raster.
TIFF	Group 4 TIFF-based printers (For example, 3M LBQ.)	Black and white; color TIFF.

In addition:

- You can use standard paper size mnemonics to configure output page size.
- HPGL output sizes and coordinates are supported.
- All formats except Calcomp 907 support raster-based output.

- When you print using the **Print** command from a UNIX system, the software creates a default margin of 0.5 inch.

Defining printer device and page specifications for UNIX printing

You can use the ***Printers:** section of the `/app_defaults/vvcp.operating-system.cfg` configuration file to configure UNIX printers and plotters and to define page specifications.

Configure printers syntax

```
printername DEVICE printertype {,density {,margin} ,other} \n\
```

Configure printers arguments

The arguments in the following table can be used to configure UNIX printers for batch printing.

Argument	Purpose
printername	Identifies the printer from the command line.
printertype	Specifies the data accepted by the defined printer.
density	Specifies the default density in pixels-per-inch format.
	Note This setting is supported by raster and PostScript printers.
margin	Specifies the margin for each page.
	Note Printer hardware preferences often set the margin value. Check your printer documentation to determine if margin settings can be set using software preferences.
Other	Include any other printer specifications provided by your printer.
Units	Specifies the unit of measure for the coordinate system.
	Note This setting supports inches or centimeters. If omitted, the default setting is inches.

Examples

```
printername DEVICE HPGL \n\

PS1 DEVICE Postscript,300,0.5,0,0,0,0 in \n\
PS2 DEVICE Postscript,300,1.27,0,0,0,0 cm \n\
HP1 DEVICE HPGL \n\
```

Define pages syntax

```
printername PAGE sizespec {text}{,window-coordinates}printcommand
\n\
```

Define pages arguments

The arguments in the following table can be used to define page specifications.

Argument	Purpose
printername	Identifies the printer from the command line. Tip Each printer or plotter can have one or more page specifications.
sizespec	Specifies the mnemonic page size. Note Supported mnemonics include: {A B C D E J A4 A3 A2 A1 A0}
Text	Specifies comment text. Note Spaces are supported; commas are not.
printer coordinates	Lists the printer coordinates from the lower-left corner to the upper-right corner of the drawing using the printer coordinate system (llx , lly , urx , ury). Coordinates must be separated by commas.
printcommand	Specifies printer commands to create printed output. Note Enclose the entire string in double quotation marks (“ ”). If quotation marks are required within the string, use single quotation marks (‘ ’). In addition, you can use an exclamation mark (!) to pipe output into a UNIX command.

Notes

- You can use sample entries as guides to changing your printer and page setup.
- For PostScript and raster-based printers, if the printer uses the lower-left corner as the origin, the first two values should be **0,0** (default value). To calculate the upper-right corner value, multiply the sheet size by the density.
- HPGL-based printers use printer logical units (PLUs) to measure the coordinate system. Each PLU is 1/40 mm; therefore, there are 1016 PLUs per inch.
- Several HPGL printers place the origin in the middle of the paper. This defines the lower-left and upper-right corners to be symmetric around the origin (**0,0**). For example, the entry for an A-sized sheet for an HP7580 is **[-4318,-5588,4318,5588]**.

Examples

- This example defines a PostScript printer with a density of 300 dpi and a margin of 0.5 inch.

To define a PostScript printer with a density of 300 dpi and a margin of 0.5 inch, type the following line in the configuration file:

```
PS1 DEVICE Postscript,300,0.5,0,0,0,0 in \n\
```

To create a portrait A-sized page definition, type the following line in the configuration file:

```
PS1 PAGE A (8.5x11 in),0,0,2550,3300 "!lpr" \n\
```

Note The page definition is determined by multiplying the A-size dimensions by the density (8.5 X 11) * (300).

x-axis

x dimension=2550 pixels
(8.5 in. * 300 dpi)

y-axis

y dimension=3300 pixels (11 in. * 300 dpi)

The first line in the example configuration file shows a defined printer (**DEVICE**) and the second line shows a defined page size (**PAGE**).

```
PS1 DEVICE Postscript,300,0.5,0,0,0,0 in \n\
PS1 PAGE A (8.5x11 in),0,0,2550,3300 "!lpr" \n\
```

- This example specifies an HPGL printer that has an automatic density setting equal to printer logical units (PLUs) and a beginning origin in the lower-left corner.

To specify the HPGL printer, type the following line in the configuration file:

```
HP1 DEVICE HPGL \n\
```

To create a page definition specifying a portrait A-sized sheet with its origin in the lower-left corner, type the following line in the configuration file:

```
HP1 PAGE A (8.5x11 in),0,0,8636,11176 "!lpr" \n\
```

Note The page definition is determined by multiplying the A-sized dimension by the PLUs. You may find it easier to perform the calculations if you convert the dimension to millimeters.

x-axis

x dimension=8636
(8.5 in. * 25.4) * 40PLU

y-axis

y dimension=11176
(11 in. * 25.4) * 40PLU

The first line in the example configuration file shows a defined printer and the second line shows a defined page size.

```
HP1 DEVICE HPGL \n\
HP1 PAGE A (8.5x11 in),0,0,8636,11176 "!lpr" \n\
```

UNIX configuration file example

The following example shows printer settings in the UNIX configuration file:

```

! =====
! PRINTER SETTINGS
! =====
! [Printers]
! Set unix printer configuration information !
! NOTES:
! 1) To use a printer on unix, you must either use the standard mechanism
! (which requires setting up the following configuration) or one of the
! alternative mechanisms (please see the documentation for more information
! on these).
! 2) THE FOLLOWING CONFIGURATIONS INCLUDE SAMPLE VALUES, BUT MAY NOT BE
! APPROPRIATE FOR YOUR ENVIRONMENT. PLEASE FOLLOW THE INSTRUCTIONS IN THE
! MANUAL TO ADJUST THESE VALUES BEFORE CALLING CUSTOMER SUPPORT.
! 3) The printer section should be set up as follows:
! -----
! -----
! lp-device DEVICE POSTSCRIPT [density, margin,x-offset,y-offset],-1,-1 in \n\
! lp-device PAGE A (8.5x11 in),x-offset,y-offset,width,height [file/device] \n\
! OR
! lp-device DEVICE HPGL \n\
! lp-device PAGE A (8.5x11 in),lx,ly,ux,uy \n\
! -----
! -----
Preview*plotters: \
Std-Post DEVICE POSTSCRIPT,300,0,0,0,-1,-1 in \n\
Std-Post PAGE A (8.5x11 in),0,0,2550,3300 "!lp -d<device>" \n\
Std-Post PAGE B (11x17 in),0,0,3300,5100 "!lp -d<device>" \n\
Std-Post PAGE C (17x22 in),0,0,5100,6600 "!lp -d<device>" \n\
Std-Post PAGE D (22x34 in),0,0,6600,10200 "!lp -d<device>" \n\
Std-Post PAGE E (34x44 in),0,0,10200,13200 "!lp -d<device>" \n\
Eur-HPGL DEVICE HPGL \n\
Eur-HPGL PAGE A4 (21x29.7 cm),0,0,8400,11880 "!lp
-d<device>" \n\
Eur-HPGL PAGE A3 (29.7x42 cm),0,0,11880,16800 "!lp -d<device>" \n\
Eur-HPGL PAGE A2 (42x59.4 cm),0,0,16800,23760 "!lp -d<device>" \n\
Eur-HPGL PAGE A1 (59.4x84.1 cm),0,0,23760,33640 "!lp -d<device>" \n\
Eur-HPGL PAGE A0 (84.1x129.7 cm),0,0,33640,51880 "!lp -d<device>" \n\

```

Chapter

14 *Configuring Microsoft Office templates*

Configuring Microsoft Office templates	14-1
Microsoft Office Word export templates	14-1
Specification templates	14-2
Specification template layout	14-3
Specification template title page	14-3
Specification template table of contents	14-4
Specification template headers and footers	14-4
Specification template styles	14-4
Specification template body contents	14-5
Create a specification template	14-6
Import a specification template	14-7
Modify a specification template	14-7
Copy a specification template	14-7
Delete a specification template	14-8
Object templates	14-8
Object template without rules	14-10
Object template with rules	14-10
Create an object template	14-10
Import an object template	14-11
Modify an object template	14-11
Copy an object template	14-12
Delete an object template	14-12
Content rules for specification templates and object templates	14-12
Property keyword syntax in specification templates and object templates	14-13
Macro capability in Microsoft Office Word templates	14-13
Associating export templates with business objects	14-14
Microsoft Office Excel export templates	14-14
Multiple sheets in Excel export templates	14-14
Cell formatting in Excel export templates	14-15
Excel export template rules	14-15
General guidelines for Microsoft Office Excel export templates	14-16
Key fields in the rule table	14-17
Property keywords in Excel export templates	14-17
Using transfer modes in Excel export templates	14-18
Rule processing	14-18
Packing multiple objects	14-18
Object packing in Excel export files	14-18

Apply packing to an Excel template	14-20
Create an Excel export template	14-20
Modify an Excel export template	14-21
Copy an Excel export template	14-21
Import an Excel export template spreadsheet	14-21
Delete an Excel export template	14-22
Insert macros in custom Microsoft Office Excel templates	14-22
Further configure Microsoft Excel Template exports	14-22

Chapter

14 *Configuring Microsoft Office templates*

Configuring Microsoft Office templates

Export templates define what information is exported from the Teamcenter database and how the information is formatted when exported to a Microsoft Word document or Microsoft Excel spreadsheet. Exported data can be sent to a static document or live document. Exported data can also be sent to a static Excel spreadsheet or live Excel document. Data sent to a live document can be edited and saved to the Teamcenter database in real time, in most, but not all cases.

You can configure the following types of export templates:

- **Specification export templates**

Microsoft Word files control document format when you export a Teamcenter structure from Systems Engineering.

Note **Closure rules** can be used to control exported content with use of **Transfer rules**.

- **Object export templates**

Microsoft Word files that control the content and property format, and the specific properties which are exported, when you export Teamcenter objects.

- **Excel export templates**

Microsoft Excel files that define the property data and the output format that is used when you export objects from Teamcenter to Excel. Property data can be exported to live Excel or to a static Excel spreadsheet.

Note Microsoft Word 2007 or later is required to generate specification and object export templates. Microsoft Excel 2007 or later is required to use live Excel.

Microsoft Office Word export templates

Export templates define what information is exported from the Teamcenter database and how the information is formatted when exported to a Microsoft Office Word document. Exported data can be sent to a static document or live document.

- Specification export templates are Word files that control document format when you export a Teamcenter structure from Systems Engineering.

- Object export templates are Word files that control the content and property format when you export Teamcenter objects.

Specification templates

Note Microsoft Office Word 2007 or 2010 is required for creating and modifying specification templates.

A *specification template* is a Microsoft Office Word document that controls the content and format of the data that is exported to Word.

- Defines the overall layout and content of the Word output document.

Caution Although specification templates can have landscape orientation, that page layout is ignored by the **Live integration with Word** output mode. The page layout is portrait orientation in all live Word documents.

For more information, see [Live integration with Word output mode](#).

- Can be associated with specific business object types using an item revision definition configuration (IRDC) object.
- Can be selected manually when you export data.
- Can be viewed and edited in Word or in the Teamcenter embedded viewer.

By maintaining the same style set in both the specification template and the object template, you can apply identical styles to edited object content and to Word export documents. Ensure that the style names and attributes match in both templates.

Note Styles in the specification template override those with the same name but different attributes in the object template.

When you edit object content in Word, you can format body text using both Word direct formatting features and Word styles. Body text content formatting supersedes the styles in specification templates and object templates.

To achieve that formatting in Word export documents, ensure that:

- In the object template, the style applied to the **{%Body_Text}** tag has the same attributes as the body text content.
- In the specification template, the style name and attributes match those in the object template.

SpecTemplate items and **SpecTemplateRevision** item revisions are Teamcenter data objects that act as containers for the **FullText** dataset. The named references of the **FullText** dataset are XML files that represent the contents of the Word file.

For more information about item revision definition configuration (IRDC) objects, see the [Business Modeler IDE Guide](#).

The following business object types are supported:

- **Specification** item objects and subtypes, such as **RequirementSpecification** objects.

- **SpecElement** item objects and subtypes, such as **Requirement** and **Paragraph** objects.
- **FullText** datasets and subtypes, such as **Note** datasets.
- **Tracelink** objects and subtypes.

Note When exporting any requirement information to Microsoft Word 2007, you must create the specification template in **Compatibility Mode**. When you are in compatibility mode, the program runs using settings from a previous version of Windows. For information about creating documents using compatibility mode, see [Create a document to be used by previous versions of Word](#).

Specification template layout

Caution Landscape orientation in specification templates is ignored by the **Live integration with Word** output mode. The page layout is portrait orientation in all live Word documents.

For more information, see the [Systems Engineering Guide](#).

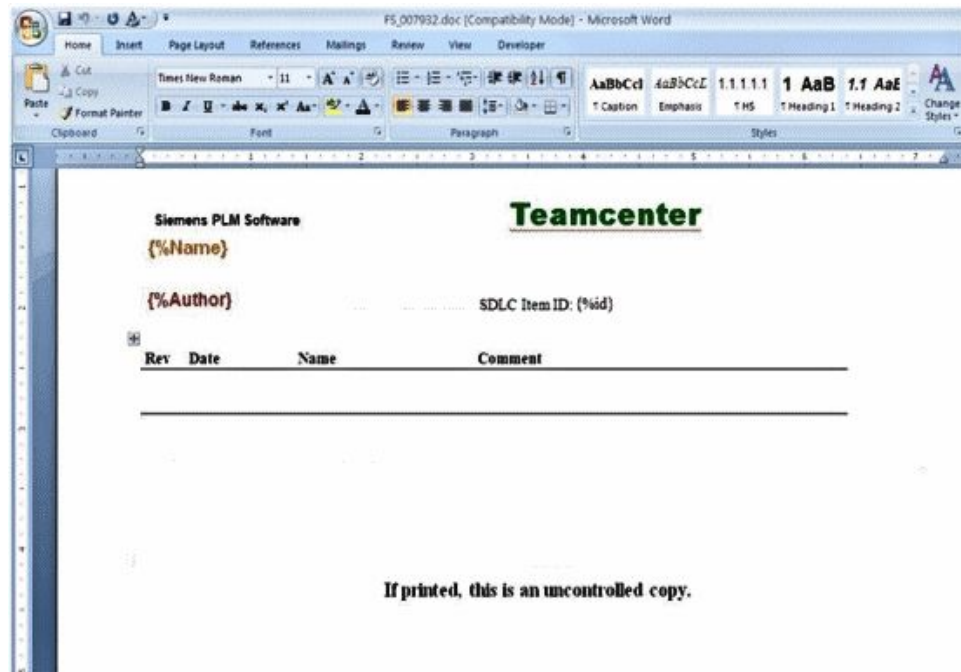
A specification template contains the following elements of Microsoft Office Word:

- Style and format definition
- Title page
- Table of contents
- Header and footer information
- Object export conditions
- Object export structure

Specification template title page

The title page of a specification template can be a combination of static and dynamic text. Title pages often include:

- Document title derived from a Teamcenter property value identified by a [keyword](#).
- Copyright and trademark information.
- Author, project ID, and revision history extracted from the Teamcenter database using a [keyword](#).



Specification template table of contents

Tables of contents are added to the specification template using the **Table of Contents** options on the **References** tab in Microsoft Word 2007.

You can specify the number of heading levels and choose a format for the table of contents.

For more information, see *Microsoft Word Help*.

Specification template headers and footers

Headers and footers can be defined for each section of the specification template. You can define:

- First page headers and footers.
- Odd page headers and footers.
- Even page headers and footers.

Headers and footers are added to the Word document using the **Header** and **Footer** options in the **Header & Footer** group on the **Insert** tab in Microsoft Word 2007.

You can also add footnotes and endnotes to the specification template.

For more information, see *Microsoft Word Help*.

Specification template styles

Specification templates use standard Microsoft Word style tools. You can define:

- Paragraph styles.
- Character styles.

- Link styles.
- Numbering styles.
- Table styles.
- Default document paragraph and character properties.
- Style inheritance.
- Style application.
- Latent styles.

By maintaining the same style set in both the specification template and the object template, you can apply identical styles to edited object content and to Word export documents. Ensure that the style names and attributes match in both templates.

Note Styles in the specification template override those with the same name but different attributes in the object template.

When you edit object content in Word, you can format body text using both Word direct formatting features and Word styles. Body text content formatting supersedes the styles in specification templates and object templates.

To achieve that formatting in Word export documents, ensure that:

- In the object template, the style applied to the **{%Body_Text}** tag has the same attributes as the body text content.
- In the specification template, the style name and attributes match those in the object template.

Specification template body contents

The body content of the specification template contains free-form text and information extracted from the Teamcenter database using keywords. Body content can reflect:

- A static structure exported from the Systems Engineering application.
- A dynamic structure generated using transfer modes and conditions in transfer modes.

Static structure in a specification template

```
Company Name
{%Name}

Requirement Specification
{%Author}

Table of Contents
1. Introduction
2. Market Challenges
3. Architecture Overview

{%Rule:Selection.Export.Static|}
```

For more information, see [Property keyword syntax in specification templates and object templates](#) and [Content rules for specification templates and object templates](#).

Dynamic structure in a specification template

```
Company Name
{%Name}

Requirement Specification
{%Author}

Table of Contents
1. Introduction
2. Market Challenges
3. Architecture Overview

{%Rule:Selection.Export.TM.My-Transfer-Mode}
```

For more information, see [Property keyword syntax in specification templates and object templates](#) and [Content rules for specification templates and object templates](#).

Create a specification template

1. In My Teamcenter, choose **File**→**New**→**Item**.
2. Select **SpecTemplate** from the list of item types, and then click **Next**.
3. Type an item ID and revision ID in the **Item ID** and **Revision ID** boxes or click **Assign** to automatically assign the IDs.
4. Type a name for the specification template in the **Name** box.

You can type an optional description in the **Description** box.

5. Click **Finish**.

The **SpecTemplate** item, **SpecTemplateRevision** item revision, and the associated **FullText** dataset are created in the **SpecTemplates** folder, which is located in the **RequirementsManagement Templates** folder in your **Home** folder.

6. (Optional) To update the specification template content, double-click the **FullText** dataset.

The template opens in Microsoft Office Word.

7. (Optional) Modify the content, save the Word document, and close Word.

Import a specification template

1. In My Teamcenter, choose **Tools**→**Import**→**Templates**→**Specification Template**.
2. In the **Import Specification Template** dialog box, click **Browse** and locate the Word file (**.docx**) containing the template you want to import.
3. Select the file and click **Save**.
4. (Optional) Type a description in the **Template Description** box.

5. Click **OK**.

The template is imported in to the Teamcenter database.

6. Click **OK** in the **Import Template** dialog box.

The **SpecTemplate** item, **SpecTemplateRevision** item revision, and the associated **FullText** dataset are created in the **SpecTemplates** folder, which is located in the **RequirementsManagement Templates** folder in your **Home** folder.

Modify a specification template

1. In My Teamcenter, select the **FullText** dataset associated with the **SpecTemplateRevision** item revision out of the database.
2. Double-click the **FullText** dataset to open it in Microsoft Office Word.
3. Edit the content of the Word document.
4. Save the file in Word.
5. Close the file and check it in to the Teamcenter database.

Copy a specification template

1. In My Teamcenter, select the **SpecTemplate** item containing the template that you want to duplicate, and choose **File**→**Save As**.
2. Click **Assign** in the **Item Details** pane of the **Save Item As** dialog box.

The system assigns an item ID and revision ID for the new item.

3. Type a name for the new item in the **Name** box.
You can type an optional description in the **Description** box.
4. (Optional) Select the **Open on Create** check box to open the item after it is created.
5. Click **OK**.

The new **SpecTemplate** item and **SpecTemplateRevision** are created. The **FullText** dataset is copied as a new object of the same name as the original dataset.

6. (Optional) Rename the **FullText** dataset.

Delete a specification template

1. Ensure that the **SpecTemplate** item is not referenced by other objects in the Teamcenter database.

For more information about performing where-referenced searches, see [Rich Client Interface Guide](#).

2. In My Teamcenter, select the **SpecTemplate** item.
3. Choose **Edit**→**Delete**.
4. Click **Yes** in the **Delete** dialog box.

The **SpecTemplate** item, **SpecTemplateRevision** item revision, and **FullText** dataset are deleted from the database.

Object templates

Note Microsoft Office Word 2007 or 2010 is required for creating and modifying object templates.

Object templates control the content and format when you export Teamcenter objects to Word. Object templates are Word documents that:

- Use *property keywords* to define which properties of the selected objects are exported.

Warning In an object template that is intended for export to the **Live integration with Word** output mode, the **{%object_name}** property keyword is required. Otherwise, structure data cannot be edited in live Word.

For more information, see [Property keyword syntax in specification templates and object templates](#) and the [Systems Engineering Guide](#).

- Specify the format of the exported property information.
For example, you can specify that the properties are exported in tabular format.
- Specify static text to be included in the export document.

- Can specify *content rules* that determine which objects are exported to the output document.

For more information, see [Content rules for specification templates and object templates](#).

- Can be associated with specific business objects types using an item revision definition configuration (IRDC) object.
- Can be selected manually when you export data.
- Can be viewed and edited in Word or in the Teamcenter embedded viewer.

By maintaining the same style set in both the specification template and the object template, you can apply identical styles to edited object content and to Word export documents. Ensure that the style names and attributes match in both templates.

Note Styles in the specification template override those with the same name but different attributes in the object template.

When you edit object content in Word, you can format body text using both Word direct formatting features and Word styles. Body text content formatting supersedes the styles in specification templates and object templates.

To achieve that formatting in Word export documents, ensure that:

- In the object template, the style applied to the **{%Body_Text}** tag has the same attributes as the body text content.
- In the specification template, the style name and attributes match those in the object template.

ObjectTemplate items and **ObjectTemplateRevision** item revisions are Teamcenter data objects that act as containers for the **FullText** dataset. The named references of the **FullText** dataset are XML files that represent the contents of the Word file.

For more information about item revision definition configuration (IRDC) objects, see the [Business Modeler IDE Guide](#).

Object template without rules

{%Name}

{%Body_Text}

This requirement is assigned to {%Assigned To}

The status of this requirement is {%Requirement Status}

The criticality level of this requirement is {%Criticality}

The following table summarizes information about the requirement:

Assigned To	Requirement Status	Criticality
{%Assigned To}	{%Requirement Status}	{%Criticality}

For more information, see [Property keyword syntax in specification templates and object templates](#) and [Content rules for specification templates and object templates](#).

Object template with rules

{%object_name}

Property	Value
Type	{%object_type}
Object Description	{%object_desc}
last_mod_date	{%last_mod_date}

{%body_text}

{%RULE:SELECTION.EXPORT.TM.My_Transfer_Mode}

Warning

The {%object_name} property keyword is required in object templates that are intended for export to the **Live integration with Word** output mode. Without this keyword in the template, such a structure cannot be edited in live Word.

For more information, see [Property keyword syntax in specification templates and object templates](#) and [Content rules for specification templates and object templates](#).

Create an object template

1. In My Teamcenter, choose **File**→**New**→**Item**.
2. Select **ObjectTemplate** from the list of item types, and then click **Next**.
3. Type an item ID and revision ID in the **Item ID** and **Revision ID** boxes, or click **Assign** to automatically assign the IDs.

4. Type a name for the object template in the **Name** box.

You can type an optional description in the **Description** box.

5. Click **Finish**.

The **ObjectTemplate** item, **ObjectTemplateRevision** item revision, and associated **FullText** dataset are created in the **ObjectTemplates** folder, which is located in the **RequirementsManagement Templates** folder in your **Home** folder.

6. (Optional) To update the object template content, double-click the **FullText** dataset.

The template opens in Word.

7. (Optional) Modify the content, save the Word document, and close Word.

Warning

If the object template is intended for export to the **Live integration with Word** output mode, the **{%object_name}** property keyword is required. Otherwise, structure data cannot be edited in live Word.

For more information, see [Property keyword syntax in specification templates and object templates](#).

Import an object template

1. In My Teamcenter, choose **Tools**→**Import**→**Templates**→**Object Template**.
2. In the **Import Object Template** dialog box, click **Browse** and locate the Word file (**.docx**) containing the template you want to import.
3. Select the file and click **Save**.
4. (Optional) Type a description in the **Template Description** box.
5. Click **OK**.

The template is imported in to the Teamcenter database.

6. Click **OK** in the **Import Template** dialog box.

The **ObjectTemplate** item, **ObjectTemplateRevision** item revision, and associated **FullText** dataset are created in the **ObjectTemplates** folder, which is located in the **RequirementsManagement Templates** folder in your **Home** folder.

Modify an object template

1. In My Teamcenter, check the **FullText** dataset associated with the **ObjectTemplateRevision** item revision out of the database.
2. Double-click the **FullText** dataset to open it in Microsoft Word.
3. Edit the content of the Word document.

Warning

If the object template is intended for export to the **Live integration with Word** output mode, the **{%object_name}** property keyword is required. Otherwise, structure data cannot be edited in live Word.

For more information, see [Property keyword syntax in specification templates and object templates](#).

4. Save the file in Word.
5. Close the file and check it in to the Teamcenter database.

Copy an object template

1. In My Teamcenter, click the **ObjectTemplate** item containing the template that you want to duplicate, and choose **File→Save As**.
2. Click **Assign** in the **Item Details** pane of the **Save Item As** dialog box.
The system assigns an item ID and revision ID for the new item.
3. Type a name for the new item in the **Name** box.
4. (Optional) Type a description of the new template item in the **Description** box.
5. (Optional) Select the **Open on Create** check box to open the item after it is created.
6. Click **OK**.

The new **ObjectTemplate** item and **ObjectTemplateRevision** are created. The **FullText** dataset is copied as a new object of the same name as the original dataset.

7. (Optional) Rename the **FullText** dataset.

Delete an object template

1. Ensure that the **ObjectTemplate** item is not referenced by other objects in the Teamcenter database.

For more information about performing where-referenced searches, see the [Rich Client Interface Guide](#).

2. In My Teamcenter, select the **ObjectTemplate** item.
3. Choose **Edit→Delete**.
4. Click **Yes** in the **Delete** dialog box.

The **ObjectTemplate** item, **ObjectTemplateRevision** item revision, and **FullText** dataset are deleted from the database.

Content rules for specification templates and object templates

Two types of rules can be specified in specification export template documents and object template. These rules determine which objects are exported.

Rule	Purpose	Rule format
Export static structure	Exports selected objects. Does not process any secondary objects that are linked to the selected objects.	{%Rule:Selection.Export.Static}
Export dynamic structure using transfer mode	Processes the specified transfer mode to obtain a list of secondary linked objects when a selected object is processed for export. Secondary objects are exported as children of the selected object.	{%Rule:Selection.Export.TM.My-Transfer-Mode}

Property keyword syntax in specification templates and object templates

The following rules apply when property keywords are used in export templates:

- The property being specified must be encapsulated by braces ({ }).
- The property name must be preceded by a percent sign (%).
- The property name must be the exact name in the database. It cannot be a localized name.

Example

To include the object name property in a template, the keyword format is:

{%object_name}

Warning

The **{%object_name}** property keyword is required in object templates that are intended for export to the **Live integration with Word** output mode. Without this keyword in the template, such a structure cannot be edited in live Word.

Macro capability in Microsoft Office Word templates

When working with Microsoft Office Word templates in Teamcenter, keep in mind the following:

- Teamcenter supports user-defined macros only for **SpecTemplates**.
- You can add macros in **SpecTemplate** and save the template.
- Teamcenter preserves the macro only for *static* export to Word. If you export the macro using the Word live option, Teamcenter does not preserve the macro.
- Object templates do not support the macro capability. If you add any user-defined macro in the object template, it is not preserved in the output.
- Teamcenter does not support ActiveX controls for **SpecTemplates** and **ObjectTemplates**.

Associating export templates with business objects

Object export templates and specification export templates can be associated with an item revision using an item revision definition configuration (IRDC) object.

IRDCs standardize item revision behavior at specific times in the life cycle, such as at item creation, checkin, checkout, save as, and revise.

Create an IRDC for every item revision business object for which you want standardized behavior.

For more information about creating item revision definition configuration objects, see the [Business Modeler IDE Guide](#).

Microsoft Office Excel export templates

Excel templates define the property data and output format used when you export objects from Teamcenter to Excel. Property data can be exported to live Excel or to a static Excel spreadsheet.

Note Live Excel provides direct connectivity to the Teamcenter server, which allows you to modify the properties of Teamcenter objects without installing the rich client. At this time, direct connectivity is only supported for objects exported from My Teamcenter using an Excel export template. Direct connectivity for objects exported from structure editors is not yet supported.

Microsoft Excel 2007 or later is required to use live Excel. However, Excel export templates can be used if Microsoft Excel 2003 and the Microsoft Office Compatibility Pack for Excel 2007 are installed.

ExcelTemplate items and **ExcelTemplateRevision** item revisions are Teamcenter data objects that act as containers for the **MSExcels** dataset. The named reference of the **MSExcels** dataset is an Excel (.xslm or .xlsx) file.

Excel export templates:

- Contain a rule table that specify rules that control which objects are exported.
- Define the overall layout of the export spreadsheet, including header, footer, and style.
- Can be selected manually when you export data.
- Can be viewed and edited in Excel or in the Teamcenter embedded viewer.

For more information about item revision definition configuration (IRDC) objects, see the [Business Modeler IDE Guide](#).

Multiple sheets in Excel export templates

Excel export templates support multi-sheet workbooks. You can create multiple sheets in the template. Each sheet can specify different objects, rules, and formatting.

Cell formatting in Excel export templates

You can use standard Microsoft Excel tools to apply cell formatting and styles, such as color, font, and shading, to the export template. The format and styles are applied to the property data in the export spreadsheet. In addition to applying formatting and styles to the Excel template, you can also add static text, such as column headers and titles.

Siemens PLM Software does not recommend applying formatting, conditional or otherwise, to empty cells.

Excel export template rules

The Excel export template file is comprised of a rule table and corresponding columns that specify properties to which the rules are applied.

The screenshot shows an Excel spreadsheet titled 'REQ_default_excel_template.xlsx'. It contains a table with columns: Type, Object, Name, ID, Owner, Group, Data Modified, and CUI. Below this, there is a detailed explanation of the rule table structure and a sample rule table. The sample rule table has columns: Level, Relation, Type, and TransferMode. The first row is a header with values: Level, Relation, Type, TransferMode. The second row is a rule entry with values: <rule>, Level, Relation, Type, TransferMode, and the rule text: {%S-Requirement%}.

The rule table begins with the column containing the **<rule>** tag.

Note The rule table is not exported to the spreadsheet.

	Level	Relation	Type	TransferMode
<rule>			{%S-Requirement%}	
<rule>				

The **key fields**, **Level**, **Relation**, and **Type**, to the right of the **<rule>** column comprise the rule applied to the tag data in the property columns to the left of the **<rule>** column.

REQ_default_excel_template.xlsm							
	A	B	C	D	E	F	G
1	Siemens PLM Software						
2	Teamcenter Excel Export						
3							
4							
5	Type	Object	Name	ID	Owner	Group	Data Modified
6	Usage tips: 1.All the content above the <start> tag will be considered as header or constant information. But if the header cell has a format (%Real Property Name), then its value will be replaced by Display Name of that property during sheet generation. 2.The yellow column indicates the start of 'Rules'. 3.The 'Rules' should be written in the required order of precedence. 4.The Properties table on the left will specify the properties to be exported for each matching rule. 5.The rule table in the template is not exported to the spreadsheet. 6.Apply Style information to cells as desired. This style information will be taken and applied to the generated excel sheet.						
7	{%object_type}	{%object_string}	{%object_name}	{%item_id}	{%owning_user}	{%owning_group}	{%last_mod_date} {%checked_out}
8	{%object_type}		{%object_name}		{%owning_user}	{%owning_group}	{%last_mod_date}
9	All the content below the <end> tag will be considered as footer or constant information. You can insert images or any other content you would like to be exported.						
10							

General guidelines for Microsoft Office Excel export templates

Caution

Users must have at least **Read** access to all Excel export templates for their projects. A user without such access to one template cannot select any template in the **Export to Excel** dialog box.

- User-defined macros are preserved both in static and live Microsoft Office Excel sheets, but not in workbooks.
- Excel templates do not support ActiveX components, OLE embeddings, charts, formulas, and so on. They do, however, support free text and images in the header and footer.
- Out-of-the-box templates have Teamcenter macros. Be sure that when you edit the macro to add custom code, you do not remove the Teamcenter macros. Removing the macros can cause issues.
- The template must contain only one **<start>** tag and only one **<end>** tag. The **<start>** tag must appear to the left of the **<rule>** column.
- All rows containing data tags must be located below the **<start>** tag and above the **<end>** tag. Cell content outside the area delimited by these tags is static text.
- All property columns must be located to the left of the rule table. Any cell content to the right of the rule table is not exported to the spreadsheet.
- A cell containing a **<rule>** tag must precede the headings (**Level**, **Relation**, **Type**, and **Transfer Mode**) in the rule table.
- The sequence of the **Level**, **Relation**, **Type**, and **Transfer Mode** columns in the rule table must not be changed.

- Additional columns cannot be inserted into the rule table.

Key fields in the rule table

The following key fields can be specified in the rule table and are applied to the properties being exported:

Key field	Description	Syntax	Example
Level	Specifies the level of the objects for which property data is exported. For example, when 2 is entered as the key argument in the Level field, objects that appear at the second level in the view are exported to the Excel spreadsheet.	{%L-Level}	{%L-1}
Relation	Filters property data for export based on the relationship between two objects.	{%R-Relation-Name}	{%R-IMAN_specification}
Type	Filters property data for export based on workspace object type.	{%S-Type-Name}	{%S-Requirement}
Transfer Mode	Filters property data based on the transfer mode associated with an object. If a transfer mode is found that matches the one specified in the template, the secondary objects associated with the object are also exported.	{%T-Transfer-Mode-Name}	{%T-JTDataExportDefault}

Property keywords in Excel export templates

Properties are specified to the left of the rule table in the Excel export template. These properties are evaluated for export against the rules specified in the rule table.

The following syntax is used to specify properties:

{%Actual-Property-Name}

For example:

{%object_type}

Using transfer modes in Excel export templates

Transfer modes combine rules and property sets to define what additional data is exported when an object is selected for export. You can specify transfer modes in the Excel export rule table. If the object being exported has an associated transfer mode that matches the transfer mode specified in the template, the secondary objects indicated in the transfer mode are processed. The secondary objects are processed through the rule table and objects that match the rules are exported.

The following syntax is used to specify transfer modes in the template:

`{%T-Transfer-Mode-Name}`

Rule processing

Rules in the Excel export template rule table are processed as follows:

1. From top to bottom, beginning with the topmost rule in the table.
2. If an object is found that matches all criteria in the current rule, the data is extracted to the spreadsheet. Rule processing ends for that object and begins for another object, starting with the topmost rule in the table.
3. If any portion of a key field is blank, the criterion is matched by default.
4. If the current rule is completely blank, all criteria are matched by default. Therefore, Siemens PLM Software recommends that you place rules containing more key field criteria above rules with fewer key field criteria.
5. If an object does not match any rule in the template, no data is extracted for the object. When the spreadsheet is generated, a warning message states that objects that do not match any template criteria have been ignored.

Packing multiple objects

Grouping multiple identical objects in one level of a structure is known as *packing*. When the **Level** key field of the rule table is used in conjunction with the **Excel Template Rules** property of the template, data for two or more objects can be placed on the same row of an Excel export file.

Note Packing applies only if the user exports all objects in the view. If the user exports selected objects only, each object is placed on a separate row.

Object packing in Excel export files

When users export objects to Excel, objects in the database are examined to determine if their levels match the key fields in the rule table of the selected template. For the object that is currently being processed, data is placed in the export file as follows:

- If the level of the current object is lower than the level of the previous object, data for the current object is placed in a new row.
- If the level of the current object is greater than or equal to the level of the previous object:

- o If the cells following the previous object are empty, data for the current object is placed in the same row.
 - o If the cells following the previous object contain any character, data for the current object is placed in a new row.
- Note**
- o Objects are packed only if the user exports all objects in the view.
If the user exports selected objects only, each object's data is placed on a separate row.
 - o A cell that contains a space is considered to be empty.
To keep a cell from being overwritten, you must enter a visible character in the cell.
 - o Siemens PLM Software recommends that you do not apply any formatting to a cell where you intend to begin one object on the same row after another object.
Cell formatting may be processed as content, which in turn may prevent the desired object placement.

For example, assume the following:

- Objects **A**, **D**, and **E** are at level one.
- Objects **B**, **C**, **F**, and **G** are at level two.

Assume also that tags for properties named **1**, **2**, **3**, and **4** are entered in the property columns of the template, with key fields entered in the rule table as in the following example.

					Level	Relationship	Subtype
{%1}	{%2}	{%3}			{%L-1}		
			{%1}	{%4}	{%L-2}		

By default, the property tags and key fields in the preceding example produce the following output in the export file.

A1	A2	A3		
			B1	B4
			C1	C4
D1	D2	D3		
E1	E2	E3		
			F1	F4
			G1	G4

When the **Apply Packing** value is applied to the template's **Excel Template Rules** property:

- **B1**, **B4**, **F1**, and **F4** are placed in the row above with the parent objects.

- **C1, C4, G1, and G4** are placed in the row immediately below.

A1	A2	A3	B1	B4
			C1	C4
D1	D2	D3		
E1	E2	E3	F1	F4
			G1	G4

For more information, see [Apply packing to an Excel template](#).

Apply packing to an Excel template

1. Select the Excel export template.
2. In the **Properties** tab, double-click the value for the **Excel Template Rules** property to display the **Multi-Choice** dialog box.
3. Select the **Apply Packing** check box.
4. In the **Level** column of the rule table, enter key fields according to the way you want to fill the rows in the export file.

Create an Excel export template

1. In My Teamcenter, choose **File→New→Item**.
2. Select **ExcelTemplate** from the list of item types.
3. Click **Next**.
4. Type an item ID and revision ID in the **Item ID** and **Revision ID** boxes or click **Assign** to automatically assign the IDs.
5. Type a name for the Excel export template in the **Name** box.
6. (Optional) Type a description in the **Description** box.
7. Click **Finish**.

The **ExcelTemplate** item, **ExcelTemplateRevision** item revision, and associated **MSEExcelX** dataset are created in the **ExcelTemplates** folder, which is located in the **RequirementsManagement Templates** folder in your **Home** folder.

Note The **REQ_default_excel_template.xlsm** file is associated with the **MSEExcelX** dataset by default.

8. (Optional) To update the contents of the specification export template document, double-click the **MSEExcelX** dataset.
9. (Optional) Modify the contents of the template document, save the Excel file, and close Excel.

Modify an Excel export template

1. In My Teamcenter, check out the **ExcelTemplateRevision** item revision from the database.
2. Click the **MSExcelsX** dataset.
3. Click **Open** in the **File Download** dialog box.
The template file is opened in Excel.
4. Modify the data in the template spreadsheet.
5. Save and close the spreadsheet.

Copy an Excel export template

1. In My Teamcenter, click the **ExcelTemplate** item containing the template that you want to duplicate, and choose **File→Save As**.
2. Click **Assign** in the **Item Details** pane of the **Save Item As** dialog box.
The system assigns an item ID and revision ID for the new item.
3. Type a name for the new item in the **Name** box.
4. (Optional) Type a description of the new template item in the **Description** box.
5. (Optional) Select the **Open on Create** check box to open the item after it is created.
6. Click **OK**.
The new **ExcelTemplate** item and **ExcelTemplateRevision** are created. The **MSExcelsX** dataset is copied as a new object with the same name as the original dataset.

Import an Excel export template spreadsheet

1. In My Teamcenter, choose **Tools→Import→Templates→Excel Template**.
2. In the **Import Excel Template** dialog box, click **Browse** and locate the Excel file (**.xlsm** or **.xlsx**) containing the template you want to import.
3. Select the file and click **Save**.
4. (Optional) Type a description in the **Template Description** box.
5. Click **OK**.
The template is imported in to the Teamcenter database.
6. Click **OK** on the **Import Template** dialog box.
The **ExcelTemplate** item, **ExcelTemplateRevision** item revision, and associated **MSExcelsX** dataset are created in the **ExcelTemplates** folder, which is located in the **RequirementsManagement Templates** folder in your **Home** folder.

Delete an Excel export template

1. Ensure that the **ExcelTemplate** item is not referenced by other objects in the Teamcenter database.

For more information about performing where-referenced searches, see the [Rich Client Interface Guide](#).

2. In My Teamcenter, select the **ExcelTemplate** item.
3. Choose **Edit→Delete**.
4. Click **Yes** in the **Delete** dialog box.

The **ExcelTemplate** item, **ExcelTemplateRevision** item revision, and **MSEExcelX** dataset are deleted from the database.

Insert macros in custom Microsoft Office Excel templates

1. Verify that the custom template has the Teamcenter macro by using the ALT and F11 keys in Microsoft Office Excel.

If the template does not have the Teamcenter macro, copy the Teamcenter macro from any out-of-the-box Excel template.

2. Add your custom macro code such that the Teamcenter macro is not affected or the methods are not overridden to cause different behavior.
3. Save the macro and the template.

Users can use the Excel template for both static and live sheets.

Teamcenter preserves the macro in the generated sheet for static and live export to Excel.

Further configure Microsoft Excel Template exports

The Microsoft Excel template business object contains a list of values (LOV) property called **excel_template_rules**, which you can use to further configure Excel template exports. This LOV property controls several additional behaviors that you can apply to the Excel template being used during the export to Excel by modifying the property in the **RAC**. The following are the possible LOV values on the **excel_template_rules** property.

LOV value	Description
disable_outline	Disables the outlining feature in Excel. If you set this LOV value on the Excel template and export a structure using this template, outlining is disabled and the exported structure is a flat structure.
apply_packing	Packs multiple rows into a single row.

LOV value	Description
absolute_level	Level of the object is considered as the absolute level within the structure. The level of the object is calculated as the actual level within the structure irrespective of the starting level of the top line object. This means that if any intermediate structure is exported, the starting level of the object is not zero, but it is considered as the absolute level of the object within the structure.
disable_export_child_lines	Disables automatic traversal of the child lines. By default, when a structure is exported, you traverse deep into the structure. But if you set this LOV value on the template and export a structure using the template, automatic deep traversal of child lines is disabled. In this case, you can use a transfer mode to traverse deep into the structure.
audit_templates	Used for templates used for audit purposes (Audit Manager).
budget_templates	Used for templates in budget use cases (Budget scenarios).

Note You can set multiple LOV values for the same property.

Chapter

*15 Configuring Web Browser
for your site*

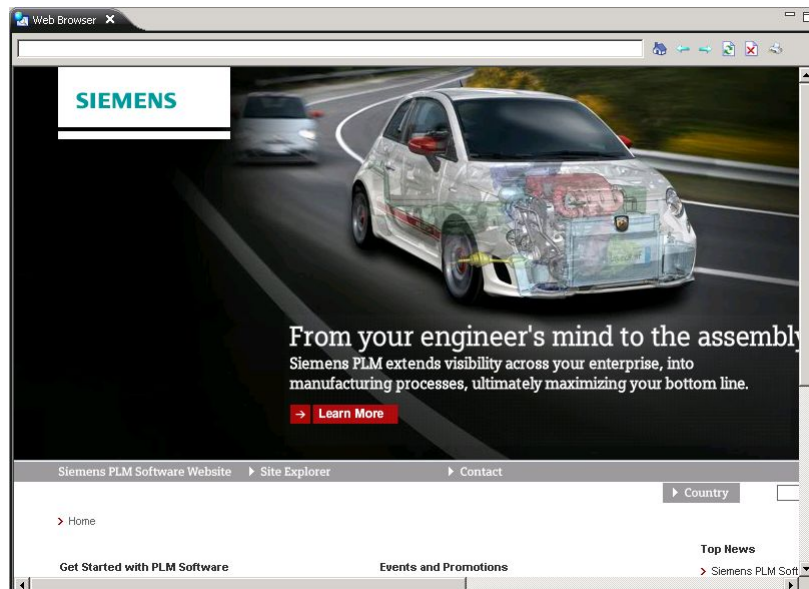
Configuring Web Browser for your site	15-1
---	------

Chapter

15 *Configuring Web Browser for your site*

Configuring Web Browser for your site

Users can access the Internet using the **Web Browser** view. Web Browser lets users navigate and view Web pages within the rich client rather than switching to a separate Web browser. The Web Browser also allows users to access MIME (Multipurpose Internet Mail Extension) file types and view files created in other applications, such as Microsoft Word and Excel.



Users access the view by choosing the **Window→Show View→Web Browser** menu command. Web Browser opens using the browser defined by the client's operating system.

Users can change the Web Browser home page using the [TC_Web_Browser_Home_URL](#) preference. By default, the URL is specified as <http://www.plm.automation.siemens.com>.

Chapter

16 *Configuring Lifecycle Visualization*

Configuring Lifecycle Visualization	16-1
Modifying Lifecycle Visualization configuration files	16-1
Specify visualization licensing options	16-1
Rich client embedded viewer compatibility	16-2
Viewing data from multiple Teamcenter servers	16-3
Multiple File Management System (FMS) client caches (FCCs)	16-4
How group and role changes affect viewer behavior	16-5
Visualization preferences	16-6
Search schemas for the Lifecycle Viewer	16-7
Creating text and watermark stamps using an MDS file	16-7
Watermark page example	16-11
MDS Watermark examples	16-11
An example using the \Halftone MDS command	16-13
An example without using the \Halftone MDS command	16-14
Add uniform watermark stamps when exporting files	16-15
Set the beginning cursor position for stamps with MDS files	16-18
Specify the orientation of text stamps with MDS files	16-19
Specify new types of visualization datasets	16-20
Specify dataset types with multiple named references	16-20
Configure automatic markups	16-21
Configure the Image Preview view	16-22
Configure the JT Preview	16-23
Configure image preview in Relation Browser	16-24
Configure image preview in the Summary view	16-24

Chapter

16 *Configuring Lifecycle Visualization*

Configuring Lifecycle Visualization

Teamcenter offers multiple software options to support your product visualization requirements, including the stand-alone viewer, the Lifecycle Viewer, and the embedded viewers located within Teamcenter applications such as My Teamcenter and the Structure Manager. Use the configuration options described in the following topics to customize Lifecycle Visualization for your organization.

Modifying Lifecycle Visualization configuration files

Because the files used to configure Teamcenter are cached, if you update **client_specific.properties** or **tcviewer.properties**, your changes may not take effect in the viewers because the rich client is still referencing the cached files. If your changes do not appear, do the following:

- To clear the cache, add the **-clean** argument to the command line in the shortcut you use to start the rich client. Once your changes appear, you can remove the argument from the command line.
- If you update a JAR file in the **../rac/plugins** directory, make sure to replace the original JAR file with the updated version. Do not leave the original version of the JAR file in the directory.

Specify visualization licensing options

Licensing for the Lifecycle Viewer and the Teamcenter embedded viewers must be specified in the **client_specific.properties** file, which is located in the **../rac/plugins/configuration_8000.0.0** directory on client machines.

Caution

These settings must match the licensing configuration available from your Teamcenter license server. They are set automatically during the client installation and in most cases should not be modified.

1. Open the **client_specific.properties** file in a text editor.
2. If necessary, add the following lines to the file:

This line	Requires this information
<code>PortalViewer_License_Level</code>	The name of your product configuration. Type the value listed below that matches your licensing level: • Base • Standard • Professional • Mockup
<code>PortalViewer_Optional_Licenses</code>	The names of functionality options not included in the default product configurations. Type the values, separated by commas, listed below that match any additional visualization licenses you have purchased: • ECAD • Analysis • Path_Planning • Concept_Desktop • Collaboration • Simplified_Rendering

Example

This example shows the visualization licensing parameters in the **client_specific.properties** file set to run the Mockup product configuration, with all available options:

```
PortalViewer_License_Level=Mockup
PortalViewer_Optional_Licenses=ECAD,Analysis,Path_Planning,Concept_Desktop,
Collaboration,Simplified_Rendering
```

Rich client embedded viewer compatibility

For Teamcenter 2007, the embedded viewer must be exactly the same version as the rich client.

Starting with Teamcenter 8, the embedded viewer can be either the same version as the rich client, or a newer version, provided both components share the same major and minor release.

Example

- The 8.3.3 version of the viewer is supported with the 8.3.2.1 version of the rich client. In this scenario, both components have the same major and minor release numbers, and the viewer is newer than the rich client.
- The 8.3.1 version of the viewer is not supported with the 8.3.2.1 version of the rich client. In this scenario, even though both components have the same major and minor release numbers, the viewer is older than the rich client.
- The 8.3 version of the viewer is not supported with the 8.2 version of the rich client. In this scenario, the viewer and the rich client do not have the same minor release number.

When using a version of the viewer that is newer than your rich client, new viewer features may not be available if they require rich client or server support. In addition, if a newer version of the viewer is installed, you must copy the **SingleEmbeddedViewer.jar** file from the `<viewer_install_location>\Program` folder to the `<rac_install_location>\rac\plugins` folder, overwriting the older version of the file. Note that upgrading an embedded viewer installation automatically upgrades any stand-alone viewer that is already installed.

Viewing data from multiple Teamcenter servers

If you plan to view data from multiple Teamcenter servers such as Teamcenter, Teamcenter engineering process management, and Teamcenter Enterprise, the various databases may not be coordinated with the viewer. you can only connect to one server at a time with only one FMS.

You must install either the rich clients for these servers or the FCC. Contact your Teamcenter administrator to obtain these client products. You must also modify the **fcc.xml** file by doing the following:

1. Contact your Teamcenter administrator for the following information about each server for each Teamcenter product you want to view visualization data from:
 - The site id number
 - The parentfsc address
2. Open a command window on your machine and navigate to `teamcenter-product-installation-directory\tccs\bin`.

Example

d:\tc\10\tccs\bin

3. At the command prompt, type: **fccstat -stop -kill**.
4. Navigate to `teamcenter-product-installation-directory\tccs`.
5. Open the **fcc.xml** file in a text editor.
6. Search in the file for the following string: **external site access definition**.
7. After the line containing this string, add the following lines:

```
<site id="site number" overridable="true">
<parentfsc address="URL" priority="1"/>
</site>
```

Example

```
<site id="101010101" overridable="true">
<parentfsc address="http://tcentserver:4444/" priority="1"/>
</site>
```

8. (If you have multiple Teamcenter product servers and you want to view visualization data from all of them) Repeat the previous step for each server.

- Note**
- For each additional **<site>** entry you add to the **fcc.xml** file, in the **<parentfsc>** tag, increment the value of the priority attribute, so that each entry has a different priority.
 - You can connect to only one server at a time.

9. Save the **fcc.xml** file.
10. At the command prompt, type: **fccstat -start**

Multiple File Management System (FMS) client caches (FCCs)

If you plan to view data from multiple Teamcenter products, such as Teamcenter, Teamcenter engineering process management, and Teamcenter Enterprise, you may have installed multiple File Management System (FMS) client caches (FCCs). If your machine has multiple FCCs and you want to view visualization data from a particular product, your machine's **FMS_HOME** environment variable must point to the location of the product's FMS.

To check or change the value of the **FMS_HOME** environment variable, do the following:

- Note** A TCCS/FCC stop is required before changing **FMS_HOME**.

Windows

1. Right-click **My Computer** and select **Properties**.
2. In the **System Properties** dialog box, click the **Advanced** tab.
3. On the **Advanced** page, click **Environment Variables**.
4. On the **Environment Variables** page, in the **User variables** section, find **FMS_HOME** and verify that its path refers to the installation path of the product from which you want to load visualization data.
5. If necessary, change the path as needed.

Example **d:\tc\10\tccs**

Linux

1. In the shell window from which the software product was launched, type **echo \$FMS_HOME**.
A path to the installation of that product is displayed.
2. Check the path to verify that the **FMS_HOME** environment variable is pointing to the right product.
3. If necessary, change the path as needed.

Example `../tc/10/tccs`

How group and role changes affect viewer behavior

If you change your Teamcenter group or role while visualizing data, the behavior of the viewer depends on your configuration.

Stand-alone viewer in four-tier mode

If you change your group or role while visualizing data in the stand-alone viewer in four-tier mode, you may need to close the viewer and reopen your data. Consider the following scenario:

1. Log on to a Teamcenter session in the rich client using the default group and role.
2. Launch data into the stand-alone viewer in four-tier mode.
3. In the rich client, switch to a different group and role.
4. To send additional data into the stand-alone viewer, you must do the following:
 - a. In the stand-alone viewer, save all unsaved data.
 - b. Close the viewer.
 - c. Launch your data into a new instance of the viewer.

Embedded Lifecycle Viewer and the stand-alone viewer in two-tier mode

If you change your group or role while visualizing data in the Lifecycle Viewer or the stand-alone viewer in two-tier mode, the data is displayed correctly without any additional actions. Consider the following scenario:

1. Log on to a Teamcenter session in the rich client using the default group and role.
2. Launch data into the Lifecycle Viewer or the stand-alone viewer in two-tier mode.
3. In the rich client, switch to a different group and role.
4. Send the same data again to the viewer.

The data is displayed as if the group and role switch did not happen. This is because the previously loaded data is cached.

5. Send different data to the viewer.

The data is displayed correctly for the current group and role. This is because the new data was not cached.

Visualization preferences

The Teamcenter visualization preferences control how the Lifecycle Viewer and the stand-alone viewers interact with Teamcenter clients. In the rich client, you can adjust the settings for these preferences by choosing **Edit**→**Options**.

Note For more information about modifying options, see [Getting Started with Teamcenter](#).

You can modify the following visualization preferences:

Use this preference	To specify this
3DMarkup_relation_primary	The dataset types that you can mark up with the 3D markup tools.
DMI_markup_relation_primary	The dataset types that you can mark up with the 2D markup tools.
INTEROP_ExtraPLMXMLInstanceAttributes	The mapping of Structure Manager BOM line attributes to Lifecycle Visualization attributes. This preference is applicable only for PLM XML generated by the BOMWriter.
Interop_Vis_Attributes	The default list of Teamcenter attributes to display in the viewer.
Interop_Vis_Attributes_Admin_Filter	The list of allowable Teamcenter attributes that the user can choose to have displayed in the viewer.
ItemRevision_DirectModel3DMarkup_default_relation	Specifies the relation that has to be created when a DirectModel3DMarkup dataset is pasted under an ItemRevision.
ItemRevision_Vis_Session_default_relation	Specifies the relation that has to be created when a Vis_Session dataset is pasted under an ItemRevision.
MetaDataStamp_template	Specifies the Item ID of the Item Revision that provides the MetaDataStamp Template dataset used to create text and watermark stamps.
PLM_End_Item_Key	The Teamcenter Occurrence Note (BOMLine attribute key/name) used to identify an Occurrence as an End Item for clearance analysis.
SecondaryVMUDatasets	The list of non-VMU dataset types that can contain VMU datasets. VMU datasets are datasets that can be directly launched from the rich client or the thin client into the Lifecycle Viewer or a stand-alone viewer
TC_DIS_2D_named_ref	Reference names of 2D file types that are capable of 2D markups.

Use this preference	To specify this
TC_DIS_3D_named_ref	Reference names of 3D file types capable of 3D markups.
TC_DIS_Search	Search schemas to be used when performing a search from within the viewer
TC_Schematic_BOMView_Types	Whether Lifecycle Visualization is used in schematic mode or 3D mode.
TC_show_open_in_vmu_button	Whether to display the Start/Open in TcVis button in the rich client and the thin client.
VIS_multi_file_datasettypes	Visualization datasets that can contain multiple named references. Users can select the reference to view in the embedded viewer.
VisMarkup_relation_primary	Indicates the valid primary object type for the relation Visualization 3DMarkup.
VisSession_relation_primary	Indicates the valid primary object type for the relation Visualization Session.
VMU_Datasets	Datasets that can be directly launched from the rich client or the thin client into the Lifecycle Viewer or a stand-alone viewer.
VMU_FileSearchOrder	The order that reference names are used to search for files in ambiguous conditions.
VMU_RelationSearchOrder	Relations searched between visualization datasets and items or item revisions.

Search schemas for the Lifecycle Viewer

By default, the following system-defined search schemas are used when searches are performed from the file dialogs within the stand-alone viewers and the Lifecycle Viewer:

- Dataset
- Item
- Item revision
- Part

To add additional system-defined search schemas, modify the **TC_DIS_Search** preference.

Note For more information about performing searches in Teamcenter, see *Rich Client Interface Guide*.

Creating text and watermark stamps using an MDS file

As an administrator you can create an MDS stamp file to add a watermark and stamp text to all printed images. You can specify a background color to the

watermark, add headers and footers, and modify the stamp further by using additional MDS commands.

Note You use the `MetaDataStamp` Template to control the formatting styles used in text and watermark stamps. The default **MDS_default_styles_template.mds** template file is located in the `TC_DATA` directory in the RAC; it is imported to the database as a dataset called **MDS_default_styles_template**. For information on importing a custom **MDS_default_styles_template.mds** template file, see the [MetaDataStamp_template](#) preference.

Caution When you enable the [MetaDataStamp_template](#) preference, end-user watermark preferences become unavailable.

The watermark MDS command operates differently from other MDS stamp commands.

- Only one **\Watermark** command is supported in an MDS file. However, you can enter several other basic MDS commands that create useful watermark stamps.
- The beginning position of the watermark text always defaults to the center of the page. However, you can modify the beginning position of the watermark by using the **\Pos** option.

- Note**
- Type each MDS command on a separate line.
 - Basic text and watermarks stamps support the following replacement strings:
 - o **%Date** – date only
 - o **%Time** – time only
 - o **%User** – the user name
 - o **%File** – a file name; stamp.mds
 - o **%FilePath** – a path and file name; c:\etc\stamp.mds
 - o **%Page** – page number; 1 of 2
 - Invalid or non-supported MDS commands are ignored.

To create an MDS stamp file, do the following:

1. Open a text editor and type in options from the following list:

Type this option	To do this
\BackgroundColor (Color or RGB value)	Set the background color for text lines, including Watermarks. Example \BackgroundColor (Red) \BackgroundColor (255,0,0) Note - Control the background color value in one of these ways: - Enter a standard RGB value. - Enter the exact name of one of the following supported colors: - white - yellow - cornsilk - green - cyan - sky blue - blue - midnight blue - purple - magenta - red - orange - salmon - gray80 - slate - gray - black - For text without a background color use \BackgroundColor (None).

Type this option	To do this
\Char(size)	Set the font size for text. Example \Char (12)
\Color(color)	Set the color for text. Example \Color (Red)
\Date	Display the system date and time.
\File	Display the document path and file name.
\Font(name)	Set the font. Example \Font (Courier)
\FooterLeft \FooterCenter \FooterRight	Enter text that is displayed at the bottom of the page. Example \FooterCenter Confidential
\Halftone(1.0 or 0.5)	Set the halftone to opaque (a value of 1.0) or to transparent (a value of 0.5). Example \Halftone (0.5)
\HeaderLeft \HeaderCenter \HeaderRight	Enter text that is displayed at the top of the page. Example \HeaderCenter Confidential
\Pos(x,y)	Set the beginning cursor position. The default cursor position is (36,36).
\Rotate(angle)	Set the rotation angle of your text. Example \Rotate(45) \Rotate(-45)
\User	Display the current user ID.
\Watermark	Specify the watermark command on one line in the MDS file. The next line of text includes your watermark message. Note - The Orientation parameter supports the following values (LL2UR) – lower left to upper right (UL2LR) – upper left to lower right (HORIZ) – horizontal If you do not specify a value, the default, HORIZ, is used. - You can use the value AUTO to set watermark text to be as large as possible. If AUTO is not specified, text size is determined by the

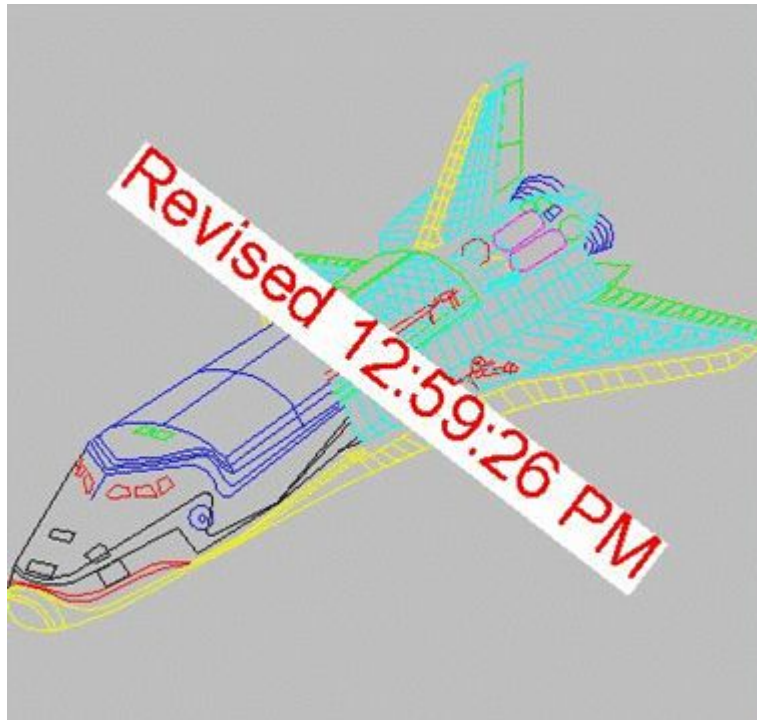
Type this option	To do this
	last value in \Char, or to the default value, \Char(12).
Example	\Watermark(HORIZ,AUTO) Sample Text

All printed images will now contain the watermark defined by the MDS file.

[MDS Watermark examples](#)

Watermark page example

The following graphic illustrates the result of using various options from the Watermark page.

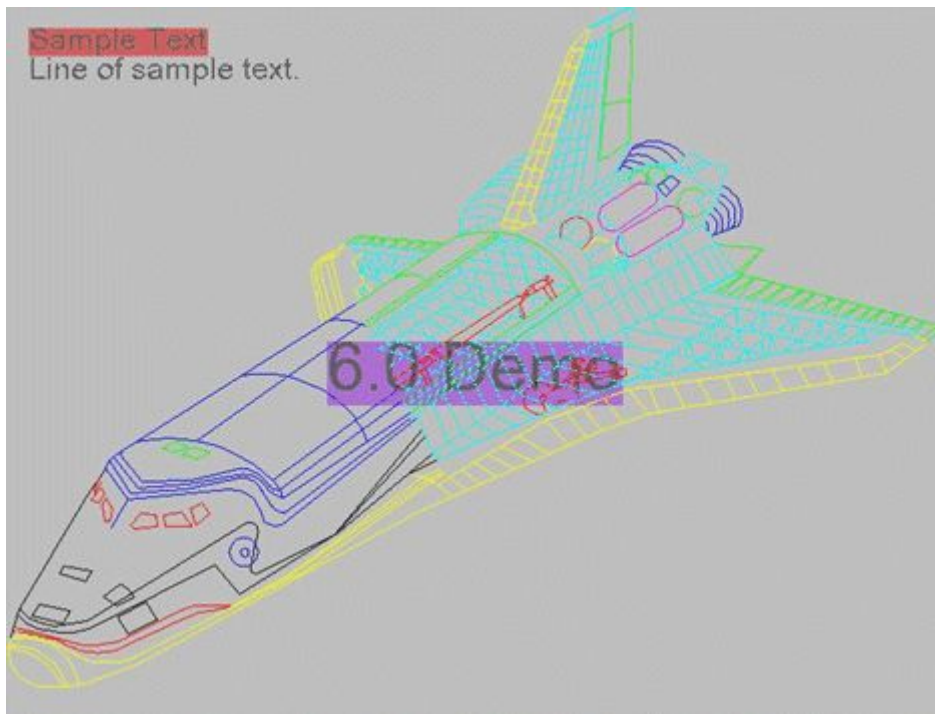


MDS Watermark examples

An example using the \Halftone MDS command

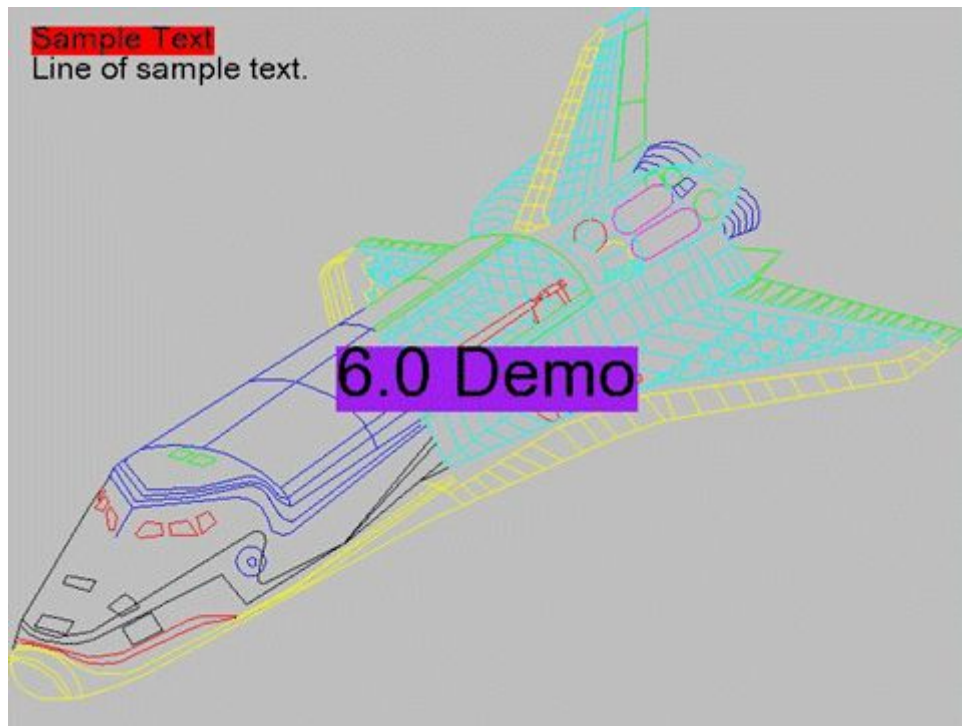
```
\Color(Black)
\Font(Arial)
\Char(16)
\Pos(20,20)
\Halftone(0.5)
\BackgroundColor(Red)
Sample Text
\BackgroundColor(None)
Line of sample text.
\BackgroundColor(Purple)
\Char(36)
\Watermark(HORIZ)
```

6.0 Demo



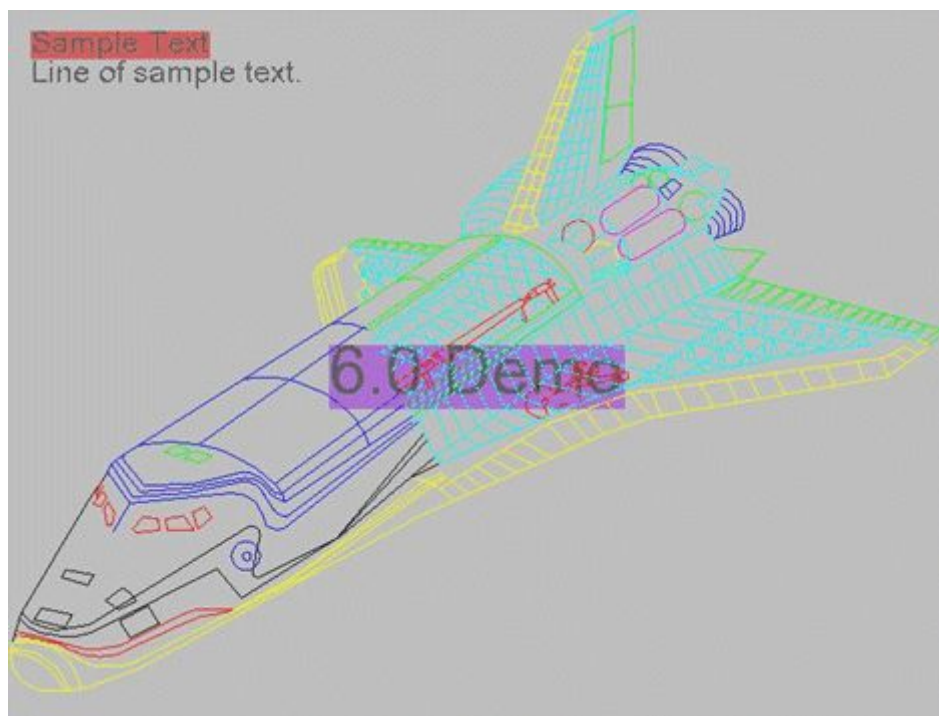
An example without using the \Halftone MDS command

```
\Color (Black)
\Font (Arial)
\Char (16)
\Pos (-20,20)
\BackgroundColor (Red)
Sample Text
\BackgroundColor (None)
Line of sample text.
\BackgroundColor (Purple)
\Char (36)
\Watermark (HORIZ)
6.0 Demo
```



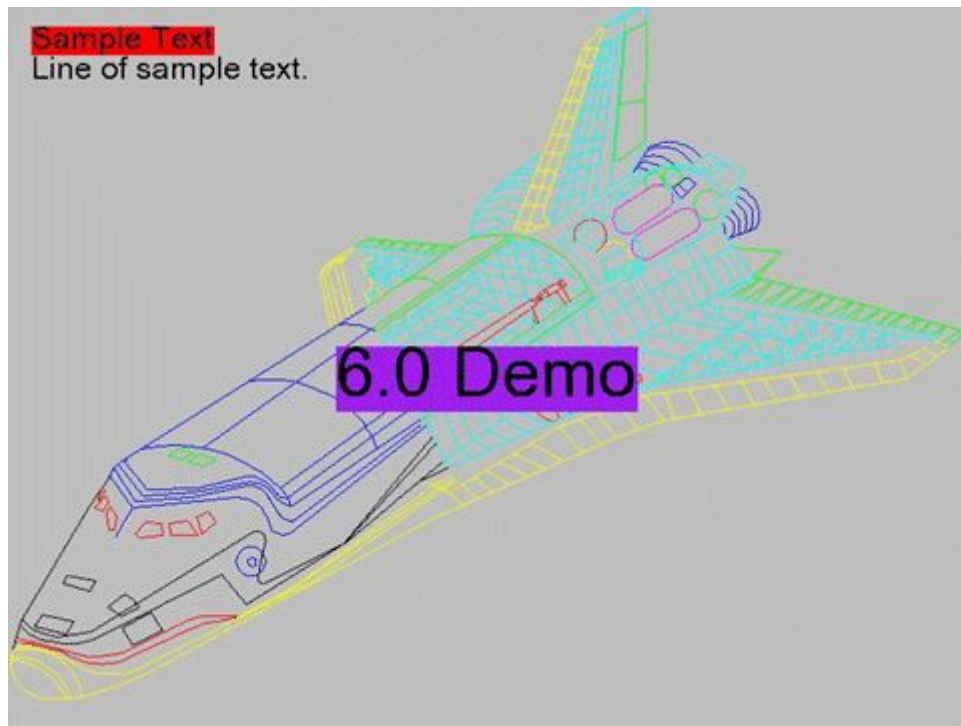
An example using the \Halftone MDS command

```
\Color(Black)
\Font(Arial)
\Char(16)
\Pos(20,20)
\Halftone(0.5)
\BackgroundColor(Red)
Sample Text
\BackgroundColor(None)
Line of sample text.
\BackgroundColor(Purple)
\Char(36)
\Watermark(HORIZ)
6.0 Demo
```



An example without using the \Halftone MDS command

```
\Color(Black)
\Font(Arial)
\Char(16)
\Pos(-20,20)
\BackgroundColor(Red)
Sample Text
\BackgroundColor(None)
Line of sample text.
\BackgroundColor(Purple)
\Char(36)
\Watermark(HORIZ)
6.0 Demo
```



Add uniform watermark stamps when exporting files

As an administrator you can create an MDS stamp file to add a watermark and stamp text to all 2D images created using **Export Image**. You write a text file that includes your MDS stamp information and name it **export.mds**. Save this file in the **\etc** folder within the installation directory. You can also set a background color to the watermark and modify the stamp further by using additional MDS commands.

Caution When you create and save the **export.mds** file, end-user watermark preferences become unavailable.

- Note**
- Type each MDS command on a separate line.
 - Invalid or non-supported MDS commands are ignored.
 - Only one **\watermark** command is supported in an MDX file.
 - The beginning position of the watermark text always defaults to the center of the page. However, you can modify the beginning position of the watermark by using the **\Pos** option.
 - Watermark stamps support the following replacement strings:
 - o *%Date* – date only
 - o *%Time* – time only
 - o *%User* – the user name

- o *%File* – a file name; stamp.mds
- o *%FilePath* – a path and file name; c:\etc\stamp.mds
- o *%Page* – page number; 1 of 2

1. Open a new file in a text editor.
2. Select from the following:

Type this option	To do this
\BackgroundColor (Color or RGB value)	Set the background color for text lines, including Watermarks.
	Example <pre>\BackgroundColor (Red) \BackgroundColor (255,0,0)</pre>
	Note • Control the background color value in one of these ways: - o Enter a standard RGB value. - o Enter the exact name of one of the following supported colors: ■ white ■ yellow ■ cornsilk ■ green ■ cyan ■ sky blue ■ blue ■ midnight blue ■ purple ■ magenta ■ red ■ orange ■ salmon

Type this option	To do this
	■ gray80 ■ slate ■ gray ■ black - For text without a background color use <code>\BackgroundColor (None)</code>.
<code>\Char(size)</code>	Set the font size for text. Example <code>\Char (12)</code>
<code>\Color(color)</code>	Set the color for text. Example <code>\Color (Red)</code>
<code>\Date</code>	Display the system date and time.
<code>\File</code>	Display the document path and file name.
<code>\Font(name)</code>	Set the font. Example <code>\Font (Courier)</code>
<code>\Halftone(1.0 or 0.5)</code>	Set the halftone to opaque (a value of 1.0) or to transparent (a value of 0.5). Example <code>\Halftone (0.5)</code>
<code>\Pos(x,y)</code>	Set the beginning cursor position. The default cursor position is (36,36).
<code>\Rotate(angle)</code>	Set the rotation angle of your text. Example <code>\Rotate(45)</code> <code>\Rotate(-45)</code>
<code>\User</code>	Display the current user ID.
<code>\Watermark</code>	Specify the watermark command on one line in the MDS file. The next line of text includes your watermark message. Note - The Orientation parameter supports the following values (LL2UR) – lower left to upper right (UL2LR) – upper left to lower right (HORIZ) – horizontal

Type this option	To do this
	If you do not specify a value, the default, HORIZ, is used. You can use the value AUTO to set watermark text to be as large as possible. If AUTO is not specified, text size is determined by the last value in <code>\Char</code>, or to the default value, <code>\Char(12)</code>. Example <code>\Watermark(HORIZ,AUTO)</code> Sample Text

3. Name the file, **export.mds**, and save it to the **etc** folder within your installation directory.

All images saved using **Export Image** are saved with the uniform watermark stamp.

Example This example shows the text of a watermark stamp that is embedded on all images saved by **Export Image**.

```

\Color(0,255,0)
\Font(Arial)
\Char(18)
\BackgroundColor(white)
\Watermark(UL2LR)
%File: Restricted Document %Time

```

Set the beginning cursor position for stamps with MDS files

If you create additional detailed stamps for your files by using MDS stamps, use the MDS command **/Pos(x,y)** to specify the beginning cursor position of the stamp. The numeric values, *x* and *y* are measured in point-size, where one point is equal to 1/72 of an inch.

The default starting position for all MDS text entries is the upper-left corner of the page, or a position equal to `/Pos(36,36)`. This means the cursor begins at a point that is 1/2 inch from the top of the page and 1/2 inch from the left edge of the page.

You can use these additional cursor position options, as needed:

Use this option	To do this
=	Position the stamp outside the stamped image and into the margin. This position indicator is only available for HPGL-based stamping and cannot be used with the ! position indicator. This may only be used for the vertical position.
!	Measure the position in point sizes (1/72 of an inch or approximately 0.35 millimeters). Without the symbol (!), the current operating coordinate system determines the units of measure.
—	Begin the cursor at the lower right-hand corner of the image. Without the symbol (—), the position is measured from the upper left-hand corner.

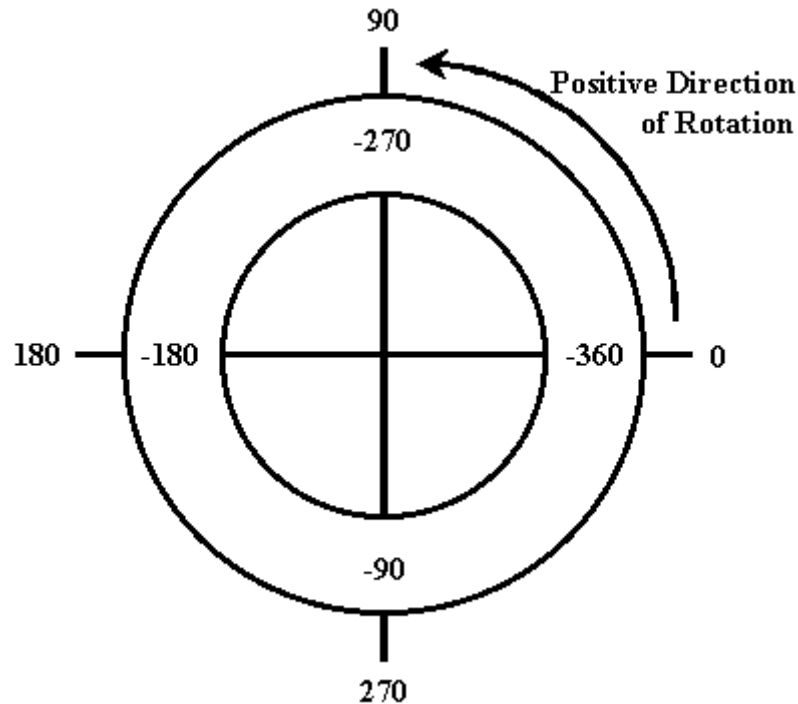
Note Allow enough space for the character size in the y direction to avoid truncating (cropping) the stamp.

Example This example shows how to position the cursor 3 inches up from the bottom and 2 inches from the right-hand edge of the page.

```
\pos(!-144,!-216)
```

Specify the orientation of text stamps with MDS files

1. Use the MDS **Rotate** command to specify how your stamp displayed on the page.
2. Use positive rotation values to rotate text counter-clockwise. The zero angle (or start) is the point of intersection on the right side of a horizontal line crossing through the center of rotation. The center of rotation is the midpoint of the height and the midpoint of the width before rotation.



Example

Type this	To do this
\Rotate(45)	Rotate the text 45 degrees counter-clockwise. The text flows from the starting position on a lower left corner to the upper right corner angle.
\Rotate(90)	Rotate the text 90 degrees counter-clockwise. The text flows from the starting position straight up the page on a perpendicular angle.
\Rotate(-90)	Rotate the text 90 degrees clockwise. The text flows from the starting position straight down the page on a perpendicular angle.

Tip

- The angle value must be an integer.
- To prevent truncated or cropped text, be mindful of your text starting point, font size, and rotation angle.

Specify new types of visualization datasets

For visualization file types defined as dataset business objects by the Business Modeler IDE, but not associated by default with the Lifecycle Viewer and the My Teamcenter viewer, you must configure each dataset type for display.

Note

New dataset types require unique names for each dataset type and named reference.

1. Add the names of the custom dataset types to the **VMU_Datasets** preference.
2. Add the names of the custom references to the **VMU_FileSearchOrder** preference.
3. If 2D markup layers are valid for these file types, add the names of the custom references **TC_DIS_2D_named_ref** preference.
4. If 3D markup layers are valid for these file types, add the names of the custom references **TC_DIS_3D_named_ref** preference.
5. (For the Lifecycle Viewer) Create entries for each dataset type in the **TcViewer_properties.txt** file.

Note

The **TcViewer_properties.txt** file is installed to the **[installation directory]\Visualization\etc\PLMIntegration** directory on client machines containing stand-alone viewer installations. This file contains examples of properly constructed dataset type definitions as well as comments that describe each of the necessary parameters.

6. (For My Teamcenter) Create entries for each dataset type using the **defaultViewerConfig.VIEWERCONFIG** preference.

Example

To enable the viewing of NX **.prt** files in the My Teamcenter viewer, add the following values to the **defaultViewerConfig.VIEWERCONFIG** preference:

```
UGMASTER.DirectModelViewer=UGPART
UGPART.DirectModelViewer=UGPART
UGALTREP.DirectModelViewer=UGPART
```

Specify dataset types with multiple named references

You must specify the visualization dataset types that can contain more than one named reference. Users can view the selected named reference in the My Teamcenter embedded viewer.

- Add the names of the dataset types to the **VIS_multi_file_datasettypes** preference.

Configure automatic markups

Automatic markup functionality allows users to markup 2D and 3D datasets directly. Nonstandard dataset types can also be marked up. Complete the following configuration steps to enable this functionality within the Lifecycle Viewer and the embedded viewers that are located within Teamcenter applications such as My Teamcenter and the Structure Manager:

1. In the Business Modeler IDE application, create a new dataset type (for example **DJ_Dwg**); define the dataset type as follows:
 - a. In the **Tools** section, add the **PV** tool to the **List of Selected Tools** field.
 - b. Click the **References** button.
 - c. In the **References** dialog box, enter the name of the dataset type in the **Reference** box, for example, **dj_drawing**.
 - d. Enter **File** in the **File/Object** box.
 - e. Enter **.jpg** in the **File** box.
 - f. Enter **BINARY** in the format box.
 - g. Click **Create** to create the new dataset type.

2. In the `\com\teamcenter\rac\common\tcviewer\tcviewer.properties` file, make the following modifications:

Note The `tcviewer.properties` is located in the `%IPR%\plugins\com.teamcenter.rac.common_teamcenter-version-number.jar` file.

- a. Add the name of the newly created dataset type to the **DatasetViewer.TYPES** property, for example, add **DJ_Dwg**.
 - b. Add the named reference to the **DatasetViewer.VIEWSEARCHORDER** property, for example, add **dj_drawing**.
 - c. Add the **VIEWPANEL** for the new dataset type, for example, add **Dj_Dwgcom.VIEWPANEL=com.teamcenter.rac.common.tcviewer.TCTwoDViewer**.
3. In the `\com\teamcenter\rac\explorer\explorer.properties` file, add the name of the new dataset type to the **explorer.CONTAINERDATASETYPES** list.

Note The `explorer.properties` is located in the `%IPR%\plugins\com.teamcenter.rac.tcapps_teamcenter-version-number.jar` file.

4. (For 2D markups) In the **Preferences** tab of the **Options** dialog box, accessed from the **Edit** menu, modify the following preferences:
 - a. Add the name of the new dataset type to the **TC_DIS_2D_named_ref** preference.

- b. Add the name of the new dataset type to the **DMI_markup_relation_primary** preference.
 - c. Add the name of the new dataset type to the **VMU_Datasets** preference.
 - d. Add the name of the new named reference (for example **dj_drawing**) to the **VMU_FileSearchOrder** preference.
5. (For 3D markups) In the **Preferences** tab of the **Options** dialog box, accessed from the **Edit** menu, modify the following preferences:
 - a. Add the name of the new dataset type to the **TC_DIS_3D_named_ref** preference.
 - b. Add the name of the new dataset type to the **3DMarkup_relation_primary** preference.
 - c. Add the name of the new dataset type to the **VMU_Datasets** preference.
 - d. Add the name of the new named reference (for example **dj_drawing**) to the **VMU_FileSearchOrder** preference.
6. In the **Preferences** tab of the **Options** dialog box, accessed from the **Edit** menu, create the new **dataset_type_DefaultChildProperties** preference, where the dataset type is the name of the new dataset type name, for example: **DJ_Dwg_DefaultChildProperties**.

Configure the Image Preview view

Use the **Image Preview** view to obtain quick previews of 2D raster images. Administrators can configure 2D dataset types, reference types, and the relation search order in the **tcgrb.properties** file.

Note For information about updating properties files in Eclipse plug-ins, see the *Rich Client Customization Programmer's Guide*.

1. Open the **tcgrb.properties** file, which is located in the **com.teamcenter.rac.tcgrb_8000.0.0** plug-in.
2. Do any of the following:

To	Do this
Specify the 2D datasets to display	Add 2D dataset types, separated by commas, to the Thumbnail2DViewer.TYPES property.
Specify the named reference types to be searched within a dataset for an image to display. Only named references listed here are valid candidates for thumbnail display.	Add named reference types, separated by commas, to the Thumbnail2DViewer.VIEWSEARCHORDER property.

Specify the order in which datasets are chosen when there are datasets attached in different relations to an item revision.	Add relation types, separated by commas, to the Thumbnail2DViewer.RELATIONSEARCHORDER property. Note Images are displayed according to the order specified here, even if there are valid 2D datasets attached in other relations to an item revision
---	---

Example

```
Thumbnail2DViewer.TYPES=Image
Thumbnail2DViewer.VIEWSEARCHORDER=Image
Thumbnail2DViewer.RELATIONSEARCHORDER=IMAN_reference,IMAN_manifestation,
IMAN_Rendering,IMAN_specification
```

3. Save the file, and then update the plug-in.

Configure the JT Preview

Use the **JT Preview** to obtain quick previews of **JT** parts. Administrators can configure 3D dataset types, reference types, and the relation search order in the **tctdv.properties** file.

Note For more information about updating properties files in Eclipse plug-ins, see the [Rich Client Customization Programmer's Guide](#).

1. Open the **tctdv.properties** file, which is located in the **com.teamcenter.rac.tctdv_8000.0.0** plug-in.
2. Do any of the following:

To	Do this
Specify the 3D datasets to display	Add 3D dataset types, separated by commas, to the Thumbnail3DViewer.TYPES property.
Specify the named reference types to be searched within a dataset for a part to display. Only named references listed here are valid candidates for thumbnail display.	Add named reference types, separated by commas, to the Thumbnail3DViewer.VIEWSEARCHORDER property.
Specify the order in which datasets are chosen when there are datasets attached in different relations to an item revision.	Add relation types, separated by commas, to the Thumbnail3DViewer.RELATIONSEARCHORDER property. Note Images are displayed according to the order specified here, even if there are valid 3D datasets attached in other relations to an item revision

Example

```
Thumbnail3DViewer.TYPES=DirectModel,DirectModelMotion
Thumbnail3DViewer.VIEWSEARCHORDER=JTPART
Thumbnail3DViewer.RELATIONSEARCHORDER=IMAN_manifestation,IMAN_Rendering,
IMAN_specification,IMAN_Motion,IMAN_3D_snap_shot,TC_Attaches
```

3. Save the file, and then update the plug-in.

Configure image preview in Relation Browser

Administrators can configure the image preview in Relation Browser by specifying the relation search order and the view search order in the **tcgrb.properties** file.

For information about updating properties files in Eclipse plug-ins, see the [Client Customization Programmer's Guide](#).

1. Open the **tcgrb.properties** file, which is located in the **com.teamcenter.rac.tcgrb_xxxx.x.x** plug-in.
2. Do any of the following.

To	Do this
Specify the order in which datasets are chosen when there are datasets attached in different relations to an item revision.	Add relation types, separated by commas, to the DatasetViewer.RELATIONSEARCHORDER property. Note Images are displayed according to the order specified here, even if there are valid datasets attached in other relations to an item revision
Specify the named reference types to be searched within a dataset for an image to display. Only named references listed here are valid candidates for image preview display.	Add named reference types, separated by commas, to the ImagePreview.VIEWSEARCHORDER property.

Example

```
DatasetViewer.RELATIONSEARCHORDER=IMAN_manifestation,IMAN_Rendering,  
IMAN_specification,IMAN_Motion,IMAN_3D_snap_shot  
ImagePreview.VIEWSEARCHORDER=UG-QuickAccess-Binary,Grahpics-Interface,  
Markup,Image,JPEG,BMP,Shaded-image
```

Note

If the first dataset found does not contain an image file, Relation Browser searches until an image file is found.

3. Save the file, and then update the plug-in.

Configure image preview in the Summary view

Administrators can configure the image preview in the **Summary** view by specifying the relation search order and the view search order in the **tcviewer.properties** file.

Note

For information about updating properties files in Eclipse plug-ins, see the [Rich Client Customization Programmer's Guide](#).

1. Open the **tcviewer.properties** file, which is located in the **com.teamcenter.rac.common.tcviewer** plug-in.

2. Do any of the following:

To	Do this
Specify the order in which datasets are chosen when there are datasets attached in different relations to an item revision.	Add relation types, separated by commas, to the DatasetViewer.RELATIONSEARCHORDER property. Note Images are displayed according to the order specified here, even if there are valid datasets attached in other relations to an item revision
Specify the named reference types to be searched within a dataset for an image to display. Only named references listed here are valid candidates for image preview display.	Add named reference types, separated by commas, to the ImagePreview.VIEWSEARCHORDER property.

Example

```
DatasetViewer.RELATIONSEARCHORDER=IMAN_manifestation,IMAN_Rendering,
IMAN_specification,IMAN_Motion,IMAN_3D_snap_shot
ImagePreview.VIEWSEARCHORDER=UG-QuickAccess-Binary,Graphics-Interface,
Markup,Image,JPEG,BMP,Shaded-image
```

Note

If the first dataset found does not contain an image file, the system does not continue searching the datasets until an image file is found.

3. Save the file, and then update the plug-in.

Chapter

17 *Configuring Teamcenter Integration for NX*

Configuring Teamcenter Integration for NX	17-1
Configuring attribute mapping	17-1
Mapping Teamcenter attributes to NX part attributes	17-1
Additional definition for attributes when mapping	17-2
Default mappings	17-3
Mapping Classification attributes to NX part attributes	17-3
Attribute synchronization	17-3
Monodirectional and bidirectional synchronization	17-4
NULL value synchronization	17-5
Extending attribute mappings	17-5
Teamcenter to Teamcenter Integration for NX attribute mapping syntax ..	17-6
Comments and contexts symbols	17-8
Mapping examples	17-9
Classification to Teamcenter Integration for NX attribute mapping syntax	17-11
Automatically classifying items and item revisions while creating parts	17-13
Troubleshooting	17-13

Chapter

17 *Configuring Teamcenter Integration for NX*

Configuring Teamcenter Integration for NX

NX is an interactive Computer-Aided Design, Computer-Aided Manufacturing, and Computer-Aided Engineering (CAD/CAM/CAE) system. The CAD functions automate the normal engineering, design, and drafting capabilities found in today's manufacturing companies. The CAM functions provide NC programming for modern machine tools using the NX design model to describe the finished part. The CAE functions provide a number of product, assembly, and part performance simulation abilities across a broad range of engineering disciplines.

Teamcenter Integration for NX is a data management tool used in conjunction with NX. When NX is used in Teamcenter Integration for NX mode, Teamcenter is running at the same time as a separate process. Because the two programs communicate, you can create, store, and access your data within a Teamcenter database. It combines the power of NX in modeling and generating data from geometric shapes, with the power of Teamcenter in storing and retrieving data in a controlled fashion.

Configuring attribute mapping

Using mapping definition files, NX part attributes can be stored in the Teamcenter database and displayed and modified both in NX and Teamcenter. These attributes can also be opened by NX GRIP and UG/Open, and they can be manipulated using ITK programs.

Mapping Teamcenter attributes to NX part attributes

To write a mapping file, you must know how to determine valid paths from one object to the next. This can include determining the following:

- The properties of the object
- Which generic relationship types exist for the object
- What named reference types exist for the object
- What roles exist for the object
- What release levels exist for the object and which roles are associated with each

This information can be determined for each object from the Business Modeler IDE.

For more information about using this application, see the [Business Modeler IDE Guide](#).

Database attributes do not have to be properties of the datasets corresponding to an NX part file. Database attributes can also be any of the following, as specified in the mapping definition file:

- Properties of other Teamcenter objects

The mapping definition specifies how to find the object beginning with the original dataset or related item or item revision and traversing a path from one Teamcenter object to another. The final step in the path produces a value, which can be an integer, a double, a string, a date, or an object. Each step may be a property, named reference, generic relationship, job, or signoff.

Note You cannot define the attribute to be a property of an *arbitrary* Teamcenter object. The object must be accessible from the original dataset, item, or item revision by a series of steps, as described above. Examples include a user name or release date.

- Teamcenter preference values

The mapping definition indicates the scope and name of the preference.

- Constants

The mapping definition indicates the constant value. An example of a constant is a copyright notice.

The set of mappings that apply to a particular dataset can be dependent on the dataset type and/or the related item type. (Teamcenter Integration for NX assumes that the dataset is attached to an item revision for which there is always a related item.)

The mappings are stored permanently in the Teamcenter database and are available when NX is running in Teamcenter Integration for NX mode. These mappings are not available in native NX sessions.

Mappings can be either real mappings or test mappings. Real mappings are those that the user can view within the application. Test mappings are used to verify the accuracy of the mapping file before implementation.

Each mapping definition has a title and description that are used when displaying the attribute value in NX. There are also optional qualifiers that affect the behavior of the mapping, such as which application has write control and whether a default value is created for an NX part file attribute if none exists in Teamcenter.

For more information, see [Qualifier](#).

For more information about implementing attribute synchronization from Teamcenter Integration for NX, see the *Teamcenter Integration for NX Guide*, delivered with NX.

Additional definition for attributes when mapping

Some attributes may require a more specific attribute mapping definition. Starting in Teamcenter Integration for NX 6.0.3 attribute mappings are resolved at startup without the objects to which they apply.

If you have mappings that are not working, check the **syslog** for messages that are similar to the following:

```
Attribute (%s) is not unique. You must supply optional parameters in the mapping
```

definition (%s) to resolve without an object.

In these cases, you need to provide a more specific mapping definition. For example, the following mapping definition may no longer be valid:

```
Last_Modifying_User : last_mod_user /description="Last Modifying User"
```

To resolve this mapping problem, append **last_mod_user** with **user_id** and **user_name** as follows to specifically identify the attribute that is being mapped:

```
Last_Mod_User_ID : last_mod_user.user_id /description="Last Mod User ID"
Last_Mod_User_Name : last_mod_user.user_name /description="Last Mod User Name"
```

Default mappings

The default mappings between Teamcenter and NX part attributes are described in the following table:

UG part attribute	Description	Corresponding Teamcenter attribute
DB_PART_NO	Part number	Item ID
DB_PART_REV	Part revisions	Item revision ID
DB_PART_TYPE	Part type	Item type
DB_PART_NAME	Part name	Item name
DB_PART_DESC	Part description	Item description
DB_UNITS	Part unit of measure	Item unit of measure

Note You cannot delete the default attribute mappings.

Mapping Classification attributes to NX part attributes

Synchronization of NX part file attributes between Classification and Teamcenter Integration for NX uses the attribute mapping syntax described in [Classification to Teamcenter Integration for NX attribute mapping syntax](#) and [Teamcenter to Teamcenter Integration for NX attribute mapping syntax](#). Classification attribute synchronization supports all types of Classification attributes, including: integer, real, double, string, date, variable length array (VLA), and key LOV.

Attribute synchronization

Synchronization allows an attribute in one application to be updated automatically when a modification is made to the corresponding attribute in another application. The synchronization of attributes between Teamcenter Integration for NX, Teamcenter, and Classification occurs when a part file is imported or exported from Teamcenter Integration for NX. The Teamcenter and Classification attribute values are updated immediately; however, the NX part attribute values are not updated until the part is opened.

Before implementing attribute synchronization, read and understand the required attribute mapping syntax described in [Teamcenter to Teamcenter Integration for NX attribute mapping syntax](#).

The **Sync Part Attributes** option initiates attribute synchronization. You can suppress the display of this option by setting the **showUGSynAttrChecckbox** preference to **False**. This preference is located in the **userpreferences.properties** file, which can be accessed by unzipping the **portal.jar** file located in the **com.teamcenter.rac.tc.commands.userpreferences** directory.

Mapping to any master form for any item objects (the Item Master form) results in one-way synchronization from Teamcenter to NX.

Monodirectional and bidirectional synchronization

Synchronization can be monodirectional or bidirectional. Siemens PLM Software strongly recommends you do not use both methods in parallel on the same database.

To define the synchronization as being monodirectional, set the **UGII_UGMGR_ATTR_SYNC** Teamcenter Integration for NX environment variable, located in the **.ugii_env** file, to **Old**.

Bidirectional synchronization includes the following behavior:

- Mapping to any master form for any item object (the Item Master form) for customer-defined object types (such as **ENDITEM**) results in a one-way mapping from Teamcenter to NX. Siemens PLM Software recommends avoiding mapping to the Item Master form and mapping only in cases where the values do not require modification in NX.
- Using the qualifier **iman** always prevents bidirectional synchronization. Use the qualifier **both**, or no qualifier at all, to create bidirectional synchronization.

By default, attributes mapped to the properties of the Item or any form attached to items (not item revisions) are synchronized from Teamcenter to Teamcenter Integration for NX; but not in the reverse direction. These attributes are read-only in Teamcenter Integration for NX.

Set the **TC_NX_allow_attrsync_master_with_item** preference to **Yes** to permit mapping in the reverse direction (Teamcenter Integration for NX to Teamcenter). Use this setting to synchronize properties that are common to items but not to item revisions.

However, synchronizing properties on the item level may lead to loss of data. When multiple item revisions exist (**Rev/A**, **Rev/B**, **Rev/C**, and so on) and an item-level attribute is modified, the last item revision that is saved overwrites any attribute values that previously existed at the item level. For example, if a user working on **Rev/A** sets **Item-Level-attribute-user_data_1** to **32** and saves the part, the form stored at the item level for **user_data_1** displays **32**. If a user working with **Rev/C** changes **Item-Level-attribute-user_data_1** to **45** and saves the part, attribute synchronization overwrites the existing **32** with **45**. When any user re-opens **Rev/A** or reloads the database attributes, the value displayed is **45**, not the value previously stored with **Rev/A** (**32**).

Caution

Attributes mapped to the properties of the Item or any form attached to items and synchronized from Teamcenter Integration for NX to Teamcenter allows multiple item revision objects (**Rev/A**, **Rev/B**, **Rev/C**, and so on) to overwrite the values stored on the Item or any form at the item level. Only synchronize properties that are common to items but not to item revisions.

NULL value synchronization

By default, Teamcenter Integration for NX and Teamcenter prevent synchronization of null values. Thus, overwriting an existing value with a null value is not supported by default. This restriction is independent of synchronization direction (Teamcenter Integration for NX to Teamcenter, or Teamcenter to Teamcenter Integration for NX). It is also independent of the synchronization master settings stored in the synchronization mapping file.

This default restriction provides safety for your data. For example, if there is an NX part with an attribute containing a value (4); this value is important to NX users. The part has not been synchronized to the database. You create a mapping definition to synchronize the attribute. If null values were synchronized, the value for the attribute in this part file is removed; replaced by the null value in the database, which did not yet have an actual value for the attribute.

Null values can be synchronized by setting the attribute mapping qualifier to **allow_null_value**. Use this synchronization method with care.

Extending attribute mappings

Mappings of NX part attributes to Teamcenter and Classification are stored in the Teamcenter database and are extended by importing a modified mapping definition file into the database. The existing file is overwritten each time a new file is imported.

Note

Siemens PLM Software recommends that this task be performed when all users are logged off of Teamcenter Integration for NX. If this is not possible, any users who were logged on to Teamcenter Integration for NX at the time the new mapping file was imported must log off and log on again.

For more information, see the *Teamcenter Integration for NX Guide*.

1. Export the existing mapping to a text file, using the **export_attr_mappings** utility.
2. Edit the file to add, remove, or modify mappings according to the syntax described in *Classification to Teamcenter Integration for NX attribute mapping syntax* and *Teamcenter to Teamcenter Integration for NX attribute mapping syntax*.
3. Import the mapping file as a test file, using the **import_attr_mappings** utility. This allows you to verify the attribute mappings without affecting the existing mapping.
4. Verify the test mappings.

Set **TC_USE_TEST_ATTR_MAPPINGS** to any value. This causes Teamcenter to access the test mappings rather than the real mappings.

When running with test mappings, Teamcenter provides extra information in the **syslog** log file to aid you in repairing invalid mappings, as shown in the following example:

```
Ifail 60003 meaning "Unresolvable mapping definition "%1$s"":
failed to resolve mapping "DB_UNITS" :
"Item.uom_tag.symbol" for rev0 00000f02 of dataset 00000f05
```

While such information does not necessarily indicate an invalid mapping (in this example, the mapping failed simply because the item had no unit of measure), it can aid you in determining the validity of a file.

For more information about implementing attribute synchronization from Teamcenter Integration for NX, see the *Teamcenter Integration for NX Guide*, delivered with NX.

Teamcenter to Teamcenter Integration for NX attribute mapping syntax

There are three syntactical forms for the specification of a Teamcenter attribute mapping, as follows:

```
Part-Attr-Name : Path Teamcenter-Property [Qualifier*]
Part-Attr-Name : Abbreviated-Path Teamcenter-Property [Qualifier*]
Part-Attr-Name : Constant [Qualifier*]
```

The syntax definitions are as follows:

Part-Attr-Name

Specifies the name of a NX part attribute.

Path

Specifies a series of Teamcenter objects denoting a traversal path starting from the dataset object that represents the corresponding NX part file. Notice that the dataset object does not appear explicitly in the mapping; rather, it implicitly precedes the path.

It is further defined as *Path* = [*Teamcenter-Step* .]* where *Teamcenter-Step* is one of the following:

GRM (*relationship-type* [, *object-type*])

Specifies a GRM relation whose type is *relation-type*, whose primary object is the current object, and whose secondary object gives the next object. In the case that more than one relation of the given type is associated with the current object, the optional second argument *object-type* is used to identify the type of the referenced object.

NR (*named-reference-type* [, *form-type*])

Specifies a named reference of the current object. The particular reference is identified by the type of the referenced object *named-reference-type* (not the name of the reference).

SIGNOFF ([*level*,] *role*)

Specifies a completed signoff whose group members' roles have the name *role*. If the same role has been specified for assignees in more than one level of the process definition, the optional argument *level* further identifies a specific level.

Item revision	Specifies the item revision object that owns the current object. That is, the item revision to which the current object is associated via a GRM link.
Item	Specifies the item object corresponding to the current owning item revision object.
Job	Specifies the job in which the current object is included; or, if there is no current job, then the most recent job that the object was in (assuming the job has not been deleted).

Property

Specifies the name of a Teamcenter property, representing a named property of the current object in the object path.

Teamcenter-Property

Specifies the name of the Teamcenter property to be synchronized with the NX part attribute.

Qualifier

Specifies various controls or conditions for the particular attribute synchronization and can be any one of the following:

/write_once	Allows Teamcenter Integration for NX to write a value for the attribute to Teamcenter once, if the Classification attribute does not already have a value.
/master=cad	Allows Teamcenter to change the value and save the change, but when opened in NX the change made in Teamcenter is ignored and the previous value stored in NX is used.
/master=iman	Prevents NX from trying to change the value (this is equivalent to none).
/master=both	Allows NX to change the value and save the change. When reading the value, if the values in NX and Teamcenter differ, the Teamcenter value takes precedence unless it is empty.
/master=none	Prevents NX from changing the value.
/required	Forces NX users to supply a value for the attribute. The NX attribute pane displays a red asterisk next to the input field to signify that the attribute is required. If the attribute does not contain a value, NX displays a warning when the File→Save option is used. The part is not saved to the database until all attributes in the mapping file identified as required have been specified.
/description= note	Enables NX to display the content of <i>note</i> , which is a character string enclosed in single or double quotation marks, to show the user the significance of the attribute.
/freezable	Directs NX to ignore the Teamcenter attribute value, as stored in the Teamcenter database, but to use the value stored in the part file if the related Teamcenter object is frozen.

/allow_null_value Allows overwriting an existing value with a null value. For example, say there is a NX part with an attribute containing a value (4); this value is important to NX users. The part has not been synchronized to the database. You create a mapping definition to synchronize the attribute. If null values were synchronized, the value for the attribute is removed; replaced by the null value in the database, which did not yet have an actual value for the attribute.

This restriction is independent of synchronization direction (NX to Teamcenter or Teamcenter to NX).

Abbreviated-Path

Represents an abbreviation for a specific object traversal path in Teamcenter. These abbreviations apply *only* to the shipped **Item** item type, and its associated **ItemRevision**. These shorthand notations do not function for any custom item types. There are two possible abbreviations:

IM This is an abbreviation for **Item.GRM(IMAN_master_form, Item Master)**.

IRM This is an abbreviation for **ItemRevision.GRM(IMAN_master_form, Item Master)**.

Note Abbreviated paths can not be used when referencing custom types.

Constant

Represents a constant value specification that the NX part attribute assume. Its specification is **CONST** *value* where *value* is a character string enclosed in single or double quotation marks.

PREF ([scope :] value)

Produces the given preference value, optionally, within the given preference scope. The scope can be one of the following:

- **ENV**
- **USER**
- **ROLE**
- **GROUP**
- **SITE**

For more information about implementing attribute synchronization from Teamcenter Integration for NX, see the *Teamcenter Integration for NX Guide*, delivered with NX.

Comments and contexts symbols

Comments are introduced by the pound sign (#). Anything on the line after this symbol is not part of the mapping definition and is ignored.

Contexts begin with the left bracket ({) followed by a context specifier which states the context for the following mappings, either dataset type or item type context, as follows:

Dataset type = *dataset-type*

or

```
Item type = item-type
```

Contexts end with the right bracket (}).

Contexts can be nested. For example, you can define an item type-based context within a dataset type-based context. The mappings defined within this nested context are then only applied to datasets of the specified type that are associated with items of the specified type.

Mappings that are not defined in a specific context apply to the global context and thus apply to any dataset type and/or item type.

In any given context, you cannot define two mappings with the same title; however, you can do this in different contexts and the context-specific mapping overrides the less specific mapping.

For example:

```
CHECK_DATE : ItemRevision.Job.SIGNOFF(Checker).decision_date
/master=iman
/description="Check Date"
APPROVAL_DATE : ItemRevision.Job.SIGNOFF(Approver).decision_date
/master=iman
/description="Approval Date"
COPYRIGHT_CLAUSE : CONST("Copyright Unigraphics Solutions Ltd 2001")
OWNER_ID : owning_user.user_id /description="Owning user id"
PROJECT_ID : IM.project_id /master=iman /description="Project Identifier"
{Dataset type=UGMASTER
REV_NAME : ItemRevision.object_name /description="Revision Name"
{Item type=Item
REV_DESCRIPTION : ItemRevision.object_desc
/description="Revision Description"
}
}
```

Mapping examples

The following examples illustrate how to map attributes for both custom and shipped item types. The examples are based on the syntax described in [Teamcenter to Teamcenter Integration for NX attribute mapping syntax](#). It is important to note that when working with custom types, only the full path syntax method can be used. The abbreviated path does not support custom types. Using the abbreviated path syntax for a custom type always fails to synchronize the value. The system does not display an error message upon running the **import_attr_mappings** utility. No message regarding this condition displays upon opening the part or listing the attributes in NX or Teamcenter.

By default, attributes mapped to the properties of the Item or any form attached to items (not item revisions) are synchronized from Teamcenter to Teamcenter Integration for NX; but not in the reverse direction. These attributes are read-only in Teamcenter Integration for NX.

Set the **TC_NX_allow_attrsync_master_with_item** preference to **Yes** to permit mapping in the reverse direction (Teamcenter Integration for NX to Teamcenter). Use this setting to synchronize properties that are common to items but not to item revisions.

However, understand that synchronizing properties on the item level may lead to loss of data. When multiple item revisions exist (**Rev/A**, **Rev/B**, **Rev/C**) and an item-level attribute is modified, the last item revision that is saved overwrites

any attribute values that previously existed at the item level. For example, if a user working on **Rev/A** sets **Item-Level-attribute-user_data_1** to **32** and saves the part, the form stored at the item level for **user_data_1** displays **32**. If a user working with **Rev/C** changes **Item-Level-attribute-user_data_1** to **45** and saves the part, attribute synchronization overwrites the existing **32** with **45**. When any user re-opens **Rev/A** or reloads the database attributes, the value displayed is **45**, not the value previously stored with **Rev/A** (**32**).

Caution Attributes mapped to the properties of the Item or any form attached to items and synchronized from Teamcenter Integration for NX to Teamcenter allows multiple item revision objects (**Rev/A**, **Rev/B**, **Rev/C**, and so on) to overwrite the values stored on the Item or any form at the item level. Only synchronize properties that are common to items but not to item revisions.

Example 1

In the following example, **user_data_1** is mapped as **SUPPLIER_GRM** to the **ItemMaster** form using the full path name. The attribute **user_data_2** is mapped as **supplier_grm2** to the **ItemMaster** form, also using the full path name. Custom item type definitions (and their associated attributes) must be nested with the owning dataset.

```
{ Dataset type="UGMASTER"
  { Item type="Basket_Part"
    SUPPLIER_GRM : Item.GRM(TC_master_form,Basket_PartMaster).
    user_data_1 /master=cad/allow_null_value/description="Supplier GRM"
    SUPPLIER_GRM2 : Item.GRM(TC_master_form,Basket_Part Master).
    user_data_2 /master=cad/both/allow_null_value/description="supplier_grm2"
  }
}
```

Example 2

To map standard item types along with custom item types, use the following syntax. The syntax in the example below states that **UGMASTER** owns the **Item** item type, which owns the **user_data_3** attribute. **Item** also owns the **Basket_Part** item type, which owns the **user_data_2** and **user_data_1** attributes.

```
{ Dataset type="UGMASTER"
  Item type=Item
    Supplier_grm3 : IRM.user_data_3 /master=both /description="supplier_grm3"
  { Item type="Basket_Part"
    supplier_grm2 : Item.GRM(TC_master_form,Basket_Part Master).
    user_data_2 /master=cad /description="supplier_grm2"
    SUPPLIER_GRM : Item.GRM(TC_master_form,Basket_Part Master).
    user_data_1 /master=cad /description="Supplier GRM"
  }
}
```

Example 3

To see values synchronized from a **UGPART** specification onto a form, map the **UGPART** attributes to the **Named References**, **UGPART-ATTR** form. The system finds the form using the **Named References**, **UGPART-ATTR** reference, which

points to the **UGPART** type using the **IMAN_specification** relation. (This relation is defined automatically when the **UGPART** is created in either Teamcenter or NX.)

DATASET_MATERIAL : NR(UGPART-ATTR). material/master=both/allow_null_value/description="Material in a Dataset Form"

In this example, the attribute named **material** is an attribute shipped with Teamcenter and already associated with the **UGPART-ATTR** properties form. To view the other attributes shipped with Teamcenter and automatically associated with this form, go to the Business Modeler IDE application, expand the **Form** node in the Types tree, and select **UGPartAttr**. The attributes associated with this form display in the **Name** column. Alternatively, view the attributes in the Business Modeler IDE application under the POM class **UGPartAttrStorage**.

Classification to Teamcenter Integration for NX attribute mapping syntax

The syntax for the specification of an Classification attribute mapping is as follows:

Part-Attr-Name : *Path* **Target.ICS** (*Classification-Attr-ID*[:*VLA-POS*])
[*Qualifier* *]

The syntax definitions are as follows:

Part-Attr-Name

Name of an NX part attribute.

Path

Specifies a series of Teamcenter engineering process management objects denoting a traversal path starting from the dataset object that represents the corresponding NX part file. Notice that the dataset object does not appear explicitly in the mapping; rather, it implicitly precedes the path.

It is further defined as *Path* = [*Teamcenter-Engineering Process Management-Step* .]* where *Teamcenter-Engineering Process Management-Step* is one of the following:

Qualifier

Specifies various controls or conditions for the particular attribute synchronization and can be any one of the following:

/write-once	Teamcenter Integration for NX can write a value for the attribute to Teamcenter engineering process management once, if the Classification attribute does not already have a value.
/master=cad	Only NX is allowed to modify the attribute value.
/master=iman	Only Teamcenter engineering process management is allowed to modify the value for the attribute.
/master=both	Both Teamcenter Integration for NX and Teamcenter engineering process management can modify the attribute value.

- /master=none** Neither Teamcenter Integration for NX or Teamcenter engineering process management can modify the attribute value.
- /description= note** NX displays the content of *note*, which is a character string enclosed in single or double quotation marks, to show the user the significance of the attribute.
- /freezable** Directs NX to ignore the Teamcenter engineering process management attribute value, as stored in the Teamcenter engineering process management database, but to use the value stored in the part file if the related Teamcenter engineering process management object is frozen.
- /allow_null_value** Allows overwriting an existing value with a null value. For example, say there is an NX part with an attribute containing a value (4); this value is important to NX users. The part has not been synchronized to the database. You create a mapping definition to synchronize the attribute. If null values were synchronized, the value for the attribute in this part file would be removed; replaced by the null value in the database, which did not yet have an actual value for the attribute.

This restriction is independent of synchronization direction (NX to Teamcenter engineering process management, or Teamcenter engineering process management to NX).

Target

Defines the target object to be classified by Classification. The target can only be either an item or an item revision.

(Classification-Attr-ID)

Defines the attribute ID of the Classification attribute to be synchronized. The ID must be an integer with a value either greater than 999 or less than -999.

[:VLA-POS]

(Optional) Maps multiple NX attributes to a single Classification attribute that contains multiple fields (VLA attribute).

The syntax for the Classification attribute specification is basically an extension from the Teamcenter engineering process management attribute mapping syntax, as follows:

Part-Attr-Name : Path Teamcenter-Engineering Process Management-Property [Qualifier]*

For example:

```
AREA : ItemRevision.ICS(6128) /description="Area" /master=cad
base_length : ItemRevision.ICS(5898) /description="V-Block Base Length"
Vblock_name : Item.ICS(1306) /description="V-Block Name" /master=iman
Project_id : Item.ICS(-1234) /master=iman /description="Project Identifier"
```

For more information about implementing attribute synchronization from Teamcenter Integration for NX, see the *Teamcenter Integration for NX Guide*, delivered with NX.

Automatically classifying items and item revisions while creating parts

You can use the attribute mapping of the **ICS_CREATE** type to automatically classify the item or item revision of a newly created part. The value of this attribute in NX is the Classification class ID (or *class ID::subclass ID* in the case of a Classification subclass) of where you want the classification object created. The syntax for the **DEFINITION** argument is:

attribute name in NX : **Item.ICS_CREATE**(number)

or

attribute name in NX : **ItemRevision.ICS_CREATE**(number)

You can choose any attribute name for NX. You must use **ICS_CREATE(1)** for the first context object created, **ICS_CREATE(2)** for the second, and so on. For example:

ICO_Number_1 : **Item.ICS_CREATE**(1)

In NX, you see the part attribute *ICO_Number_1* where you can assign the appropriate classification class ID. In Teamcenter, a new classification object classifying the item containing the NX part appears in the specified class.

Troubleshooting

The following table lists common issues that may arise when mapping attributes and using attribute synchronization:

Symptom	Probable cause	Action
The following message is in the syslog log file: <pre>+++Failed to get global mapping at startup.</pre>	NX interprets that attribute mapping is enabled and Teamcenter does not.	Upgrade the Teamcenter database.
The following message is in the syslog log file: <pre>+++Failed to get global mapping at startup. Contents of the error store:- ERROR 60011 Only SAs may use the test mappings +++Failed to load mappings from database</pre>	A user without system administration privilege attempted to use the test mappings.	Change the setting of the TC_USE_TEST_ATTR_MAPPINGS preference, or run Teamcenter Integration for NX from an account with system administration privileges.

Symptom	Probable cause	Action
A form attribute always displays in NX as No Value Set , even after you have attempted to set it.	The system is attempting to use a file-based form.	Determine whether the relevant form type is file-based or class-based. Forms are class based if there is a name in the Form Definition Class box of the Form Type dialog box. Convert the form type, if necessary. For more information, see <i>Client Customization Programmer's Guide</i>.
The attribute always displays in NX as No-value-set , even after an apparently successful save. If the mapping is a test mapping, running in test mode generates unexpected and consistent messages in the syslog log file about failure to resolve the mapping.	An invalid mapping file component.	Verify the mapping file, as described in Mapping Teamcenter attributes to NX part attributes.
An attribute is not modifiable in NX though you think it should be.	By default, Teamcenter restricts the attributes that are potentially modifiable in NX and the mapping file is unable to override this restriction. Change the default behavior by setting the TC_NX_allow_attrsync_master_with_item site preference to Yes. This allows attributes mapped to the properties of the Item or any form to synchronize bidirectionally between Teamcenter Integration for NX and Teamcenter.	A modifiable attribute must be one of the following: • A property of a dataset • A property of any form that is a named reference of the specific dataset, and no other dataset Additionally, if the dataset is a master dataset, a modifiable attribute must be: • A property of the related item revision • A property of the related item revision master form Where a master dataset is one that has type UGMASTER and is attached to an item revision via a <i>specification</i> relationship. Alternatively, the attribute may not be modifiable due to access controls.

Symptom	Probable cause	Action
An attribute is not modifiable in NX though you think it should not be.	The /master= <i>value</i> qualifier may be missing in the mapping definition.	Modify the mapping definition file.

Appendix

A Glossary

Appendix

A Glossary

A

abstract smart folders

Pseudofolders that serve as filters for project data. See also *Pseudofolder*.

access control list (ACL)

Access Manager component that contains a list of accessors and, for each accessor, the privileges granted, denied, and not set.

Access Manager (AM)

Teamcenter application that enables the system administrator to grant users access to Teamcenter objects.

accessor

Access Manager component that grants or denies privileges to clusters of users who share certain common traits (for example, perform the same function or work on the same project).

ADA License

Teamcenter application that administers International Traffic in Arms Regulations (ITAR), intellectual property (IP), and exclude licenses. It provides enhanced control and new attributes for these licenses. ADA stands for Authorized Data Access.

AM

See *Access Manager (AM)*.

Appearance Configuration

Teamcenter application used to configure the use of appearance:

- Defining the appearance sets for which the site tracks appearances.
- Configuring the mapping of occurrence notes into appearance attributes.

attribute

Named storage variable that describes an object and is stored with the object. Users can search the database for objects using object attributes.

In an object, an attribute is a name/value pair; in the database, an attribute is a field.

Audit Manager

Teamcenter application that enables a system administrator to create audit definition objects, enable/disable audit trail logging, and control audit log access. Audit definition objects create audit logs that users can view from Teamcenter applications. Users can audit any Teamcenter object and event type with an audit definition.

Authorization

Teamcenter application that enables access control of administrative applications and utilities.

authorization rule

Allows you to control access to Teamcenter administrative applications and utilities by group. System-level rules are delivered as part of your Teamcenter installation, and you can create additional rules to support your business processes using the Authorization application.

B**Business Modeler IDE**

Teamcenter application that enables a customer to define data model objects such as business objects, classes, attributes, lists of values, and rules.

business object

Logical grouping of data attributes and properties that are manipulated at the enterprise level.

A Global Services business object allows users to query for and update information in multiple data sources.

C**change**

Temporary or permanent change to a configuration, design requirement, or documentation.

change notification

Notice to vendors, manufacturing, marketing, and other entities describing part or all of the implemented change to a single change order. A change notification can also provide information about the effectivity of the change order.

change order

Teamcenter object that references the problem, originating change requests, solutions, and related information. A change order contains sufficient information to incorporate the change.

change request

Teamcenter object that requests a change, reference the problem, proposed solutions, and related information. A change request contains sufficient information to allow a decision to proceed further with the change.

change review board

Group of users who review and authorize an issue. A review board provides a checkpoint for determining whether an issue should proceed.

change specialist

Issue Manager user who manages the processing of issues from their initiation until their final resolution.

class

Set of objects that share the same list of attributes but distinguishable by the value the attributes acquire for specific objects. For example, the Automobile class can be

defined by the brand, color, and price, but each car associated to the Automobile class has a different brand, color, and price combination.

classification

Process of categorizing objects according to similarity in characteristics. While the objects in one class share the same characteristics, the values of these characteristics may differ. For example, drill bits all share the length and diameter characteristics, but drill bit objects differ in length and diameter.

Classification Administration

Teamcenter application that enables a system administrator to define the groups, classes, subclasses, and views that form the Classification hierarchy.

client

Role played by a software component of a system when it requests particular services be performed on its behalf by another entity, a server. See also *server*.

client tier

Teamcenter architectural tier that comprises the Teamcenter clients, Teamcenter integrations with third-party applications, and the third-party applications associated with the integrations.

Closure Rule

Rule that controls the scope of the translation of data imported to and exported from Teamcenter. Closure rules specify how the data structure is traversed by specifying the relationships of interest and the actions to occur when these relationships are encountered.

Command Suppression

Teamcenter application that enables the system administrator to control the display of menu and toolbar commands within Teamcenter applications. Command Suppression allows suppressing the display of commands for an entire group hierarchy or a specific role within the hierarchy, for entire groups of users, and for users who are assigned a role within a group. It also allows suppressing the display of specific commands on a designated menu or the display of entire menus.

compound property rule

Business rule that defines run-time properties on an object that references and relates to a property on a source object. Compound property rules relate to the property associated with type components. Applied types are item, item revision, dataset, and form.

Content Management

Teamcenter application that enables users to manage SGML and XML information stored in a Teamcenter database and publish documents in multiple formats using SGML/XML editing and publishing tools. Content Management is used to collect and publish content, storing the content as components that can be reused in multiple publications, while links are maintained between components. The application includes processes for structuring publications so that they can be stored, published, and used for different purposes. Content Management also stores and manages graphics for use in publications, provides content revision control, and can incorporate Teamcenter workflows to facilitate the content development process.

D**data-driven smart folders**

Pseudofolders representing the leaf nodes in the project hierarchy. Data-driven smart folders display project data objects.

Data Integration Services Adapter

Teamcenter software component that integrates Teamcenter lifecycle visualization with both the thin client and the rich client.

data model

Abstract model that describes how data is represented and used.

dataset

Teamcenter workspace object used to manage data files created by other software applications. Each dataset can manage multiple operating system files, and each dataset references a dataset tool object and a dataset business object.

Dispatcher service configuration

Object that defines the visualization file format that a dataset file is translated into.

document management

Management of real-world documents through the use of item revision definition configuration (IRDC) objects to configure document management behavior. IRDC simplifies user interaction with information and reduces the need for users to understand the Teamcenter data model and relationship model. See also *item revision definition configuration (IRDC) objects*.

E**enterprise change notice (ECN)**

Provides a detailed work plan to resolve one or more enterprise change requests (ECRs) or a portion of one ECR. An ECN identifies all items and documents affected by a change and authorizes the actions that address a change.

enterprise change request (ECR)

Initiates proposals that recommend changes and capture business decisions associated with the change. Change requests state the cost estimates and benefits of a change and provide recommended actions. Change requests are sometimes responses to problem reports.

F**form**

Teamcenter workspace object used to display product information (properties) in a predefined template. Forms are often used to create an electronic facsimile of a hardcopy form in Teamcenter. See also *master form*.

G**group (Classification)**

Container used to organize large sets of related classes in Classification Administration. Groups reside at the highest level in the classification hierarchy and can be nested to include other groups.

group (Organization)

Organizational grouping of users at a site. Users can belong to multiple groups and must be assigned to a default group.

H**hierarchy**

Structure in which each node can have only one parent but possibly multiple siblings and children.

I**item**

Workspace object generally used to represent a product, part, or component. Items can contain other workspace objects including other items and object folders.

item revision

Workspace object generally used to manage revisions to items.

item revision definition configuration (IRDC) objects

Objects that define how item revisions are handled at specific times in the life cycle, such as at item creation, checkin, checkout, save as, and revise. IRDC objects are defined in the Teamcenter Business Modeler IDE application. Any item can be associated with an IRDC. For example, IRDC objects typically support business processes by managing document checkin and checkout and by specifying rendering of visualization or markup versions from source datasets.

item revision relation

Description of an association between a Teamcenter item revision and a piece of information that describes or is related to the item revision.

J**JT cache**

Data Integration Services Adapter cache of JT files managed by the JT cache service and accessible to Teamcenter lifecycle visualization users. See also *world access cache* and *limited access cache*.

JT cache populator

Stand-alone Data Integration Services Adapter utility that prepopulates the JT cache with JT part and assembly files.

L**Lifecycle Visualization**

Visualization components available in the rich client, the thin client, and as a stand-alone application. A scalable solution for the viewing and analysis of 2D and 3D product data, Lifecycle Visualization is available in a number of configurations to meet the requirements of different organizations.

limited access cache

Data Integration Services Adapter cache of JT files that are accessible only to privileged users. The limited access cache is managed by the JT cache service.

Permission checks are performed when a user accesses a file from the limited access cache. Compare with *world access cache*.

List of Values (LOV)

Pick list of values accessed by end users from a menu at the end of a data field. LOVs ensure consistent data entries in the rich client.

M**master form**

Teamcenter workspace object used to display product information (properties) in a predefined template. Master forms are used to display product information in a standardized format.

My Teamcenter

Teamcenter rich client application that is the main access point for managing product information. My Teamcenter provides the functionality for creating objects in the Teamcenter database, querying the database for objects, checking in and checking out objects, and managing tasks. Users can also open objects, automatically launching the related application.

Each user has a personal My Teamcenter window that displays product information as graphical objects. Although users share product information across the enterprise, they organize this information individually in personal workspaces.

N**NX**

Siemens PLM Software's next-generation digital product development system that helps companies transform the product life cycle. It provides the complete life cycle of development processes in product design, manufacturing, and simulation.

NX Integration

Integration between Teamcenter and NX. NX Integration users have full access to the Teamcenter user interface from NX, and they can also access NX from the Teamcenter user interface.

Teamcenter Integration for NX and NX Integration have the identical user interface in NX. The difference between the two products is the level of Teamcenter functionality available. Teamcenter Integration for NX excludes certain Teamcenter functionality, such as workflow and product structure editing.

O**Organization**

Teamcenter application that enables a system administrator to create and manage critical Teamcenter files and database entries. It is the point of access for creating a company's virtual organization and for performing system administration activities such as volume creation, maintenance, and site administration. Organization enables creation and management of person, user, role, and group definitions; definition of the hierarchical structure of the Teamcenter organization; management of data volumes; and establishment and maintenance of Teamcenter sites.

organization tree

Graphic display of the Teamcenter organization structure. Expanding and collapsing branches of the tree enables viewing and managing the organizational structure. Selecting a node starts Organization wizards used to create groups, subgroups, roles, users, and persons.

P**person**

Definition containing real-world information about each Teamcenter user, such as name, address, and telephone number. Person definitions are stored as simple text strings so that they can be easily changed and updated. The name must be unique.

PLM XML

Siemens PLM Software format for facilitating product life cycle interoperability using XML. PLM XML is open and based on standard W3C XML schemas. Representing a variety of product data both explicitly and via references, PLM XML provides a lightweight, extensible, and flexible mechanism for transporting high-content product data over the Internet.

preference

Configuration variable stored in a Teamcenter database and read when a Teamcenter session is initiated. Preferences allow administrators and users to configure many aspects of a session, such as user logon names and the columns displayed by default in a properties table.

project

Basis for identifying a group of objects available to multiple organizations, such as project teams, development teams, suppliers, and customers for a particular piece of work.

Project

Teamcenter application used to define projects and assign project team members, project team administrators, and privileged team members.

propagation

Process of transferring characteristics of one object to another object.

properties

Keys and values that specify the configuration settings for an application in the Teamcenter rich client.

property rule

Business rule that allows an administrator to control access to and the behavior of object properties.

Pseudofolder

Special container in Teamcenter that stores and displays item and item revision relations in My Teamcenter.

Q**Query Builder**

Teamcenter application that enables a system administrator to create and maintain customized searches for objects in the Teamcenter databases, both local and remote.

Saved queries are subject to standard object protection and can be accessed by users through the search feature in My Teamcenter.

R

Report Generator

Teamcenter manufacturing process management application that provides a format for producing reports about information in Teamcenter manufacturing process management.

Rich Client

Java-based user interface to Teamcenter installed on user workstations. The rich client accesses Teamcenter databases using a remote or local server.

role

Function-oriented cluster of users that models skills and/or responsibilities. The same roles are typically found in many groups. In Access Manager, role is an accessor used to grant privileges to all users with the same skills and/or responsibilities regardless of project.

role in group

Specific role in a specific group. In Access Manager, role in group is an accessor used to grant privileges to all users with the same skills and/or responsibilities in the same group.

root

Starting point of a hierarchy. Hierarchies are usually displayed as hanging trees with the root of the structure at the top and the leaves at the bottom.

S

Security Services

Services that eliminate prompts for logon credentials when users switch Teamcenter products within a user session. Authentication is performed by an external identity service provider, such as lightweight directory access protocol (LDAP), instead of the Teamcenter product. At a site that deploys multiple Teamcenter products and Security Services, users log on once to access all participating Teamcenter products.

server

System software component that performs a specifically defined set of software services on behalf of one or more clients. In a typical Teamcenter installation, servers are centralized on dedicated hosts that support a large number of clients. Clients are distributed on hosts connected to the servers via various networking techniques. See also *client*.

Setup Wizard

Teamcenter application that facilitates postinstallation setup of a Teamcenter database using an input file to populate the information required to create the basic components of the Teamcenter organization. Using Setup Wizard, a system administrator can create user/person definitions, assign a group/role to a user, and optionally define a default volume for assigned groups.

single sign-on

Mechanism that allows a user to log on once to multiple services. Following a single sign-on to Teamcenter, the user is connected to multiple services without logging on to each service.

site

Individual installation of Teamcenter comprising a single Teamcenter database, all users accessing that database, and additional resources such as hardware, networking capabilities, and third-party software applications (tools) required to implement Teamcenter at that site.

site preference

Teamcenter preference that applies to the entire site.

smart folder

Pseudofolders configured in a hierarchical structure used to apply hierarchical levels of filtering to project data. See also *Pseudofolder*.

SQL

See *Structured Query Language*.

Structured Query Language

ANSI standard command and embedded language for manipulating data in a relational database.

subscription

Combination of a workspace object and an event for which a Teamcenter user requests notification of occurrence. Teamcenter notifies a subscribed user when the event occurs in association with the object. Users can subscribe to objects from Teamcenter applications, such as My Teamcenter and Structure Manager.

Subscription Monitor

Teamcenter application that enables a system administrator to query subscriptions created in the database, to monitor and delete subscription events and actions, and to generate statistical reports in either text or bar chart format regarding subscriptions, events, and actions.

system administrator

Teamcenter user who is a member of the system administration group.

T**Teamcenter Integration for NX**

NX-based entry-level set of Teamcenter data management functions, such as vaulting, checking in, checking out, revision management, attribute synchronization, and searching CAD data. Teamcenter Integration for NX manages NX files and their data without exposing the user to full Teamcenter functionality. Teamcenter Integration for NX users have only limited access to the Teamcenter user interface.

Teamcenter Integration for NX and NX Integration have the identical user interface in NX. The difference between the two products is the level of Teamcenter functionality available. Teamcenter Integration for NX excludes certain Teamcenter functionality, such as workflow and product structure editing.

Thin Client

Teamcenter user interface that provides a streamlined browser-based view of product information stored in a Teamcenter database. The thin client is configured in the Web tier, which creates and serves its Web pages to the client.

transfer mode

Objects composed of rules that configure PLM XML import and export operations. Transfer mode allows users to import and export data with little knowledge other than the name of the transfer mode object.

U**user**

Definition that is the mechanism by which Teamcenter identifies and interacts with each user. User definitions contain a name (derived from the person definition), user ID, operating system name, and password.

V**value**

Content of a field or variable. It can refer to alphabetic, numeric, or alphanumeric data.

W**Web Browser**

Teamcenter application that provides access to Internet Web pages from within the rich client framework. The Web browser is a rich client window that acts as a Web browser, enabling you to navigate and view Web pages within the rich client rather than switching to a separate Web browser. The Web browser also provides the ability to access MIME (Multipurpose Internet Mail Extension) file types and to view files created in other applications, such as Microsoft Word and Excel, through the Web browser.

work context

Object containing user, group, role, and project requirements for a particular task. Work contexts streamline the definition of the correct user settings required to perform tasks. For example, a user typically assigned three types of tasks can create three work contexts reflecting the required group/role settings. Rather than logging out and logging back in with the required group/role settings, right-clicking a task displays the user's defined work contexts. Selecting the appropriate work context automatically changes the user's group/role settings. Work contexts can also define the current project, if project functionality is used at the site.

workflow

Automation of the concept that all work flows through one or more business processes to accomplish an objective. Using workflow, documents, information, and tasks are passed between participants during the completion of a particular process.

Workflow Designer

Teamcenter application that enables administrators to graphically design workflow process templates, incorporating company business practices and procedures into the templates. Teamcenter users initiate workflow processes using these templates.

world access cache

Data Integration Services Adapter cache of JT files that are accessible to all users. The world access cache is managed by the JT cache service. No permission checks are performed when a user accesses a file from the world access cache. Compare with *limited access cache*.

Index

Numerics/Symbols

2D markup
Automatic markup 16-20

A

Access Manager 5-1
Access rules 5-1
Accessors 5-1
Action Parameter 8-4
actions.properties file 3-5
Adding applications 3-6
Adding new types for indexing 7-16
Advanced search 7-1
Application access, granting 4-4
Applications tree pane 3-2
Associating export templates with business
objects 14-13
Attribute mapping 17-1
Authorization interface 4-2
Authorization rules
Exporting 4-5
Group hierarchy behavior 4-3
Importing 4-5
System level 4-3
Automatic markup 16-20

B

Basic concepts 4-2
Batch indexing full-text search
objects 7-14
Body contents, specification template .. 14-5
build_fts_index utility 7-9
Buttons
Hide 3-3
Show 3-3

C

Cell formatting in Excel export
templates 14-14
Checking status of indexed objects 7-15
Choice Parameter 8-4
Command nodes 3-3

Command Suppression

Customizing 3-6
Display commands 3-4
Display commands for roles 3-6
Interface 3-2
Suppress commands 3-3
Suppress commands for roles 3-5
Command tree pane 3-3
Configure Web Browser 15-1
Configuring document management ... 13-1
Configuring export templates 14-1
Configuring search 7-1
Converting legacy preference data 6-1
Copying
Object templates 14-12
Specification templates 14-7
Copying Excel export templates 14-21
CounterKey Parameter 8-8
Creating
Object templates 14-10
Specification templates 14-6
Creating Excel export templates 14-20
Custom notes
Configuring the behavior of custom
notes 12-2

D

Data security 5-1
Defining queries and reports 9-1
Deleting
Object templates 14-12
Specification templates 14-8
Deleting Excel export templates 14-21
Dispatcher service configurations 13-4
Document management 13-1
Creating item revision definition
configurations 13-3
Dispatcher service configurations ... 13-4
Templates 13-1
Document management templates 13-1
Attaching datasets to 13-2
Creating in rich client 13-1
Creating in thin client 13-2
Releasing 13-2–13-3

Index

Dynamic structure, specification
templates 14-6

E

Enabling search 7-1

Enabling Smart Codes 8-3

Excel

Export

Property keywords in

templates 14-17

Excel export rule table 14-17

Level 14-17

Relation 14-17

Transfer Mode 14-17

Type 14-17

Excel export template guidelines 14-16

Excel export template rule table 14-15

Key fields 14-15, 14-17

Excel export template rules 14-15

Excel export templates 14-1, 14-14

Cell formatting in 14-14

Copying 14-21

Creating 14-20

Deleting 14-21

Guidelines 14-16

Importing 14-21

Modifying 14-20

Multiple sheets in 14-14

Packing

Apply to template object 14-20

Definition 14-18

Object packing in export files .. 14-18

Rule processing 14-18

Rules 14-15

Using transfer modes 14-17

Excel templates

Packing data 14-19

ExcelTemplate items 14-14

ExcelTemplateRevision item

revisions 14-14

Export

Excel

Property keywords in

templates 14-17

Word

Property keyword syntax in

templates 14-13

Specification templates 14-2

Export templates 14-1

Associating with business objects .. 14-13

Export dynamic structure rule 14-13

Export static structure rule 14-13

Microsoft Excel 14-1, 14-14

Microsoft Word 14-1

Packing, Excel export

Apply to template object 14-20

Definition 14-18

Object packing in export files .. 14-18

Exporting authorization rules 4-5

Extends Parameter 8-11

F

FieldSet Parameter 8-9

File Name Parameter 8-7

Files

actions.properties 3-5

portal.properties 3-6

portal_user.properties 3-6

FirstValue Parameter 8-10

Footers, specification templates 14-4

ForType Parameter 8-11

FTS indexing 7-13

FTS indexing methods 7-14

Full-text search 7-8

Full-text search indexes 7-15

Updating 7-15

Full-text search indexing 7-13

FullText datasets 14-2, 14-8

G

Generate Parameter 8-7

Getting started 4-1

Granting application access 4-4

Granting utilities access 4-4

Group hierarchy behavior 4-3

H

Headers, specification templates 14-4

Hide button 3-3

I

ID Parameter 8-4

Importing

Object templates 14-11

Specification templates 14-7

Importing authorization rules 4-5

Importing Excel export templates 14-21

Indexing methods for full-text search

objects 7-14

Indexing methods for full-text search

objects 7-14

Indexing new types 7-16

Indexing search objects 7-13

- Instant messaging 11-1
- Interface 15-1
- Item revision definition
 - configuration 14-13
- Item revision definition
 - configurations 13-3
- K**
- Keys, TC_CS.APPLICATIONS 3-6
- Keywords
 - Properties
 - Excel export templates 14-17
- Keywords, properties
 - {%object_name} 14-13
 - Word export templates
 - Syntax 14-13
- L**
- Label Parameter 8-4
- LeadingChar Parameter 8-9
- Legacy preference data 6-1
- Length Parameter 8-9
- M**
- Managing data security 5-1
- Manually indexing full-text search
 - objects 7-14
- Mapping Classification attributes 17-3
 - attribute synchronization 17-3
- Mapping NX part attributes 17-9
 - Additional attribute definitions 17-2
 - extending attribute mappings 17-5
 - mapping syntax 17-6
 - TC to NX 17-1
 - TC to NX defaults 17-3
- Menu nodes 3-3
- Modifying
 - Object templates 14-11
 - Specification templates 14-7
- Modifying Excel export templates 14-20
- MSExcels datasets 14-14
- Multiple sheets in Excel export
 - templates 14-14
- N**
- Nodes
 - Command 3-3
 - Menu 3-3
 - Submenu 3-3
- NX part additional attribute
 - definitions 17-2
- NX part attributes 17-1
- O**
- Object export templates 14-1
- Object template
 - With rules, example 14-10
 - Without rules, example 14-9
- Object templates 14-8
 - Content rules 14-12
 - Copying 14-12
 - Creating 14-10
 - Deleting 14-12
 - Importing 14-11
 - Modifying 14-11
- Object templates, Word export
 - Property keywords
 - Syntax 14-13
 - {%object_name}, property keyword, object
 - templates for Word export 14-13
- ObjectTemplate items 14-8
- ObjectTemplateRevision item
 - revisions 14-8
- Organization tree pane 3-2
- P**
- Packing data, Excel export
 - templates 14-19
- Packing, Excel export templates
 - Apply to template object 14-20
 - Definition 14-18
 - Object packing in export files 14-18
- Panes
 - Applications tree 3-2
 - Command tree 3-3
 - Organization tree 3-2
- Perspectives
 - Web Browser 15-1
- portal.properties file 3-6
- portal_user.properties file 3-6
- Preferences
 - TC_fts_indexed_types 7-17
 - TC_fts_real_time_indexing 7-17
- Preferences, WEB_menu_entry_
 - suppressions 3-4
- Prefix Parameter 8-9
- Property keywords
 - Excel export templates 14-17
 - Word export templates
 - Syntax 14-13
- Q**
- Queries and reports 9-1

Index

Query Builder 9-1
Quick search 7-1

R

Releasing document management
 templates 13-2–13-3
Report Builder 9-1
Repository Parameter 8-6
Restriction Parameter 8-7
ReuseNumber Parameter 8-8

S

Search 7-1
 Status of indexed objects 7-15
 Synonym 7-17
Search schema 16-7
SecondValue Parameter 8-10
Security Services
 TC_SSO_APP_ID environment
 variable 2-1
 TC_SSO_CHANGE_PASSWORD_PAGE
 environment variable 2-1
 TC_SSO_LOGIN_PAGE environment
 variable 2-1
 TC_SSO_LOGIN_URL environment
 variable 2-1
 TC_SSO_SERVICE environment
 variable 2-1
Select Parameter 8-11
Show button 3-3
Simple search 7-1
Smart Codes
 Configuration File 8-3
 Configuration Parameters 8-4
 Action 8-4
 Choice 8-4
 CounterKey 8-8
 Extends 8-11
 FieldSet 8-9
 File name 8-7
 FirstValue 8-10
 ForType 8-11
 Generate 8-7
 ID 8-4
 Label 8-4
 LeadingChar 8-9
 Length 8-9
 Prefix 8-9
 Repository 8-6
 Restriction 8-7
 ReuseNumber 8-8

 SecondValue 8-10
 Select 8-11
 StepFill 8-10
 StepLength 8-10
 Suffix 8-9
 Type 8-5
 Configuring 8-1
 Enabling 8-3
 Example 8-1
 Overview 8-1
Specification export templates 14-1
Specification template layout 14-3
Specification templates
 Body contents 14-5
 Content rules 14-12
 Copying 14-7
 Creating 14-6
 Deleting 14-8
 Dynamic structure 14-6
 Footers 14-4
 Headers 14-4
 Importing 14-7
 Layout 14-3
 Modifying 14-7
 Static structure 14-5
 Styles 14-4
 Table of contents 14-4
 Title page 14-3
Specification templates, Word export ... 14-2
 Property keywords
 Syntax 14-13
SpecTemplate items 14-2
SpecTemplateRevision item revisions ... 14-2
Standard notes 12-2
 Set the delimiter 12-1
Static structure, specification
 templates 14-5
StepFill Parameter 8-10
StepLength Parameter 8-10
Styles, specification templates 14-4
Submenu nodes 3-3
Suffix Parameter 8-9
Supported applications 4-3
Supported utilities 4-4
Synonym search 7-17
 Configuring 7-17
System-level rules 4-3

T

Table of contents, specification
 templates 14-4
TC_CS.APPLICATIONS key 3-6

TC_fts_indexed_types preference 7-17
 TC_fts_real_time_indexing
 preference 7-17
 tcx_sc_config.txt file 8-3
 Teamcenter Integration for NX
 Attribute mapping 17-1
 Teamcenter Integration for NX
 Configuration 17-1
 Teamcenter search 7-1
 Templates
 Excel export
 Packing, apply to template
 object 14-20
 Packing, definition 14-18
 Packing, objects in Excel export
 files 14-18
 Word export
 Specification templates 14-2
 Title page, specification template 14-3
 Type Parameter 8-5
 Types
 Added for indexing 7-16

U

Updating search indexes 7-15

Utilities access, granting 4-4
 Utility
 build_fts_index 7-9

V

Visualization
 Custom dataset types 16-20
 Multiple named references in
 datasets 16-20
 Preferences 16-6
 Search schema 16-7
 Search schemas 16-7

W

Web Browser 15-1
 WEB_menu_entry_suppressions
 preference 3-4
 Word
 Export
 Property keyword syntax in
 templates 14-13
 Specification templates 14-2